

江南大学

硕 士 学 位 论 文

题 目: 基于 FPGA 的神经网络加速器的规模可伸缩性研究

英文并列题目: Research of Scalability on
FPGA-based Neural Network Accelerator

研 究 生: 陈 辰

专 业: 计算机科学与技术

研 究 方 向: 计算机系统结构

导 师: 柴志雷

指导小组成员: _____

学位授予日期: 2019 年 6 月

答辩委员会主席: 李光辉

江 南 大 学

地址: 无锡市蠡湖大道 1800 号

二〇一九 年 五 月

摘要

近年来,以深度神经网络为代表的深度学习算法在许多计算机视觉任务上取得了巨大突破,如图像分类、目标检测、画质增强等应用。由于 FPGA 低功耗、低延时的特性,使其适用于对功耗有严格限制的小批量流式应用场合。此外,通过配置改变 FPGA 硬件逻辑,对具体应用定制硬件,非常适合深度学习这种日新月异、不断改变场合。因此, FPGA 成为了一种极具潜力的选择。

目前,大规模使用 FPGA 加速深度学习的制约主要有两点:

(1) 开发效率低。然而,随着高层次综合技术不断成熟,已经可以使用 C、C++ 等高级语言进行 FPGA 的算法描述,开发效率低的问题正被逐渐缓解。

(2) 规模可伸缩性差。受制于芯片资源,对于复杂算法往往需要升级芯片或进行多片划分,没有办法灵活地部署到不同硬件上去。

因此本文立足于基于 FPGA 的神经网络加速器的规模可伸缩性研究。通过研究现有的基于 FPGA 的神经网络加速器的特点和 FPGA 本身特有的可重构特性,探索如何将针对特定神经网络算法的加速器灵活地部署到不同规模的 FPGA 上去。

研究并分析了基于 SIMD 架构的卷积神经网络加速器的硬件设计参数以及数据复用模式对规模可伸缩性的影响。设计并实现了一种基于 SIMD 架构二维展开的 CNN 加速器,并采用了参数重排序、多通道数据传输等优化策略来减少访存与传输时延。改进了加速器架构中的输入和输出模块,有效提高了总线带宽的利用率,同时降低了传输与访存时延。以目前在业界被广泛应用的 YOLOv2 目标检测算法为例,叙述将 CNN 模型映射到 FPGA 上的完整流程。同时,对加速器的性能和所需资源进行深入分析和建模,将实际传输延时考虑在内,极大地缩小了加速器理论时延与实际时延的误差。对由不同硬件设计参数产生的不同规模的 CNN 加速器进行评估与对比。实验结果表明,在 Zedboard 上获得了 30.15GOP/s 的性能,与 Xeon E5-2620 v4 CPU 相比,能效是其 120.4 倍、性能是其 7.3 倍;与双核 ARM-A9 CPU 相比,能效是其 86 倍,性能是其 112.9 倍。并且在性能上也超过之前的工作。相关代码已在 github 开源。

关键词: 卷积神经网络; FPGA; 硬件加速器; 高层次综合

Abstract

In recent years, deep learning algorithms represented by deep neural networks have made great breakthroughs in many computer vision tasks, such as image classification, object detection, and image quality enhancement. Due to its low power consumption and latency, the FPGA is suitable for small batch streaming applications where power consumption is severely limited. In addition, by configuring the FPGA hardware logic to customize the hardware for specific applications, it is ideal for deep learning. As a result, FPGAs have become a highly promising option. At present, there are two main restrictions on the large-scale use of FPGAs to accelerate deep learning:

(1) Low development efficiency. However, as high-level integrated technologies continue to mature, FPGAs can be described in high-level languages such as C and C++, and the problem of low development efficiency is being gradually alleviated.

(2) Poor scale scalability. Due to the limitation of chip resources, it is often necessary to upgrade chips or multi-chip partitioning for complex algorithms, and there is no way to flexibly deploy to different hardware.

Therefore, this paper is based on the scale scalability research of FPGA-based neural network accelerators. By studying the characteristics of existing FPGA-based neural network accelerators and the reconfigurable features of FPGAs, we explore how to flexibly deploy accelerators for specific neural network algorithms to FPGAs of different sizes.

The hardware design parameters of the convolutional neural network accelerator based on SIMD architecture and the effect of data multiplexing mode on scale scalability are studied and analyzed. An FPGA-based SIMD convolutional neural network accelerator with two-dimensional expansion is designed and implemented. Optimization strategies such as parameter rearrangement, ping-pong buffering, and multi-channel data transmission are used to reduce the memory access and transmission delay. Improved input and output modules in the accelerator, effectively improving bandwidth utilization while reducing transmission and access latency. Taking the YOLOv2 target detection algorithm widely used in the industry as an example, the complete process of mapping the CNN model to the FPGA is described. At the same time, the performance and required resources of the accelerator are deeply analyzed and modeled, taking into account the actual transmission delay, which greatly reduces the error between the theoretical and actual delay. Evaluate and compare different scales of CNN accelerators generated by different hardware design parameters. Experimental results show that an average 30.15GOPS performance achieved when the Zedboard with Zynq-7020 is used. It is 7.3x and 120.4x of performance and energy gains respectively when comparing with the software Darknet on an 8-core Xeon server, and 86x and 112.9x over the software version on the dual-core ARM cortex-A9 on Zynq. And in performance, it also exceeded previous work. The relevant code has been open sourced on github.

Keywords: Convolutional neural network; FPGA; hardware accelerator; High-level synthesis

目 录

摘 要	I
Abstract	II
第一章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.3 研究内容及组织结构安排	4
1.3.1 研究内容	4
1.3.2 组织结构安排	5
第二章 神经网络加速器与 FPGA 可重构技术基础	6
2.1 神经网络概述	6
2.1.1 人工神经网络与卷积神经网络介绍	6
2.1.2 YOLOv2 相关概念介绍	7
2.2 FPGA 及动态可重构技术概述	11
2.2.1 FPGA 介绍	11
2.2.2 FPGA 动态可重构技术介绍	12
2.2.3 Zynq 异构平台介绍	13
2.3 神经网络加速器相关概念介绍	14
2.3.1 循环展开	14
2.3.2 循环分块	14
2.3.3 循环交换	14
2.3.4 突发传输	15
2.3.5 Roofline 模型	16
2.4 本章小结	16
第三章 基于 SIMD 架构的 CNN 加速器规模可伸缩性研究	18
3.1 循环展开	19
3.2 循环分块	21
3.3 循环交换	23
3.4 加速器的设计空间探索与评估	27
3.5 本章小结	29
第四章 基于 FPGA 的 SIMD 二维展开的 CNN 加速器设计与评估	30
4.1 从 CNN 模型到 FPGA 加速器构建	30
4.1.1 YOLOv2 任务划分	30
4.1.2 动态定点 16 位量化	31
4.2 基于 SIMD 架构的 CNN 加速器的设计与优化	32
4.2.1 输入输出模块	33
4.2.2 卷积模块	36

4.2.3 池化模块.....	37
4.2.4 重排序模块.....	38
4.3 优化策略	39
4.3.1 参数重排序.....	39
4.3.2 乒乓缓冲.....	40
4.3.3 多通道数据传输.....	41
4.4 实验评估	42
4.4.1 软硬件实验环境.....	42
4.4.2 加速器的规模可伸缩性评估.....	42
4.4.3 硬件设计参数与异构系统.....	43
4.4.4 资源耗费评估.....	45
4.4.5 性能评估.....	45
4.5 本章小结	49
第五章 基于 FPGA 动态可重构的 CNN 加速器设计与评估	50
5.1 可重构加速器架构研究	50
5.2 基于 FPGA 的动态可重构 CNN 加速器设计.....	51
5.2.1 初始子图划分.....	51
5.2.2 架构设计.....	52
5.2.3 子图再划分.....	54
5.3 实验评估	55
5.3.1 软硬件实验环境.....	55
5.3.2 加速器的规模可伸缩性评估.....	55
5.3.3 资源耗费评估.....	56
5.3.4 性能评估.....	57
5.4 本章小结	59
主要结论与展望.....	60
主要结论	60
展望	61
致 谢.....	62
参考文献.....	63
附 录: 作者在攻读硕士学位期间发表的论文、成果与科研项目	67

第一章 绪论

1.1 研究背景及意义

近年来,以深度神经网络(Deep Neural Network, DNN)为代表的深度学习算法在许多计算机视觉任务上取得了巨大突破,如图像分类、目标检测、画质增强等^[1-4]。DNN 不像传统机器学习算法使用手工制作的特征,而是利用高强度的计算能力从更大的训练集中学习得到的特征,从而获得接近甚至优于人类感知的识别率。

然而,随着识别率的提高,深度学习算法的计算复杂度和内存需求也急剧增加,当前的通用处理器无法满足其计算需求。主流的解决方法是采用图形处理器(Graphics Processing Unit, GPU)、专用集成电路(Application Specific Integrated Circuit, ASIC)芯片或现场可编程门阵列(Field-Programmable Gate Array, FPGA)来提升计算性能^[5-7]。

GPU 采用单指令流多数据流(Single Instruction Multiple Data, SIMD)架构,并行度高,想要充分发挥计算性能,需要较大批量的数据,不适用于小批量流式应用,这对于一些低延时的场合是难以接受的^{[8][9]}。其次,由于并行度高导致的高功耗,很难应用于对功耗有严格限制的场合。

ASIC 对于具体应用可以获得最佳性能和能效,但是研发周期长,需要对市场有长久的预见性,不适合深度学习这种正在快速演变的领域^[10]。

FPGA 作为一种高性能、低功耗的可编程芯片,可以使用硬件描述语言(Hardware Description Languages, HDL)来设计数字逻辑电路,以形成对应具体算法的加速电路结构。

深度学习一般分为两个阶段:训练和推断。

训练阶段,由于深度学习的大部分计算都可以转换成矩阵相乘或者矩阵向量乘法,而 GPU 在矩阵乘法上是非常高效的,并且已经建立了很好的生态和开发环境,所以目前主流采用 GPU。

推断阶段,分为数据中心和嵌入式两种场合。在数据中心场合,主要应用于机器翻译,图像识别,搜索引擎等低延时、及时响应的服务。在嵌入式场合,考虑到能源受限,一般只接受低功耗、低延时的应用。

与 GPU 相比, FPGA 低功耗、低延时,适用于小批量流式应用^[11],很适合推断阶段。与 ASIC 相比, FPGA 可以通过配置重新改变硬件结构,对具体应用定制硬件,适用于深度学习这种日新月异、不断改变的场景。所以, FPGA 是一种极具潜力的选择,也有一些数据中心已经采用 FPGA 进行推断^[12-13]。

目前,大规模使用 FPGA 加速深度学习的制约主要有两点:

(1) 开发效率低。由于 FPGA 的编程一般使用 Verilog 或者 VHDL 这两种硬件描述语言。与传统的软件语言的最大不同之处在于: HDL 描述的是硬件实现电路;而软件语言描述的是指令流,不需要去理解硬件的具体实现。然而,随着高层次综合(High-level synthesis, HLS)技术不断成熟,已经可以使用 C、C++等高级语言进行 FPGA 的算法描述^{[14][15]},开发效率低的问题正被逐渐缓解。

(2) 规模可伸缩性差。受制于芯片资源,对于复杂算法往往需要升级芯片或进行多片划分,没有办法灵活地部署到不同硬件上。

因此本文重点研究基于 FPGA 的神经网络加速器的规模可伸缩性。通过研究现有的基于 FPGA 的神经网络加速器的特点和 FPGA 本身特有的可重构特性,探索如何将针对特定神经网络算法的加速器灵活地部署到不同规模的 FPGA 上去。

1.2 国内外研究现状

当前基于 FPGA 的神经网络加速器的相关研究主要集中于以下六个方向:

(1) 矩阵乘法优化

Lu 等人提出结合卷积和 Winograd 快速矩阵乘法,有效地减轻了对 DSP 资源的消耗,同时也极大地提高了计算性能,很好地适应当前小卷积核(1×1 和 3×3)的现状^[16]。Zhang 等人提出采用 FFT 将卷积计算转换到频域,从乘加运算转换为点乘运算,有效提高了计算性能,但是一般只对 5×5 及以上规模的卷积有加速效果^[17]。Li 等人针对全连接层参数过多导致的带宽瓶颈问题,通过提高批处理大小和对矩阵分块计算的方法,复用片上卷积核参数,有效减少了访存次数和数据量^[18]。Guan Y 等人对卷积层、长短期记忆层、全连接层等深入分析,将其中的计算密集部分转换为矩阵乘法,设计并行矩阵乘法计算单元对各层进行处理^[19]。

(2) 针对低位宽、二值、三元神经网络的优化

Nakahara H 等人提出基于二值神经网络的全连接层消失策略,利用二值神经网络的特殊性,用异或代替乘法,极大提高计算性能,但在数据精度上损失较大^[20]。Prost-Boucle A 等人提出了基于 CNN 的三元可拓展高性能加速器结构,使用三元神经网络,经合理训练后,可以达到与单精度浮点接近的准确度,并且在性能、能效、资源耗费等方面均优于一般 CNN 加速器^[21]。Wang 等人提出一种最高 8 位的混合精度框架,在损失较少精度的前提下,极大地提高计算性能,同时保证设计的低功耗^[22]。

(3) 量化、压缩、编码

Guo K 等人发布了将 CNN 模型映射到 FPGA 上的神经网络加速软硬件协同设计方法。通过对模型进行剪枝、编码和量化,大幅度减小模型体积,同时保持模型的准确度;将产生的模型编译成适用于特定 FPGA 加速器的指令流,具有一定的通用性^[23]。Parashar A 等人提出一种稀疏 CNN 加速器,在训练阶段对网络剪枝,训练完成后对网络参数编码,减少数据传输和存储量^[24]。韩松等人提出一种对神经网络进行压缩编码的方式,结合为压缩编码后的网络特殊设计的处理单元对全连接层进行加速,和 Titan X 相比,在性能和能效上分别是其 13 倍和 3400 倍^{[25][26]}。

(4) 带宽优化: 突发传输

实际的数据传输(专指访存)带宽,往往与突发传输长度有关。Ma 等人表示使用 DMA 进行数据传输时,如果对读取的参数重新排序,使得每次访存读取连续的数据更有利于读取操作^[27]。Zhang 等人通过实验表明,在给定最大突发传输长度的前提下,所传连续数据越多,越会逼近实际极限带宽(一般为理论极限带宽的 80%左右);换句话

说,在传输同样数据量的连续数据时,所给的最大突发传输长度越大,实际带宽越大^[28]。Qiu 等人对卷积层和全连接层的参数进行重排序,以增大突发传输长度同时减少传输次数,有效提高了带宽的利用率^[29]。

(5) 与 FPGA 动态可重构性结合

Venieris S I 等人提出 fpgaConvNet: 对给定 CNN 模型网络结构进行分析,将其映射到 FPGA 上。亮点在于,第一次提出将 FPGA 的动态可重构和 CNN 加速器相结合的可能性。虽然每次都对整片 FPGA 重构,极大地增加了重构时延,但对于结合 FPGA 动态可重构特性与神经网络加速器的研究仍具有重大意义^{[30][31]}。

Kanstner F 等人基于 FPGA 的部分可重构技术来减少配置文件大小和配置时延,同时通过软硬件协同的方式来进行 CNN 前向推断加速^[32]。在进行 FPGA 逻辑配置时,CPU 端也在进行计算,以掩盖配置时延。与纯 CPU 相比,性能提升 1.54~2.24 倍。虽然重构逻辑只占总逻辑的接近 1/3,但是配置时延仍要 120~150ms 左右,重构时延代价仍然很大。

(6) 架构优化

1、SIMD 架构

Zhang 等人设计出一种在输入和输出特征图数二维展开的 SIMD 卷积神经网络加速器架构,并且提出采用 Roofline 模型进行加速器设计空间的探索^[33]。Qiu 等人深入分析了 CNN 模型的计算与存储瓶颈,并设计出了一种在卷积核、输入和输出特征图数三维展开的 SIMD 加速器架构。但是,在建模时并没有将传输延时也考虑在内,导致实际性能与理论性能相差近 47%^[29]。Rahman 等人提出了一种将带宽考虑在内的加速器模型,但只是根据传输数据量简单建模,并没有考虑实际传输时延^[34]。Ma 等人深入分析了基于卷积循环展开的 SIMD 架构加速器的设计可能性,并提出在输出特征图数和输出特征图二维展开的 SIMD 架构加速器^[35]。然而,由于提出的架构将特征图像素和权重都缓存在片上,导致所耗费的 FPGA 资源过多,很难在小规模 FPGA 上使用。

2、脉动阵列架构

Wei 等人实现了基于 FPGA 的脉动阵列架构,同时提出可根据卷积层各维度的大小,改变复用模式,提高脉动阵列的利用效率^[36]。随后,又针对当前加速器只关注吞吐率而牺牲了处理延时的问题,提出一种多加速器并发架构,各加速器内部仍基于脉动阵列架构,然而都是基于大规模的脉动阵列部署^[37]。Shi 等人提出将 Winograd 快速矩阵乘法与小规模脉动阵列相结合;与传统实现相比,得到近 5 倍的性能提升,然而所谓的小规模实现仍然需要中等规模的 FPGA^[38]。

3、流式架构

Li 等人将设计了一种层间流水,层内输入、输出特征图数和卷积核三维展开的加速器架构,将完整的 Alexnet 网络映射到整块 FPGA 上,只适用于资源足够的情况^[18]。Wang 等人提出的混合精度框架也采用全流水架构,各层按顺序在整块 FPGA 上流水处理输入特征图^[22]。Zhang 等人采用层内 SIMD 展开,层间全流水的架构,通过合理的设计,极大地提高了计算资源的动态利用率^[39]。

综上所述,可以看到当前基于 FPGA 的神经网络加速器的研究方向较多,但是与加速器的规模可伸缩性关联较紧密的只有两个方向:(1)加速器的架构优化(2)与 FPGA 动态可重构特性相结合。所以,本文主要就此两个方向对加速器的规模可伸缩性进行研究。考虑到神经网络的范畴较大,本文主要研究基于 FPGA 的卷积神经网络加速器的规模可伸缩性。

1.3 研究内容及组织结构安排

1.3.1 研究内容

本文在数学工程与先进计算国家重点实验室开发基金的支持下,研究并设计了一种具有可伸缩性的基于 FPGA 的 SIMD 二维部分展开的 CNN 加速器架构。通过对所设计的加速器深入分析和建模,使得加速器性能与资源耗费模型与实际加速器基本一致。同时,通过改变加速器的硬件设计参数来调整加速器规模,以达到规模可伸缩性的目的。此外,结合 FPGA 的动态可重构特性,使得在一定配置时延的代价下,各层的计算可以与对应加速器架构相匹配,有效提高了计算资源的利用效率。本文的主要内容及创新点如下:

一、完成基于 FPGA 的规模可伸缩性的研究

根据现有研究,对 SIMD 架构的神经网络加速器进行研究,分析硬件设计参数以及数据复用模式对规模可伸缩性的影响,并设计一种具有规模可伸缩性的 CNN 加速器进行评估。

二、完成基于 FPGA 的 SIMD 二维部分展开的 CNN 加速器设计评估

使用高层次综合设计并实现了一种在输入和输出特征图数二维部分展开的 SIMD 架构的 CNN 加速器,并采用了参数重排序、多通道数据传输等优化策略来减少访存与传输时延。以目前在业界被广泛应用的 YOLOv2 目标检测算法为例,叙述将 CNN 模型映射到 FPGA 上的完整流程。同时,对加速器的性能和所需资源进行深入分析和建模,将实际传输延时考虑在内,极大地缩小了加速器理论时延与实际时延的误差。对由不同硬件设计参数与数据复用模式产生的不同规模的 CNN 加速器进行评估与对比。相关代码已在 github 开源。

三、改进基于 FPGA 的 SIMD 二维展开的 CNN 加速器架构中的输入和输出模块

改进了基于 FPGA 的 SIMD 二维展开的 CNN 加速器架构中的输入和输出模块,使得输入输出特征图的像素块能够以较大的突发长度进行传输,并对输入输出模块进行了双缓冲设计,使得片上偏移和访存地址的计算、片上缓存的传输时延与片外存储的访存时延能够重叠,有效提高了总线带宽的利用率,同时降低了传输与访存时延。

四、结合 FPGA 的动态可重构特性

结合 FPGA 的动态可重构特性,使得在一定配置时延的代价下,各层的计算可以与重构的加速器架构相匹配,通过批量数据复用加速器架构的方式分摊重构时延,有效提高了加速器性能,将卷积层与级联的最大池化层融合以减少访存时延。

1.3.2 组织结构安排

本文共有六个章节，具体的组织安排如下：

第一章：绪论。首先介绍本课题的研究背景及意义，对目前使用 FPGA 进行深度学习加速所遇到的问题进行了分析，并引出本课题的研究意义。其次，结合国内外基于 FPGA 的神经网络加速器的研究现状，总结出六个主要的研究方向，并从中选出与加速器规模可伸缩性相关的两个方向，作为本课题的主要研究方向。最后提出研究内容并交代本文的组织安排。

第二章：相关背景介绍。本章对涉及到的相关概念进行简单介绍。首先，介绍人工神经网络和卷积神经网络。其次，对于用于评估的 YOLOv2 目标检测模型进行了详细介绍，给出其涉及到的主要层的定义。然后，介绍 FPGA，包括 FPGA 本身的结构、FPGA 动态可重构特性和本文所使用的 Zynq FPGA+CPU 异构平台。

第三章：基于 FPGA 的 SIMD 架构的 CNN 加速器规模可伸缩性研究。以卷积循环为例，结合当前研究，针对三种影响加速器硬件设计的优化策略（循环展开、循环分块和循环交换），分析其对硬件设计参数、数据复用模式以及相关时延模型的影响。

第四章：基于 FPGA 的 SIMD 二维展开的 CNN 加速器的设计与评估。使用高层次综合设计并实现了一种在输入和输出特征图数二维部分展开 SIMD 架构的 CNN 加速器，并采用了参数重排序、多通道数据传输等优化策略来减少访存与传输时延。以 YOLOv2 目标检测模型为例，叙述将 CNN 模型映射到 FPGA 上的完整流程。将实际传输延时考虑在内，对加速器的性能和资源耗费进行了分析和建模。改进了基于 FPGA 的 SIMD 二维展开的 CNN 加速器架构中的输入和输出模块，有效提高了总线带宽的利用率，降低了传输与访存时延。对由不同硬件设计参数与数据复用模式产生的不同规模的 CNN 加速器进行评估与对比。

第五章：基于 FPGA 动态可重构的 CNN 加速器的设计与评估。首先，讨论结合 FPGA 的动态可重构特性与 CNN 加速器的可能性与主要问题。以 YOLOv2 为例，通过批量数据复用加速器架构的方式分摊重构时延代价，融合卷积层与级联的最大池化层减少访存时延，对结合动态可重构特性的加速器架构进行实验与评估。

总结与展望。总结了全文的内容与评估得到的结论，并且就基于 FPGA 的神经网络加速器的规模可伸缩性研究的进一步研究方向进行讨论和说明。

第二章 神经网络加速器与FPGA可重构技术基础

2.1 神经网络概述

2.1.1 人工神经网络与卷积神经网络介绍

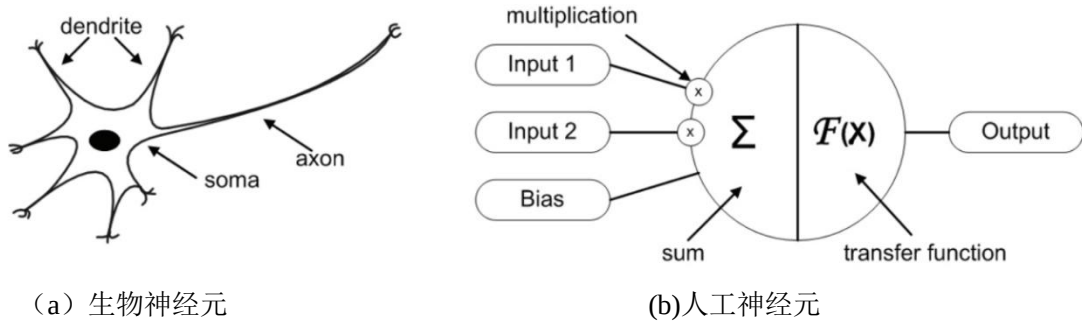


图 2-1 生物神经元与人工神经元^[40]

人工神经元是人工神经网络的基本组成模块，它源于对生物神经元的观察。如图 2-1 (a) 所示具有体细胞 (soma)，树突 (dendrite) 和轴突 (axon) 的生物神经元。单神经元由多个树突从其他神经元处得到输入信号。当多个输入信号的累积达到一定值时，通过轴突向其他神经元发送信号^[n8]。图 2-1 (b) 所示的人工神经元与图 2-1 (a) 类似，公式如 (2-1) 所示：

$$y = F(\sum_{i=1}^m W[i] * Input[i] + Bias) \quad (2-1)$$

其中 $Input[i]$ 表示由树突信号 i 传来的信号。 $W[i]$ 表示由树突 i 传来的信号对该神经元的影响，负值表示受到抑制。 $Bias$ 表示偏置，即神经元本身具有的电位信息。 $F(x)$ 表示激活函数，用于模拟对输入信号汇总后通过轴突传给其他神经元的行为。

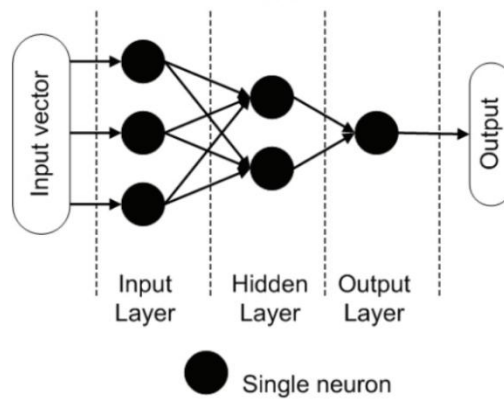
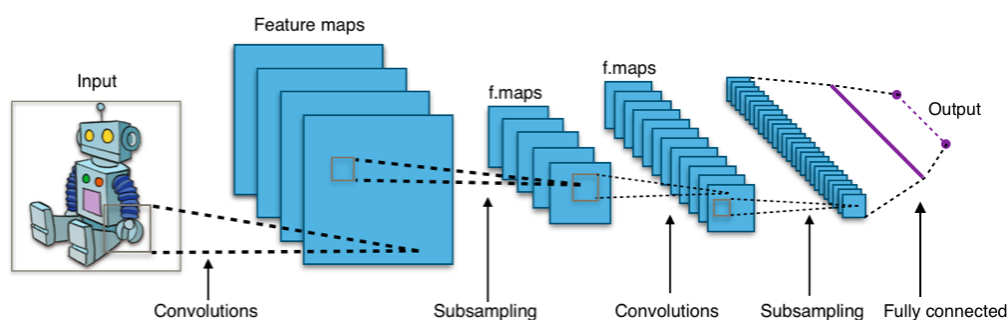


图 2-2 前向神经网络^[40]

将多个神经元按层排列，前后级联后，就形成了前向神经网络 (Feed-Forward Neural Network, FNN)，以此来模拟神经元连接。通过对神经网络各神经元权重的改变来实现对某种特定输入输出映射的拟合。由于本课题不涉及神经网络的训练，其他诸如反向传播权值更新等不再详细说明。

卷积神经网络，作为当前主流的神经网络中的一种，在图像分类、目标检测、视频编解码等领域都显露出惊人的效果。典型的卷积神经网络架构如图 2-3 所示：

图 2-3 典型卷积神经网络架构^[41]

输入一张包含机器人的图片，经 CNN 处理后，会输出包含机器人类别信息的结果。整个 CNN 模型主要由级联的卷积层（Convolution Layer, Conv）、降采样层和全连接层（Fully connected Layer, FC）组成。其中，卷积层通过卷积操作从输入特征图中提取特征；降采样层主要用于降低输入特征的规模；全连接层，与卷积层类似，不同之处在于从一维向量中提取特征。

2.1.2 YOLOv2 相关概念介绍

对 YOLO 的论文^{[3][42]}及框架源码^[43]分析，总结 YOLOv2 目标检测步骤如下：

（1）输入任意分辨率 RGB 图像，各像素除以 255 转化到[0,1]区间，按原图长宽比缩放至 416x416，不足处填充 0.5，得到 416x416x3 的数组。原图与转换回 RGB 图像的缩放图对比，如图 2-4 所示：



图 2-4 原图与缩放图对比

（2）将（1）得到的 416x416x3 数组输入 YOLOv2，网络检测后输出 13x13x425 的数组。对于 13x13x425 数组的理解：将 416x416 的图像划分为 13x13 的网格。针对每个网格，预测 5 个 bounding box(边框)，每个边框包含 85 维特征($5 \times 85 = 425$ 维)。每个 bounding box 的 85 维特征由 3 部分组成：对应边框中包含的 80 类物体的概率（80 维），边框中心点的相对偏移以及边框相对长宽的预测（4 维），边框是否包含物体的可信度（1 维）。

（3）对（2）中得到 13x13x425 数组进行处理，得到边框的中心位置和长宽，再根据各边框覆盖度、可信度和物体的预测概率等，对 13x13x5 个边框进行处理，得到最有可能包含某物体的边框。根据原图的长宽比，将得到的边框调整到原图尺度。原始图像与经目标检测后的结果对比，如图 2-5 所示：



图 2-5 原图与结果图对比

如表 2-1 所示, YOLOv2 总共由 32 层组成, 其中涉及 5 种层: 卷积层(conv), 最大池化层(max), 路由层(route), 重排序层(reorg)以及最后的检测层(detection)。其中, 卷积层起到特征提取的作用, 池化层用于像素抽样。路由层用于多层次的特征融合, 重排序层用于特征的抽样重排。在本课题中, 将步骤(1)称为图像的预处理, 步骤(3)称为图像的后处理(后处理中包含第 31 层检测层的处理)。

表 2-1 YOLOv2 网络结构

	layer	filters	Size / Stride	input	output
0	conv	32	3 x 3 / 1	416 x 416 x 3	416 x 416 x 32
1	max		2 x 2 / 2	416 x 416 x 32	208 x 208 x 32
2	conv	64	3 x 3 / 1	208 x 208 x 32	208 x 208 x 64
3	max		2 x 2 / 2	208 x 208 x 64	104 x 104 x 64
4	conv	128	3 x 3 / 1	104 x 104 x 64	104 x 104 x 128
5	conv	64	1 x 1 / 1	104 x 104 x 128	104 x 104 x 64
6	conv	128	3 x 3 / 1	104 x 104 x 64	104 x 104 x 128
7	max		2 x 2 / 2	104 x 104 x 128	52 x 52 x 128
8	conv	256	3 x 3 / 1	52 x 52 x 128	52 x 52 x 256
9	conv	128	1 x 1 / 1	52 x 52 x 256	52 x 52 x 128
10	conv	256	3 x 3 / 1	52 x 52 x 128	52 x 52 x 256
11	max		2 x 2 / 2	52 x 52 x 256	26 x 26 x 256
12	conv	512	3 x 3 / 1	26 x 26 x 256	26 x 26 x 512
13	conv	256	1 x 1 / 1	26 x 26 x 512	26 x 26 x 256
14	conv	512	3 x 3 / 1	26 x 26 x 256	26 x 26 x 512
15	conv	256	1 x 1 / 1	26 x 26 x 512	26 x 26 x 256
16	conv	512	3 x 3 / 1	26 x 26 x 256	26 x 26 x 512
17	max		2 x 2 / 2	26 x 26 x 512	13 x 13 x 512
18	conv	1024	3 x 3 / 1	13 x 13 x 512	13 x 13 x 1024
19	conv	512	1 x 1 / 1	13 x 13 x 1024	13 x 13 x 512
20	conv	1024	3 x 3 / 1	13 x 13 x 512	13 x 13 x 1024
21	conv	512	1 x 1 / 1	13 x 13 x 1024	13 x 13 x 512
22	conv	1024	3 x 3 / 1	13 x 13 x 512	13 x 13 x 1024
23	conv	1024	3 x 3 / 1	13 x 13 x 1024	13 x 13 x 1024
24	conv	1024	3 x 3 / 1	13 x 13 x 1024	13 x 13 x 1024
25	route	16			
26	conv	64	1 x 1 / 1	26 x 26 x 512	26 x 26 x 64

27	reorg		/2	26 x	26 x	64	13 x	13 x	256
28	route	27 24							
29	conv	1024	1 x 1 / 1	13 x	13 x	1280	13 x	13 x	1024
30	conv	425	1 x 1 / 1	13 x	13 x	1024	13 x	13 x	425
31	detection								

(1) 卷积层(Convolutional Layer)

对输入特征图以对应的卷积核进行卷积来实现特征提取，伪代码如图 2-6 所示：

```

for (m=0;m<Nof;m++) → Loop-4
  for (r=0;r<Noy;r++)
    for (c=0;c<Nox;c++) } → Loop-3
    for (n=0;n<Nif;n++){ → Loop-2
      for (ky=0;ky<Nky;ky++)
        for (kx=0;kx<Nkx;kx++){ } → Loop-1
      pixelL(m,r,c) += pixelL-1(n,r×S+ky,c×S+kx)
      × weightL-1(m,n,ky,kx);}
      pixelL(m,r,c)=pixelL(m,r,c)+bias(m);}

```

图 2-6 卷积层伪代码

其中(Noy, Nox)、 Nof 、 Nif 、(Nky, Nkx)、 S 分别代表输出特征图、输出特征图数、输入特征图数、卷积核大小和步长。 $pixel_L(m,r,c)$ 代表输出特征图 m 中 r 行 c 列的像素。全连接层可看做 $Nky=Nkx=1, S=1$ 的特殊卷积层。形象的卷积过程，如图 2-7 所示：

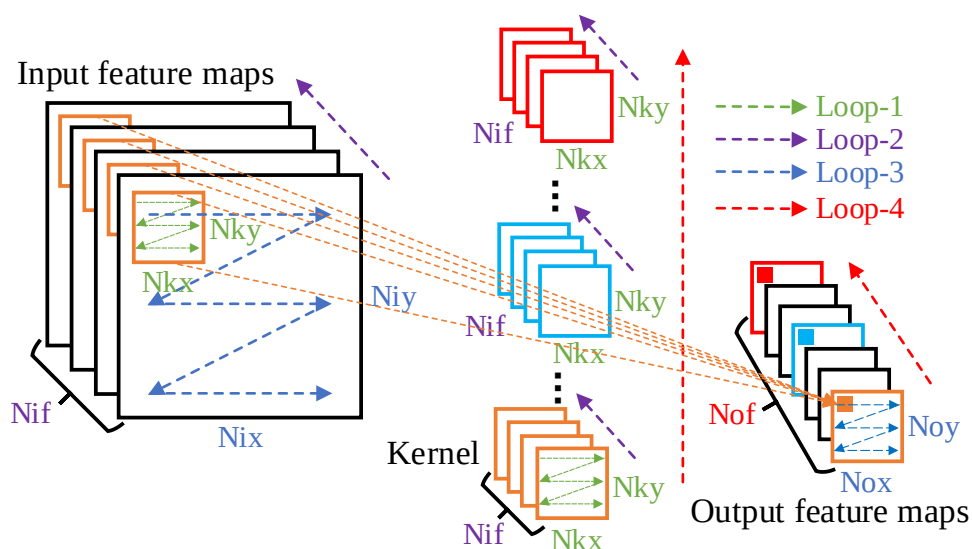


图 2-7 卷积过程

如图 2-7 所示，各卷积循环按图中的箭头顺序进行卷积运算，需要对 Nif 张输入特征图中相同位置的某像素块与对应的卷积核在该输入特征图维度的卷积模板进行 $Nkx \times Nky$ 个乘加，最后将得到的 Nif 个结果累加，再加上对应卷积核的偏置参数，才能得到某输出特征图上的一个像素。

(2)池化层(Pool Layer)

对输入特征图降采样，降低特征图的规模，同时也具有保持某种不变性（旋转、平移、伸缩等）的功能，一般跟在卷积层后。通常采用最大池化或平均池化，最大池化层伪代码如图 2-8 所示：

```
for (r=0;r<Noy;r++)
  for (c=0;c<Nox;c++)
    for (m=0;m<Nof;m++){
      pixelL(m,r,c) = Max(pixelL-1(n,r×S+ky,c×S+kx));
      ky,kx∈{0,1,⋯,Nk-1}
```

图 2-8 最大池化层伪代码

其中 Max 函数表示在 $Nk \times Nk$ 的邻域中返回最大像素，其他参数含义与卷积层类似。

(3) 路由层(Route Layer)

将多层的输出特征图级联，作为下一层的输入特征图，实质是将多维度的特征信息简单融合。

(4)重排序层(Reorg Layer)

输入特征图在邻域内按位置抽样，相同位置的像素输出形成子图，如图 2-9 所示：

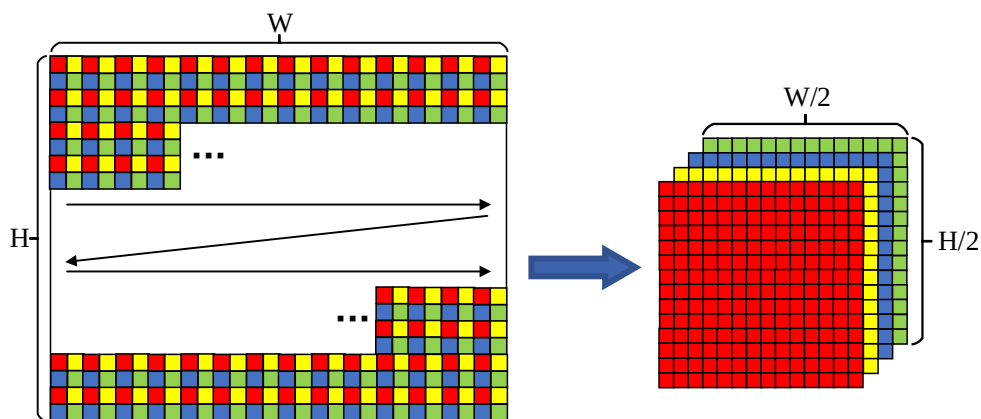


图 2-9 步长 S 为 2 时的重排序过程

若步长 $S=2$ ，则一张 $W \times H$ 的输入特征图在 2×2 的邻域内按位置抽样，最终输出 4 张 $W/2 \times H/2$ 的子图。

(5)批归一化(Batch Normalization, BN)

深度学习中通过批归一化来加快训练收敛^[44]。因此，在推断阶段，使用批归一化的卷积层的每个输出特征图像素需要再进行以下映射：

$$y = \frac{x - bn0}{\sqrt{bn1}} \quad (2-2)$$

$$z = sc0 \times y + sc1 \quad (2-3)$$

其中 $bn0, bn1, sc0, sc1$ 在训练完成后为常数。可以将公式 2-2 与 2-3 合并，得到：

$$\text{应该为 } z =, \text{ 而非 } y = A \times x + B \quad (2-4)$$

其中 $A = \frac{sc0}{\sqrt{bn1}}$, $B = sc1 - \frac{sc0 \cdot bn0}{\sqrt{bn1}}$ 为常数。

由于 BN 一般跟在卷积计算后，激活函数前。由公式 2-4 可知，BN 实际是做线性变换，所以可以将其与对应的卷积计算融合，得到新的权重和偏置参数如下：

$$\mathbf{weight}'_{L-1}[m] = \mathbf{weight}_{L-1}[m] \times A_m \quad (2-5)$$

$$\mathbf{bias}'_{L-1}[m] = \mathbf{bias}_{L-1}[m] \times A_m + B_m \quad (2-6)$$

其中， A_m 和 B_m 是针对不同输出特征图 m 的常数。本页后的权重和偏置参数默认都已经过 BN 融合。

(6) 激活函数

激活函数对每个输出特征图像素做一个非线性变换，主要用于给网络增加非线性拟合的能力，一般位于批归一化后。YOLOv2 中用到的激活函数是 Leaky ReLU：

$$f(x) = \begin{cases} x, & x > 0 \\ x \times 0.1, & \text{else} \end{cases} \quad (2-7)$$

2.2 FPGA 及动态可重构技术概述

2.2.1 FPGA 介绍

FPGA 是一种集成电路（Integrated Circuit, IC）芯片，可以在成片后针对不同的算法进行定制硬件加速逻辑设计。现代 FPGA 由多达 200 万个逻辑单元组成，可通过对逻辑单元的配置实现各种软件算法。FPGA 的基本结构由以下 4 个元素组成：

- 查找表（Look-up table, LUT）：该元素用于形成简单数字逻辑，通过多查找表级联形成不同的逻辑。
- 触发器（Flip-Flop, FF）：寄存器，一般用于存储 LUT 的结果。
- 导线：将各元素相互连接。
- 输入/输出（I/O）端口：这些物理端口将数据输入和输出 FPGA。

这些元素的组合产生了基本的 FPGA 架构，如图 2-10 所示。尽管通过该基本结构可以实现任何当前 CPU 可以实现的算法，但是实现算法的实际效果受限于可用资源、时钟频率和输入输出端口的带宽。

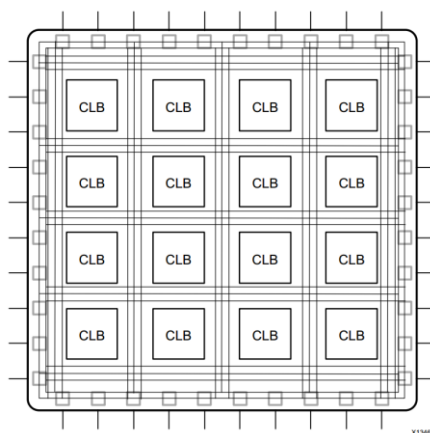


图 2-10 基本 FPGA 架构^[45]

如图 2-11 所示, 现代 FPGA 架构除了包含基本元素外, 还包含额外的计算模块 (Digital Signal Processor, DSP) 和存储模块 (BRAM, Block RAM) 来减少对 LUT 的耗费以及提高计算和存储效率。附加元素如下:

- BRAM: 与 CPU 不同, 由于 FPGA 可以自由设计数字逻辑电路, 所以将 BRAM 资源在整块 FPGA 上分布式部署, 可以减少访问 BRAM 的延时, 同时各 BRAM 间互相独立, 可以并发读写。

- 锁相环 (Phase-locked loops, PLL): 用于以不同的时钟速率驱动 FPGA 逻辑, 一般可以达到 100MHz~500MHz

- 高速串行收发器: 考虑到 IO 端口性能不足而添加的高速端口, 一般可以达到 10Gb/s 数量级。

- 片外存储控制器: 考虑到 BRAM 的宝贵性(往往中高端 FPGA 只有几 MB 的 BRAM 资源), 处理大规模数据时还是会需要使用片外 DRAM, 所以也会集成 DRAM 控制器。

- DSP: 由于 FPGA 常用于数字信号处理, 所以添加 DSP 来处理一些乘加运算, 减少对 LUT 资源的耗费以及降低布线难度。

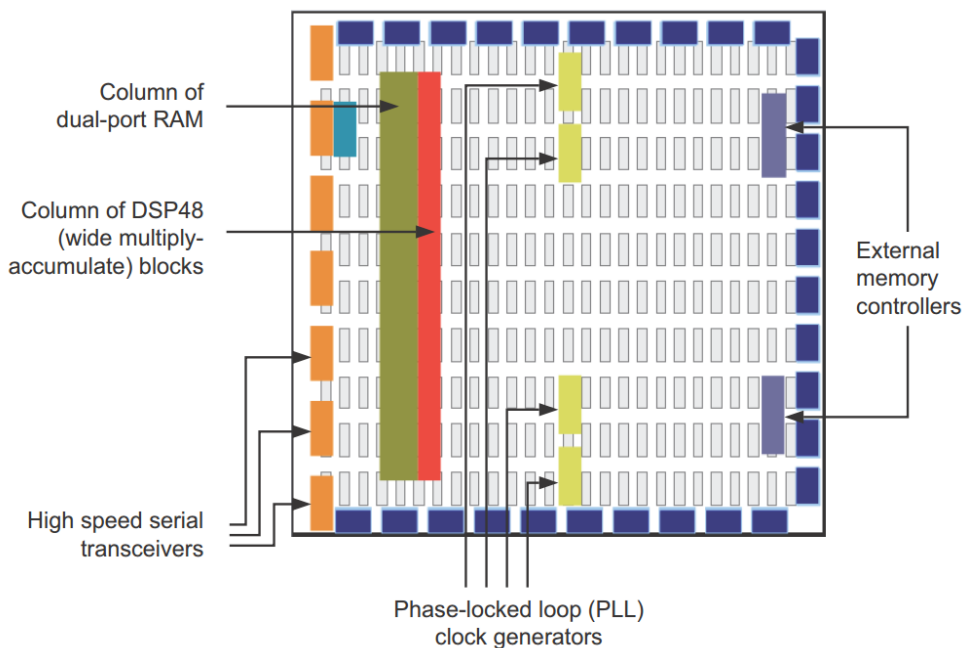


图 2-11 现代 FPGA 架构^[45]

2.2.2 FPGA 动态可重构技术介绍

FPGA 通过对片内各种资源配置, 来实现不同的数字逻辑。因此, 可以通过对 FPGA 的重新配置 (通过特定端口给 FPGA 写入配置流文件), 将改变其中的数字逻辑, 使之适用于新的应用。此处将这种对整片 FPGA 重新配置的方式称为全重构。现代 FPGA 具有部分重配置 (Partially Reconfigurable, PR) 的特性, 在进行逻辑设计时, 可以提前对 FPGA 进行划分, 将设计放置到所划分的 FPGA 区域, 在布局布线完成后, 得到只对该块 FPGA 区域进行配置的部分配置流文件, 将此种配置方式称为部分可重构。

因此, 在使用完整的配置流文件全重构 FPGA 后, 可以通过部分配置流文件修改划分区域的逻辑, 而不会影响到其他区域上正在运行的逻辑。如图 2-12 所示, 针对可重构

区域块 A，可以通过写入不同的部分配置流文件 A1.bit 或 A2.bit，在 A 区域实现不同的硬件逻辑，而不会影响到除此之外的其他 FPGA 区域。

但是，不论是全重构还是部分可重构，在对 FPGA 进行重新配置时，都具有配置时延的代价。在进行 FPGA 配置的阶段，FPGA 本身是无法工作的，这也导致了对于想要将动态可重构特性加入到对时延非常敏感的 FPGA 设计中是比较困难的。

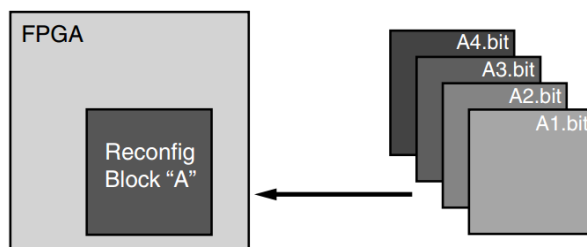


图 2-12 部分可重构示意图^[46]

2.2.3 Zynq 异构平台介绍

如图 2-13 所示，本文评估使用的硬件平台基于 Xilinx 的 Zynq-7000 SoC^[47]，Zynq 系列将传统嵌入式的 ARM+外设控制器（Processing System，PS）与 FPGA 逻辑（Programmable Logic，PL）通过一些接口和互联路由连接。从而，开发者可以在 PL 端设计对应具体应用的加速逻辑，由 ARM 对所设计的逻辑进行控制，将计算密集型的任务交由 PL 端执行，PS 端主要负责控制和中断处理。可以看到，图中 PS 和 PL 的接口主要由 3 类组成：（1）EMIO：对应某些在 PL 端输入输出的外设，通过 EMIO 连接到 PS 端的外设控制器。（2）GP（General-Purpose，GP）端口：基于 AXI3 Lite 协议，对应控制总线，负责读写控制、数据和状态寄存器等。（3）HP（High-Performance，HP）端口：基于 AXI3 协议，对应数据总线，负责和 PS 端的片上缓存（On-chip memory，OCM）和内存进行数据交互。

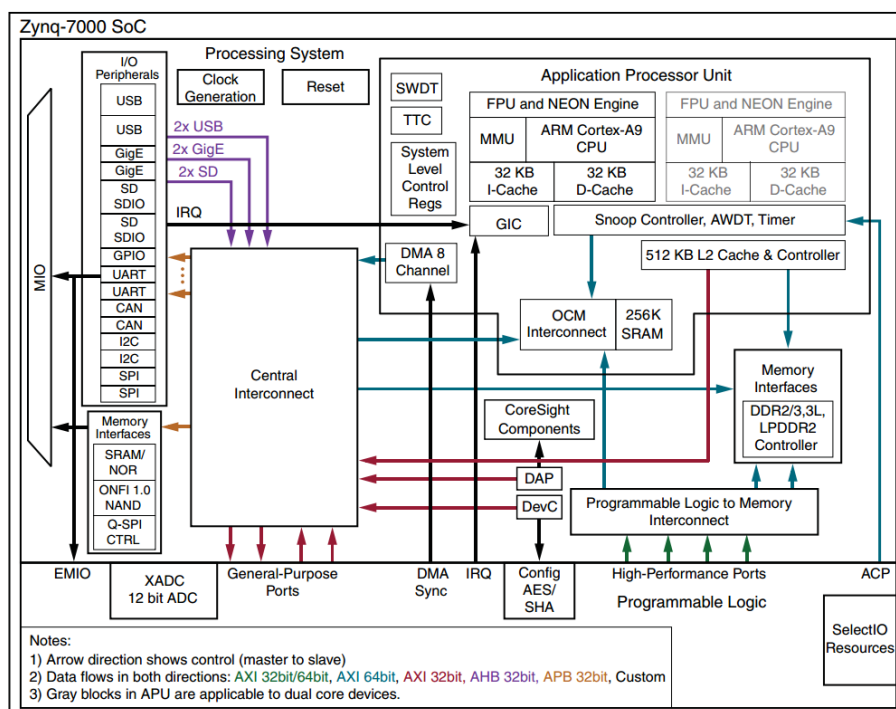


图 2-13 Zynq-7000 SoC 架构^[47]

2.3 神经网络加速器相关概念介绍

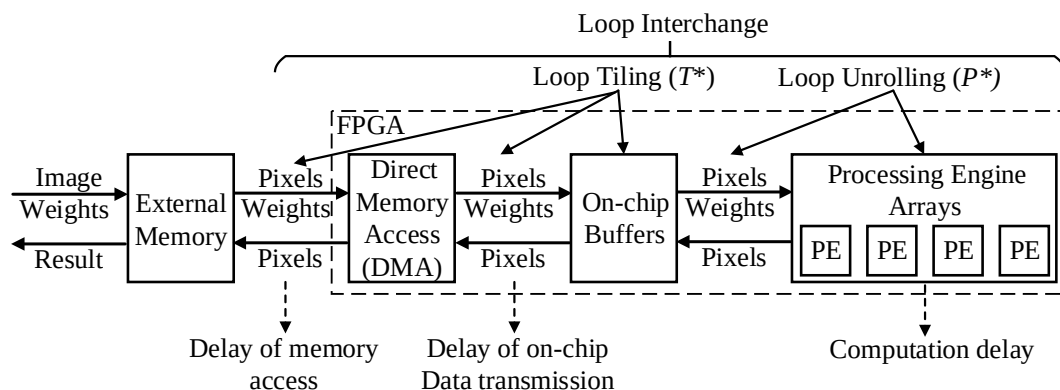


图 2-14 基于 FPGA 的神经网络加速器

如图 2-14 所示,当前基于 FPGA 的神经网络加速器基本使用类似架构。考虑到 FPGA 片上存储资源的稀缺(一般小于 10MB),通常使用三层存储架构:片外存储、片上缓存和处理单元内的局部寄存器。与 Cache 缓存不同之处在于,专用加速器可以预先得知下次访存所要读取或写入的数据地址和数据量,因此可以针对访存时延进行特定优化。由上图可以看到,加速器的延时主要由三部分组成:访存时延、片上传输时延和计算时延。除了使用乒乓缓存的设计将各时延重叠掩盖外,当前为了减少时延的优化策略主要可归纳分为 3 类^{[35][48-51]}:循环展开(Loop Unrolling)、循环分块(Loop Tiling)、循环交换(Loop Interchange)。

2.3.1 循环展开

在不考虑访存时延和片上传输时延的前提下,如果计算单元无法快速地计算所给数据,计算将成为瓶颈。由于卷积层在神经网络中占主要的计算量^[29],当前主要集中于对卷积层的加速研究,通过对卷积循环的多维进行展开,设计多个并行乘加计算单元来提高计算能力^{[18][22][27-35][39][48-52]}。

2.3.2 循环分块

由于 FPGA 的片上存储资源相对于计算所需的存储空间来说是极其不足的,当前大部分研究主要利用卷积计算的数据局部性,对卷积多维划分,使得在进行卷积计算时,多次复用片上数据。从而,某些数据只需要从片外存储中读取或写入少量次数,极大降低访存次数和访存数据量,使得访存不至于成为瓶颈^{[16][18][19][22][25-35][39][48-52]}。

2.3.3 循环交换

由于卷积运算由六维循环组成,所以对卷积循环先后顺序的改变并不影响卷积的最终结果。而在加速器的架构设计中,卷积循环的顺序实际代表着不同的数据访问模式,同时也影响着数据的复用模式,因此会对访存和片上传输时延造成影响^[48-51]。

2.3.4 突发传输

实际读写 DRAM 的带宽往往与突发传输长度、数据位宽和时钟频率有关^[28]。当数据位宽和时钟频率保持不变时，传输同样长度的连续数据，突发传输长度越长，传输时延越小。在 Xilinx 的 Zedboard 开发板上测试 32 位 AXI4 Master 接口在不同突发传输长度下，向 DRAM 写连续的 64MB 数据得到的带宽。纵轴是实测带宽 MB/s，横轴是突发传输长度，从 1 至 256。100MHz 工作频率下，突发长度为 256 时，最大有效带宽为 332MB/s，此时带宽效率约为 83%。150MHz 工作频率下，突发长度为 256 时，能够得到最大有效带宽 489MB/s，带宽效率约为 81%。如图 2-15 所示：

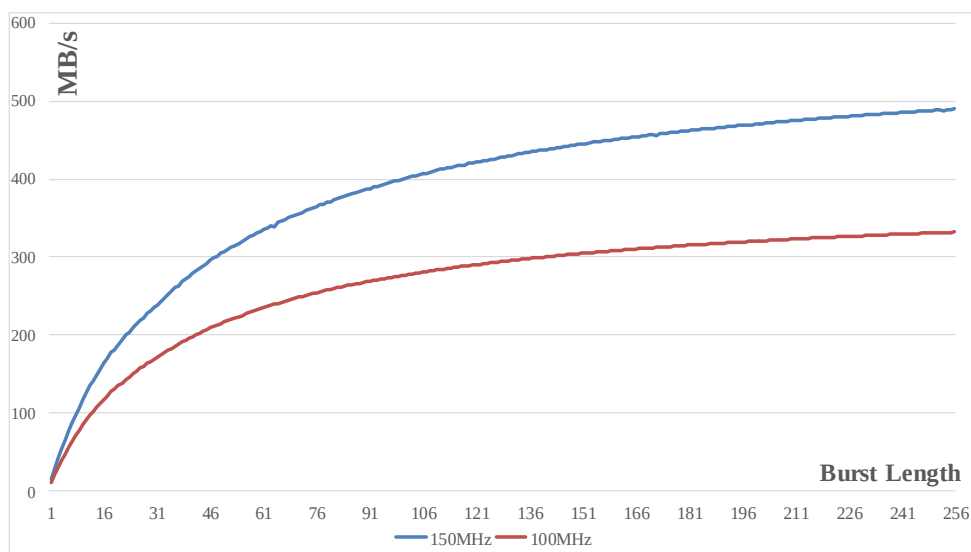


图 2-15 传输连续 64MB 的有效带宽

若已知不同突发长度的有效带宽，根据算法 2-1，对读写 DRAM 时延进行仿真。

算法 2-1 读写 DRAM 时延仿真

输入:(1)连续传输 32 位数据长度 $Length$ (32bit)

(2)最大突发长度 $Burst Length_{Max}$ (1~256)

(3)对应 1~256 突发长度的有效带宽数组 $Bandwidth[256]$ (MB/s)

输出: 时延 $Latency$ (s)

算法:

初始化时延 $Latency = 0.0$

以最大突发长度传输的次数: $Num_{Max} = \lfloor Length / Burst Length_{Max} \rfloor$

计算剩余项的突发长度: $Burst Length_{Rm} = Length \% Burst Length_{Max}$

计算以最大传输长度传输的传输时延:

$$Latency_{Max} = \frac{Num_{Max} \times Burst Length_{Max}}{Bandwidth[Burst Length_{Max} - 1] \times 2^{18}}$$

若剩余项不为 0，计算剩余项的传输时延:

$$Latency_{Rm} = \frac{Burst Length_{Rm}}{Bandwidth[Burst Length_{Rm} - 1] \times 2^{18}}$$

否则，剩余项的传输时延: $Latency_{Rm} = 0$

$$Latency = Latency_{Max} + Latency_{Rm}$$

下文以 $MM(Length, Burst\ Length_{Max})$ 来表示在最大突发长度为 $Burst\ Length_{Max}$ 的前提下，传输连续 $Length$ 个 32 位数据的时延。

2.3.5 Roofline 模型

Roofline 模型是一种直观的性能评估模型，用于评估和探索在给定硬件系统（即已知峰值计算性能和理论最大带宽）的情况下，对于各种应用所能达到的峰值性能与优化方向^[53]。所以，也可用于评估和探索基于 FPGA 的神经网络加速器的设计空间^[33]。典型的 Roofline 模型，如图 2-16 所示：

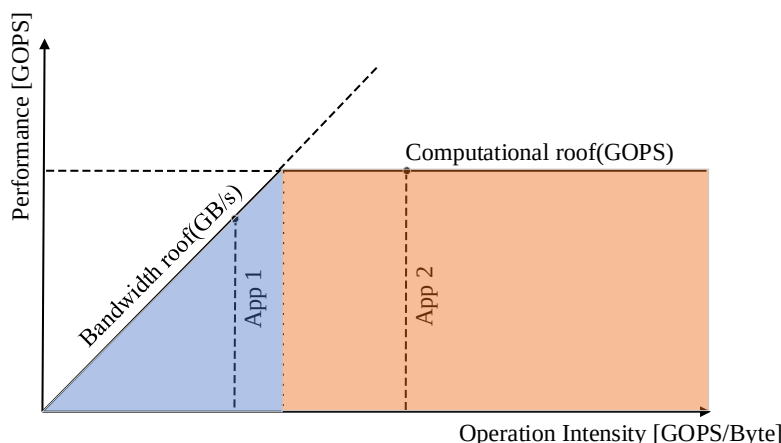


图 2-16 Roofline 模型

其中，计算上限（Computational roof）表示给定硬件系统所能达到的理论峰值计算性能，带宽上限（Bandwidth roof）表示给定硬件系统与内存交互的最大带宽。横坐标计算密度（Operation Intensity）表示平均从内存读取或写入单位数据所参与的计算量，一般用总计算量除以总访存数据量得到；纵坐标表示系统以及各应用所能达到的性能。

图 2-16 中，应用 1 落在了带宽受限区域（蓝色区域），如果应用 1 的总计算量与访存量不发生改变，最大只能达到带宽上限所限定的性能。应用 2 落在了性能上限的区域（橘色区域），此时应用 2 可以达到系统的最大性能。实际应用中，希望应用能够落在橘色区域，且越靠近右上方的计算上限越好，主要有两个原因：（1）越向上靠近性能上限，说明实际性能越接近最大性能，系统利用率越高。（2）越靠近右方，说明计算密度越大，即单位内存数据交换对应的计算量越大，能效越好。

2.4 本章小结

本章主要是介绍相关概念，主要分为 3 部分：（1）与神经网络相关：简单介绍人工神经网络和卷积神经网络，以及在下文中会涉及到的 YOLOv2 目标检测算法模型。（2）与 FPGA 硬件平台相关：介绍了 FPGA 的组成元素与 FPGA 动态可重构技术，以及在下文中使用的 FPGA+CPU 异构平台 Xilinx 公司的 Zynq-7000 SoC。（3）与神经网络加速器相关：主流的基于 FPGA 的神经网络加速器的基本结构，三类优化策略，对突发传输的仿真和评估模型性能的 Roofline 模型。

第三章 基于SIMD架构的CNN加速器规模可伸缩性研究

本节主要介绍基于 SIMD 架构的 CNN 加速器的三大优化策略，以及相应的加速器设计参数。加速器的可伸缩性实质可以看做对加速器设计空间的探索：给定某 CNN 模型，在有限的 FPGA 片上资源和片外存储带宽的前提下，如何设计出性能较好的加速器。不同的加速器设计参数组合实际上就代表了不同规模的加速器。

在神经网络加速器中，卷积层占据了大部分计算量，其他层的处理或类似卷积层（如最大池化层），或可以转换为特殊的卷积层（如全连接层）。下文主要针对卷积循环进行量化分析的说明，对其他层的处理可以以此为参考。其中，关于循环展开和硬件设计参数的定义主要来自[51]。

卷积循环的硬件设计参数如表 3-1 所示，第一行表示图 2-6 与图 2-7 中所对应的不同循环； (N_{kx}, N_{ky}) , (N_{ox}, N_{oy}) , N_{if} , N_{of} 表示卷积层的各维度大小。循环分块的各变量 T^* ，以及各维度循环的展开度 P^* 。各设计参数满足以下约束：

$$1 \leq P^* \leq T^* \leq N^* \quad (3-1)$$

其中 P^* 、 T^* 和 N^* 表示前缀不同但后缀相同的各设计参数。输入特征图宽高与分块设计参数可以由输出特征图的对应变量得到：

$$N_{ix} = (N_{ox} - 1) \times S + N_{kx} \quad (3-2)$$

$$N_{iy} = (N_{oy} - 1) \times S + N_{ky} \quad (3-3)$$

$$T_{ix} = (T_{ox} - 1) \times S + T_{kx} \quad (3-4)$$

$$T_{iy} = (T_{oy} - 1) \times S + T_{ky} \quad (3-5)$$

此处列出的硬件设计参数不包括由循环交换产生的数据复用模式的选择。针对某特定网络，给定硬件设计参数，若不考虑传输和访存时延，某些层可能在某个维度并不能完美地匹配当前的加速器，这就会导致实际资源利用率低下。

另外，如果将传输和访存时延考虑在内，由于各层维度“形状”的不同，往往也会产生计算或访存瓶颈。针对以上问题，本章首先分析 4 种最基本的循环展开与对应的硬件结构；其次，分析循环分块与对应的片上缓存的影响；然后，分析由循环交换产生的 3 种主要的数据复用模式与对应的时延模型；最后，给出一种在资源受限情况下，搜索加速器的设计空间探索算法。

通过对加速器设计空间的探索，能够在给定网络结构与 FPGA 片上资源和片外带宽资源的前提下，得到各种满足约束与需求的硬件设计参数组合（即对应的不同规模的加速器）。

表 3-1：卷积循环维度与硬件设计参数

	Kernel (width/height)	OFM (width/height)	Num of IFMs	Num of OFMs
Conv Loops	Loop-1	Loop-3	Loop-2	Loop-4
Conv Dimensions (N^*)	N_{kx}, N_{ky}	N_{ox}, N_{oy}	N_{if}	N_{of}
Loop Tiling (T^*)	T_{kx}, T_{ky}	T_{ox}, T_{oy}	T_{if}	T_{of}
Loop Unrolling (P^*)	P_{kx}, P_{ky}	P_{ox}, P_{oy}	P_{if}	P_{of}

3.1 循环展开

从硬件设计的角度，对卷积循环不同维度的展开实质是依据卷积循环的计算特点，根据不同的展开维度设计相应的并行乘法和加法计算单元，以达到加速卷积计算，减少计算时延的目的。基本的维度展开可以分为以下 4 类：

(1) 卷积核维度的展开

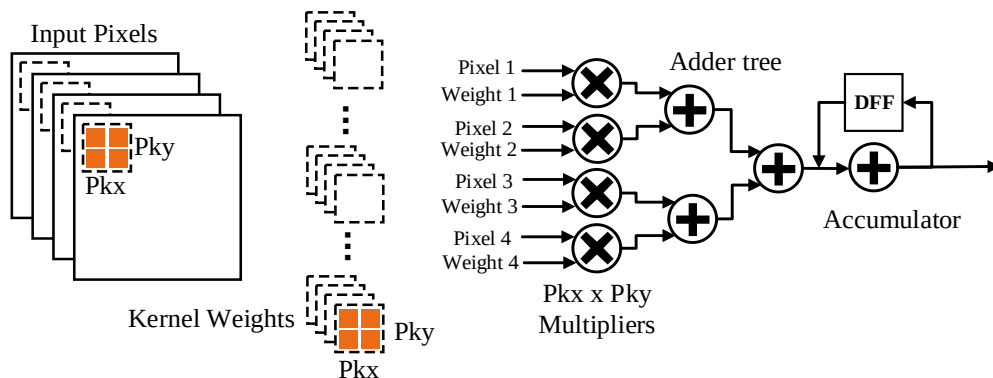


图 3-1 展开卷积核维度与对应的硬件架构

如图 3-1 所示，展开卷积核的宽高，展开度分别为 P_{kx} 与 P_{ky} 。初始化流水线后，每个时钟周期，并行计算同一输入特征图的 $P_{kx} \times P_{ky}$ 个相邻像素与同一卷积核内对应不同位置权重的乘积。然后，通过 1 个深度为 $\lceil \log_2(P_{kx} \times P_{ky}) \rceil$ 的加法树将乘积两两相加得到中间结果。最终，将中间结果通过 1 个累加器，与之前得到的部分和相加，并存储在寄存器或片上存储中。在此维展开，需要 $P_{kx} \times P_{ky}$ 个乘法器，1 个深度为 $\lceil \log_2(P_{kx} \times P_{ky}) \rceil$ 的加法树和 1 个累加器。

(2) 输入特征图数维度的展开

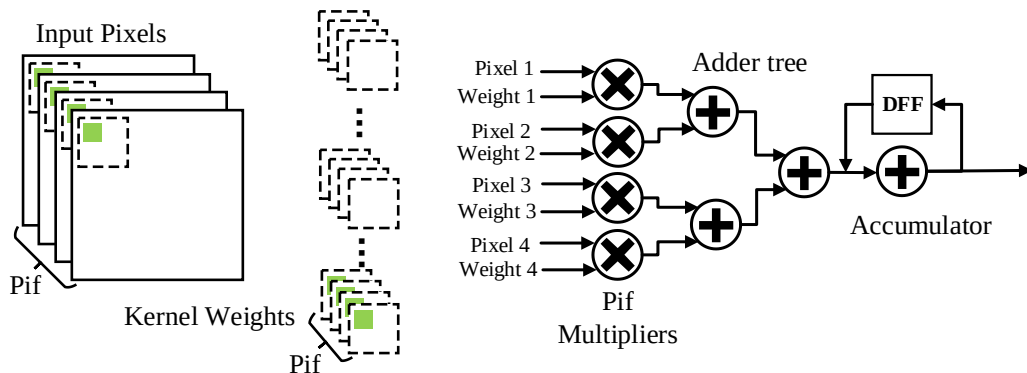


图 3-2 展开输入特征图数维度与对应的硬件架构

如图 3-2 所示，展开输入特征图数维度，展开度为 P_{if} 。每个时钟周期，并行地从 P_{if} 张输入特征图的相同位置读取一个像素与同一卷积核内的对应权重进行乘法运算。乘积通过 1 个深度为 $\lceil \log_2(P_{if}) \rceil$ 的加法树两两相加，得到中间结果。最终，将中间结果通过 1 个累加器，与之前得到的部分和相加，存储在寄存器或片上存储中。在此维展开，需要 P_{if} 个乘法器，1 个深度为 $\lceil \log_2(P_{if}) \rceil$ 的加法树和 1 个累加器。

(3) 输出特征图维度的展开

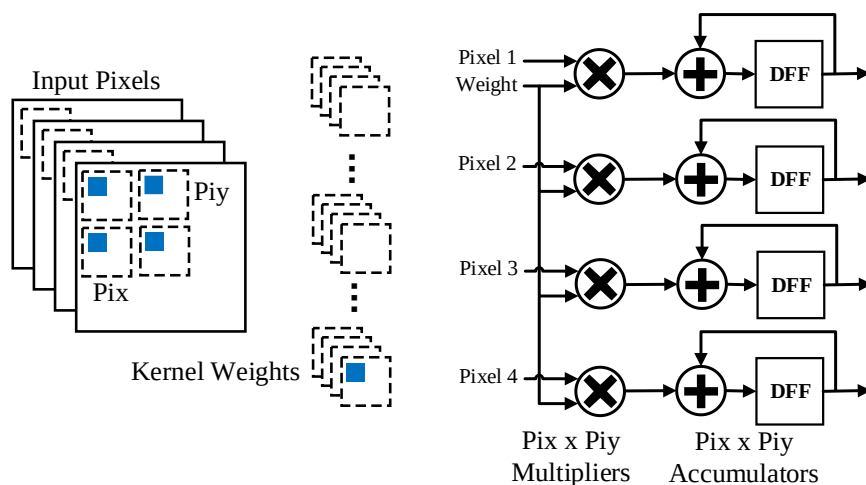


图 3-3 展开输出特征图维度与对应的硬件架构

如图 3-3 所示，展开输出特征图宽高维度，对应展开度为 P_{ix} 与 P_{iy} 。在每个时钟周期，同时计算同一输出特征图 $P_{ix} \times P_{iy}$ 个相邻像素的值，此时对同一输入特征图中按步长 S 偏移的 $P_{ix} \times P_{iy}$ 个相近元素与同一卷积核内的同一权重进行乘法运算，将得到的 $P_{ix} \times P_{iy}$ 个乘积与对应的累加和累加，并存储到相应寄存器或片上存储中。在此维展开，需要 $P_{ix} \times P_{iy}$ 个乘法器和 $P_{ix} \times P_{iy}$ 个累加器。同时，并行计算同一输出特征图的 $P_{ix} \times P_{iy}$ 个相邻像素对卷积核复用了 $P_{ix} \times P_{iy}$ 次。

(4) 输出特征图数维度的展开

如图 3-4 所示，展开输出特征图数维度，展开度为 P_{of} 。在每个时钟周期，将同一输入特征图的某一像素与对应不同卷积核相同位置处的权重进行 P_{of} 个并行乘法运算。将得到的乘积与对应的 P_{of} 个部分和累加，并存储到对应寄存器或片上存储中。在此维展开，需要 P_{of} 个乘法器和 P_{of} 个累加器。此时，并行计算 P_{of} 张输出特征图相同位置的像素对输入特征图复用了 P_{of} 次。

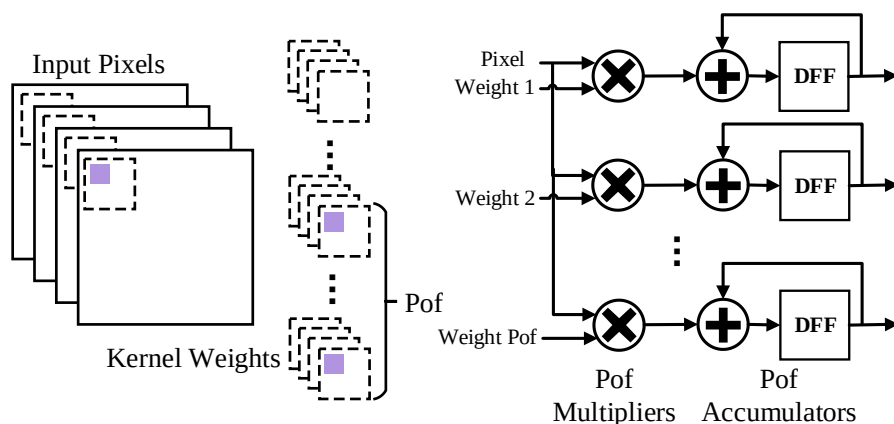


图 3-4 展开输出特征图数维度与对应的硬件架构

由于各卷积层维度“形状”的不同（卷积各维的长度不同），对不同维度展开设计的硬件架构针对不同的卷积层往往会导致不同的计算效率。因此，当前的研究主要通过

以上 4 种基本的循环展开的组合来设计针对整个网络来说整体计算效率较高的并行硬件架构。

3.2 循环分块

如图 3-5 所示，由于 FPGA 片上存储资源的不足，所以通过对卷积循环分块，使得每次访存只涉及 T_{if} 张输入特征图的 $T_{ix} \times T_{iy}$ 大小的像素块、 T_{of} 个卷积核中对应的 $T_{if} \times T_{ky} \times T_{kx}$ 个权重参数，以及 T_{of} 张输出特征图的 $T_{oy} \times T_{ox}$ 大小的像素块。结合循环交换，确定数据复用模式后，通过合理地设计分块参数 (T^*)，复用片上存储中的数据，有效地减少访存次数和访存数据量，使得访存不至于成为加速器时延的主要瓶颈。片上输入特征图缓存、卷积核缓存和输出特征图缓存对应耗费的最小 BRAM 资源数如下所示：

$$N_{MinBRAM IFM} = \left\lceil \frac{T_{if} \times T_{ix} \times T_{iy} \times datawidth}{C_{OneBRAM}} \right\rceil \quad (3-6)$$

$$N_{MinBRAM W} = \left\lceil \frac{T_m \times T_n \times T_{kx} \times T_{ky} \times datawidth}{C_{OneBRAM}} \right\rceil \quad (3-7)$$

$$N_{MinBRAM OFM} = \left\lceil \frac{T_m \times T_{ox} \times T_{oy} \times datawidth}{C_{OneBRAM}} \right\rceil \quad (3-8)$$

$C_{OneBRAM}$ 表示单一 BRAM 的存储能力，典型值为 18Kb。所列公式只表示对应耗费的最小 BRAM 资源数，主要原因如下：在不同维度展开形成的加速硬件架构需要同时从/向不同数目的缓冲中读取/写入数据，而 BRAM 本身的读写接口数是有限的（不超过 4 个），所以需要多个独立的子缓冲共同组成一个大缓冲，由此也会导致 BRAM 资源耗费的增加。其次，考虑到常使用乒乓缓冲来掩盖延时等因素，实际 BRAM 耗费需要具体设计具体分析。此外，如果所需缓存的数据量相对于单一 BRAM 的存储能力来说过小（如只使用了单一 BRAM 的 10%），一般会采用 LUT+FF 资源来设计此类小规模缓存。

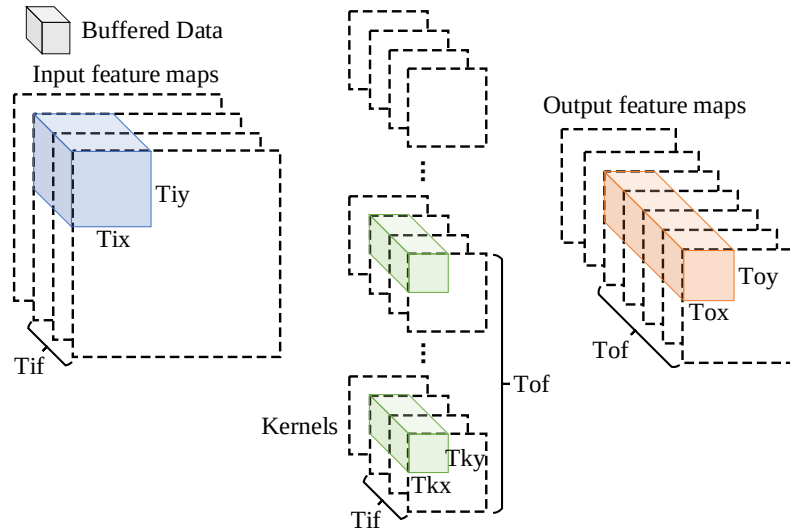


图 3-5 循环分块

循环分块使得整个卷积循环被分为了内外嵌套的两个循环组（内循环与外循环）。内循环的硬件架构主要由不同维度的循环展开度决定，外循环的架构则是由循环分块和

循环交换决定。循环分块的设计参数决定了对应片上缓存的大小，即每次访存的最大数据量；循环交换则决定了数据复用模式，同时也会影响控制输入输出模块的状态机设计。

在此，以第四章在输入特征图数和输出特征图数二维展开的输出特征图复用的卷积层伪代码为例。如图 3-6 所示，其中 I 、 O 、 W 分别代表输入特征图、输出特征图和卷积核参数，一般存放于片外存储 DRAM 中。 I_{buf} 、 O_{buf} 、 W_{buf} 分别代表在 FPGA 加速器上设计的片上输入缓存、输出缓存和参数缓存。偏置参数由于数据量较少，在进行卷积计算前可以全部读取到片上存储 bias 中。

```

I[Nif][(Noy-1)×S+Nky][(Nox-1)×S+Nkx] //input feature maps
O[Noi][Noy][Nox] //output feature maps
W[Noi][Nif][Nky][Nkx] //weights
Ibuf[Tif][(Toy-1)×S+Tky][(Tox-1)×S+Tkx] //on-chip input buffers
Obuf[Tof][Toy][Tox] //on-chip output buffers
Wbuf[Tof][Tif][Nky][Nkx] //on-chip weight buffers
for (r=0;r<Noy;r+=Toy)
  for (c=0;c<Nox;c+=Tox)
    for (m=0;m<Noi;m+=Tof){
      for (n=0;n<Nif;n+=Tif){
        Ibuf = I[n:n+Tif][r×S:(r+Toy-1)×S+Tky][c×S:(c+Tox-1)×S+Tkx]
        Wbuf = W[m:m+Tof][n:n+Tif]
        for (ky=0;ky<Nky;ky++)
          for (kx=0;kx<Nkx;kx++)
            for (tr=0;tr+r<min(Noy,Toy+r);tr++)
              for (tc=0;tc+c<min(Nox,Tox+c);tc++) #PIPELINE
                for (tm=0;tm<Pof;tm++) #UNROLL
                  for (tn=0;tn<Pif;tn++) #UNROLL
                    Obuf[tm][tr][tc] =
                      (kx==0&&ky==0&&tn==0&&n==0)?bias[tm]:Obuf[tm][tr][tc]
                      + Ibuf[tn][tr×S+ky][tc×S+kx]×Wbuf[tm][tn][ky][kx];
              }
            O[m:m+Tof][r:r+Toy][c:c+Tox]=Obuf
      }
    }
  }

```

Outer-Loop

Inner-Loop

图 3-6 卷积分块与展开伪代码

外循环（Outer-Loop）由输出特征图维度、输出特征图数维度和输入特征图数维度顺序组成，按外循环的给定顺序设计状态机控制片外存储与片上缓存间的数据交互。由于采用输出特征图复用的数据复用模式，所有的中间结果都保存在片上缓存 O_{buf} 中，只有最终结果才会写回片外。外循环状态机控制输入输出模块（包括输入特征图读取模块，卷积核参数读取模块和输出特征图写回模块），根据循环分块的硬件分块参数从片外存储中读取输入特征图像素块和卷积核参数或从片上输出缓存将输出特征图像素块写回片外。因此，产生对应的访存时延和 DMA 缓冲与片上缓存的传输时延。

内循环（Inner-Loop）将展开的维度放到了循环最内层，而其他未展开的维度则在外层，实际是设计对应状态机按内循环的外层对卷积循环展开形成的硬件计算架构进行控制和流水复用。

此时，主要的硬件设计参数满足以下约束如下： $Pkx=Pky=Pox=Poy=1$ ， $Tkx=Nkx$ ， $Tky=Nky$ ， $Pif=Tif$ ， $Pof=Tof$ 。

3.3 循环交换

循环交换是通过改变卷积循环的嵌套顺序以达到改变数据复用模式的目的。主要的循环交换方式分为以下 3 类：输入特征图复用、输出特征图复用和卷积核复用。

(1) 输入特征图复用模式 (Input feature maps Reuse mode, IR)

```

for (r=0;r<Noy;r+=Toy)
  for (c=0;c<Nox;c+=Tox) } Loop-3
    for (n=0;n<Nif;n+=Tif){ \\Loop-2
      从片外读取IFM到输入缓存
      for (m=0;m<Nof;m+=Tof){ \\Loop-4
        从片外读取部分和到输出缓存
        从片外读取卷积核权重到权重缓存
        计算部分和写入局部寄存器或输出缓存
        将部分和或最终结果从输出缓存写回片外
      }
    }
  }

```

图 3-7 输入特征图复用伪代码

如图 3-7 所示, 外循环按 Loop-3 (输出特征图宽/高), Loop-2 (输入特征图数), Loop-4 (输出特征图数) 顺序嵌套, Loop-1 (卷积核宽/高) 放入内循环 (将 Loop-1 放入内循环, 主要是因为当前主流卷积核较小如 1×1 或 3×3 , 可以全部读取到片上缓存, 即 $Nkx=Tkx$ 且 $Nky=Tky$)。此时, 尽可能地复用输入特征图, 每张输入特征图只从片外存储中读入片上缓存 1 次; 相应的, 必须从片外存储读取卷积核参数 ($\lceil \frac{Noy}{Toy} \rceil \times \lceil \frac{Nox}{Tox} \rceil$) 次, 读取和写回输出特征图部分和各 $\lceil \frac{Nif}{Tif} \rceil$ 次。伪代码中标红的部分是参与输入特征图复用的部分。对应的硬件结构如图 3-8 所示:

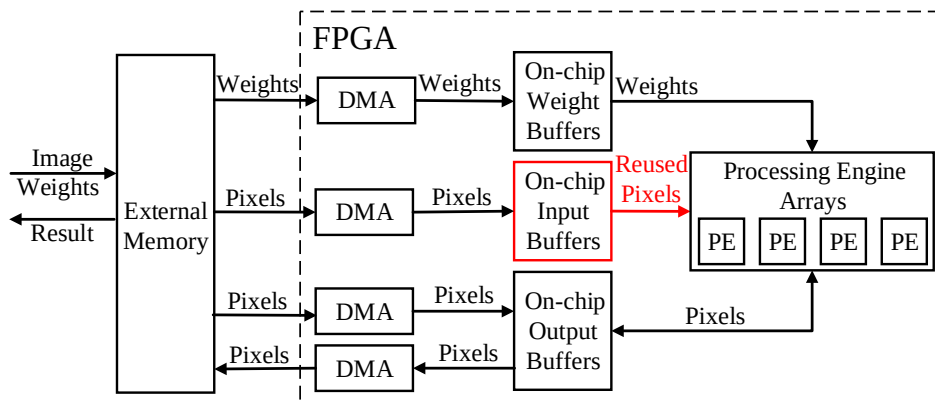


图 3-8 输入特征图复用的硬件结构

卷积核权重与输入特征图像素只需从片外存储读取到片上缓存, 所以各有 1 个对应的单向 DMA; 输出特征图的部分和需要写回片外存储中, 所以需要 2 个 DMA, 图中标红的部分即复用的片上输入特征图像素块。

以下就输入特征图复用模式下, 各模块的时延与卷积层的总时延进行建模。此处将图 3-7 中的 5 个处理过程视为 5 个模块, 分别为: 输入特征图读取模块、部分和读取模块、卷积核读取模块、计算模块与部分和写回模块。

Loop-2 循环内的输入特征图读取模块的时延 $\text{Latency}_{\text{Load IFM}}$ 由两部分组成:

(1) 通过 DMA 从片外读取 Tif 张输入特征图像 $Tiy \times Tix$ 大小的像素块到片上的访存时延 $Latency_{Load IFM M2DMA}$:

$$Latency_{Load IFM M2DMA} = Tif \times Tiy \times MM(Tix, Burst Length_{max}) \quad (3-9)$$

(2) 将输入特征图像像素块传输到片上缓存的传输时延 $Latency_{Load IFM DMA2B}$:

$$Latency_{Load IFM DMA2B} = Tif \times Tiy \times Tix \quad (3-10)$$

若不考虑其他对时延的优化策略, Loop-2 循环内输入特征图读取时延为:

$$Latency_{Load IFM} = Latency_{Load IFM M2DMA} + Latency_{Load IFM DMA2B} \quad (3-11)$$

Loop-4 循环内的部分和读取模块的时延 $Latency_{Load PSUM}$ 由两部分组成:

(1) 通过 DMA 从片外读取 Tof 张输出特征图的 $Toy \times Tox$ 大小的部分和到片上的访存时延 $Latency_{Load PSUM M2DMA}$:

$$Latency_{Load PSUM M2DMA} = Tof \times Toy \times MM(Tox, Burst Length_{max}) \quad (3-12)$$

(2) 将部分和传输到片上缓存的传输时延 $Latency_{Load PSUM DMA2B}$:

$$Latency_{Load PSUM DMA2B} = Tof \times Toy \times Tox \quad (3-13)$$

此时, Loop-4 循环内部分和的读取时延为:

$$Latency_{Load PSUM} = Latency_{Load PSUM M2DMA} + Latency_{Load PSUM DMA2B} \quad (3-14)$$

Loop-4 循环内的卷积核读取模块的时延 $Latency_{Load W}$ 由两部分组成:

(1) 通过 DMA 从片外读取 Tof 个卷积核对于 Tif 张输入特征图的 $Nky \times Nkx$ 个权重参数到片上的访存时延 $Latency_{Load W M2DMA}$:

$$Latency_{Load W M2DMA} = Tof \times MM(Tif \times Nkx \times Nky, Burst Length_{max}) \quad (3-15)$$

(2) 将卷积核传输到片上缓存的传输时延 $Latency_{Load W DMA2B}$:

$$Latency_{Load W DMA2B} = Tof \times Tif \times Nkx \times Nky \quad (3-16)$$

Loop-4 循环内部分和的读取时延为:

$$Latency_{Load W} = Latency_{Load W M2DMA} + Latency_{Load W DMA2B} \quad (3-17)$$

Loop-4 循环内计算模块的计算时延 $Latency_{Compute}$ 为:

$$Latency_{Compute} = \left\lfloor \frac{Tof}{Pof} \right\rfloor \times \left\lfloor \frac{Tif}{Pif} \right\rfloor \times \left\lfloor \frac{Tkx}{Pkx} \right\rfloor \times \left\lfloor \frac{Tky}{Pky} \right\rfloor \times \left\lfloor \frac{Tox}{Pox} \right\rfloor \times \left\lfloor \frac{Toy}{Poy} \right\rfloor \quad (3-18)$$

表示通过对各维展开形成的硬件加速结构流水复用来实现所需的卷积计算。

Loop-4 循环内的部分和写回模块的时延 $Latency_{Store PSUM}$ 由两部分组成:

(1) 将部分和传输到 DMA 的传输时延 $Latency_{Store PSUM B2DMA}$:

$$Latency_{Store PSUM B2DMA} = Tof \times Toy \times Tox \quad (3-19)$$

(2) 通过 DMA 将 Tof 张输出特征图的 $Toy \times Tox$ 大小的部分和写回片外的访存时延 $Latency_{Store PSUM DMA2M}$:

$$Latency_{Store PSUM DMA2M} = Tof \times Toy \times MM(Tox, Burst Length_{max}) \quad (3-20)$$

Loop-4 循环内的部分和写回模块的时延 $Latency_{Store PSUM}$:

$$Latency_{Store PSUM} = Latency_{Store PSUM B2DMA} + Latency_{Store PSUM DMA2M} \quad (3-21)$$

在无流水与其他优化的情况下, 此时计算某卷积层的总时延为:

$$Latency = \left\lfloor \frac{Noy}{Toy} \right\rfloor \times \left\lfloor \frac{Nox}{Tox} \right\rfloor \times \left\lfloor \frac{Nif}{Tif} \right\rfloor \times \left\{ Latency_{Load IFM} + \left\lfloor \frac{Nof}{Tof} \right\rfloor \times (Latency_{Load PSUM} + Latency_{Load W} + Latency_{Compute} + Latency_{Store PSUM}) \right\} \quad (3-21)$$

(2) 输出特征图复用模式 (Output feature maps Reuse mode, OR)

```

for (m=0;m<Nof;m+=Tof){ \Loop-4
  for (r=0;r<Noy;r+=Toy)
    for (c=0;c<Nox;c+=Tox) } Loop-3
    for (n=0;n<Nif;n+=Tif){ \Loop-2
      从片外读取IFM到输入缓存
      从片外读取权重参数到权重缓存
      计算部分和，写回局部寄存器或输出缓存
    }
  }
}
最终结果OFM从输出缓存写回片外

```

图 3-9 输出特征图复用伪代码

如图 3-9 所示，输出特征图复用模式下外循环按 Loop-4, Loop-3, Loop-2 顺序嵌套，Loop-1 放入内循环；也可以按 Loop-3, Loop-4, Loop-2 顺序嵌套。此时，尽可能地复用输出特征图部分和，将输出特征图部分和存储在片上缓存或局部寄存器中，所有的输出特征图像素只有得到最终结果后才会写回片外存储 1 次；相应的，必须从片外存储读取卷积核参数 ($\lceil \frac{Noy}{Toy} \rceil \times \lceil \frac{Nox}{Tox} \rceil$) 次，输入特征图 $\lceil \frac{Nof}{Tof} \rceil$ 次。伪代码中的标红部分是参与输出特征图复用的部分。

对应的硬件结构如图 3-10 所示：

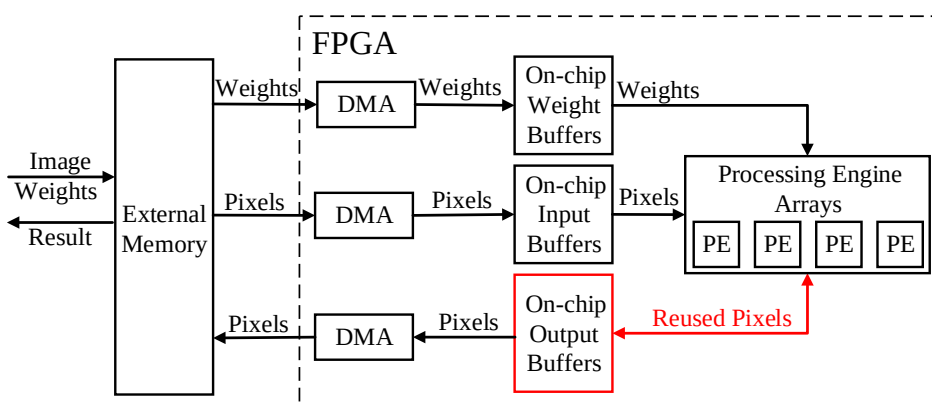


图 3-10 输出特征图复用硬件架构

此时，由于卷积核权重与输入特征图像素只需从片外存储中读入片上缓存，所以各有 1 个对应的 DMA；输出特征图的部分和存储在片上缓存或局部寄存器中，只需写回片外 1 次，所以也只需要 1 个 DMA，标红部分即被复用的片上输出特征图部分和。

输出特征图复用模式下，将图 3-9 中的 4 个处理过程视为 4 个模块分别为：输入特征图读取模块、卷积核读取模块、计算模块与输出特征图写回模块。由于输出复用模块下的模块实际与输入复用模式下基本类似，所以此处只列出计算某卷积层的总时延：

$$\text{Latency} = \left\lceil \frac{Noy}{Toy} \right\rceil \times \left\lceil \frac{Nox}{Tox} \right\rceil \times \left\lceil \frac{Nof}{Tof} \right\rceil \times \left\{ \text{Latency}_{\text{Store OFM}} + \left\lceil \frac{Nif}{Tif} \right\rceil \times (\text{Latency}_{\text{Load IFM}} + \text{Latency}_{\text{Load W}} + \text{Latency}_{\text{Compute}}) \right\} \quad (3-22)$$

其中，输出特征图写回模块的时延 $\text{Latency}_{\text{Store OFM}}$ 与输入复用模式中的部分和写回模块的时延 $\text{Latency}_{\text{Store PSUM}}$ 类似。

(3) 权重复用 (Weight Reuse mode, WR)

如图 3-11 所示, 卷积核权重复用模式下将外循环按 Loop-2, Loop-4, Loop-3 顺序嵌套, Loop-1 放入内循环。此时, 尽可能地复用卷积核权重, 所有的卷积核权重只从片外存储中读入片上缓存 1 次; 相应的, 必须从片外读取输入特征图 $\left\lceil \frac{Nof}{Tof} \right\rceil$ 次, 输出特征图部分和 $\left\lceil \frac{Nif}{Tif} \right\rceil$ 次。同时, 还需要将输出特征图写回 $\left\lceil \frac{Nif}{Tif} \right\rceil$ 次。伪代码中标红的部分是参与权重复用的部分。

```

for (m=0;m<Nof;m+=Tof)\\Loop-2
  for (n=0;n<Nif;n+=Tif){\\Loop-4
    从片外读取权重参数到权重缓存
    for (r=0;r<Noy;r+=Toy)
      for (c=0;c<Nox;c+=Tox){ } Loop-3
      从片外读取IFM到输入缓存
      从片外读取部分和到输出缓存
      计算部分和, 写回局部寄存器或输出缓存
      将部分和或OFM最终结果从输出缓存写回片外
    }
  }

```

图 3-11 权重复用伪代码

对应的硬件结构如图 3-12 所示:

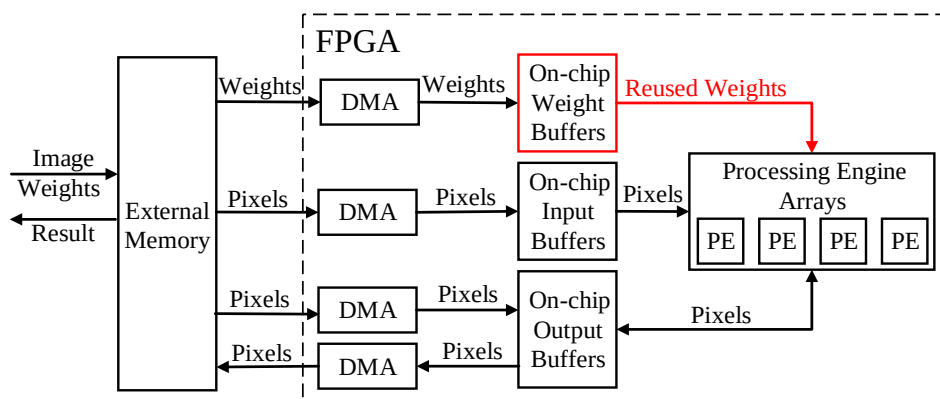


图 3-12 权重复用硬件架构

由于卷积核权重与输入特征图像素只需从片外存储中读入片上缓存, 所以各有对应的 1 个单向 DMA; 输出特征图的部分和需要写回片外存储中, 所以需要 2 个 DMA, 图中标红部分即复用的片上卷积核权重的部分。

卷积核复用模式下, 将图 3-11 中的 5 个处理过程视为 5 个模块分别为: 输入特征图读取模块、部分和读取模块、卷积核读取模块、计算模块与部分和写回模块。

由于卷积核复用模式下的模块与输入复用模式下基本类似, 只列出计算某卷积层的总时延:

$$\text{Latency} = \left\lceil \frac{Nof}{Tof} \right\rceil \times \left\lceil \frac{Nif}{Tif} \right\rceil \times \left\{ \text{Latency}_{\text{Load } W} + \left\lceil \frac{Noy}{Toy} \right\rceil \times \left\lceil \frac{Nox}{Tox} \right\rceil \times (\text{Latency}_{\text{Load IFM}} + \text{Latency}_{\text{Load PSUM}} + \text{Latency}_{\text{Compute}} + \text{Latency}_{\text{Store PSUM}}) \right\} \quad (3-23)$$

通过对循环交换产生的 3 种不同数据模式的分析可知,不同数据模式下的各模块基本类似,只是由于不同的数据复用导致各模块分时复用的次数发生了变化,相应的也导致不同数据在三层存储架构中的搬运次数随着改变,也就影响到了访存时延和传输时延。另一方面,这里只是讨论最简单的延时模型,关于各延时间如何通过乒乓缓冲掩盖重叠、如何缩短传输时延等等减少加速器时延的优化策略,为了不增加本章对规模可伸缩性分析的复杂性,放到了第四章设计与评估中。

3.4 加速器的设计空间探索与评估

设计空间即在给定某具体 CNN 网络结构,相应 FPGA 资源数和片外存储带宽的约束,找到满足约束的 CNN 加速器的设计参数与数据复用模式。由前 3 节可知,通过硬件设计参数和数据复用模式可以描述出一个基于 SIMD 架构的 CNN 加速器,对加速器的设计空间的探索,实质是在给定约束条件下,求可行解空间。设计者往往会添加额外的约束来寻找某种最优解或者次优解,当前的设计空间探索常使用穷举法、遗传算法、模拟退火算法等来寻找最优解或次优解^{[52][54][55]}。

当前研究没有涉及到多 FPGA 间的加速器规模可伸缩性,故对加速器的规模可伸缩性的研究等价于对加速器的设计空间探索。在此,给出以穷举遍历进行设计空间探索寻找时延最小的加速器的算法。

算法 3-1 穷举法寻找时延最小的加速器设计

输入:(1)网络结构 $CNN_{Related}$ 与网络计算量 OP_{Total}

(2)FPGA 最大资源数 N_{Res} (N_{LUT} 、 N_{DSP} 、 N_{BRAM} 、 N_{FF}) 与工作时钟频率 CLK

(3)片外存储的最大带宽 $Bandwidth_{Max}$

(4)数据精度 $Datawidth$

输出: 时延最短的加速器对应硬件设计参数与数据复用模式

算法:

初始化 Step=0.1, Target=1.0

计算最小时延 $Latency_{Min} = OP_{Total} / (Perf(Datawidth, N_{Resource}) \times CLK)$

While(1)

计算目标时延 $Latency_{Target} = Latency_{Min} / Target$

返回满足展开约束的集合 A 与剩余 FPGA 资源 N'_{Res}

$\{A, NSet'_{Res}\} = OptimizeCompute(CNN_{Related}, N_{Res}, Latency_{Target})$

If($A = \emptyset$)

Target = Target - Step

Continue

Else

返回满足带宽限制与片上资源限制的集合 A'

$A' = OptimizeMemory(CNN_{Related}, A, NSet'_{Res}, Bandwidth_{Max}, Latency_{Target})$

If($A = \emptyset$)

Target = Target - Step

Continue

Else

Return PickBest(A')

其中, $Perf(Datawidth, N_{Resource})$ 表示在给定数据精度与 FPGA 资源的情况下, 不考虑数据相关性等因素, 能达到的理论最大性能。所以, $Latency_{Min}$ 实际是无法达到的时延下界。整个 While 循环实际就是不断增大目标时延 $Latency_{Target}$, 寻找能找到满足目标时延的设计的过程。OptimizeCompute() 函数确定硬件设计参数中的各维展开度 P^* , 寻找计算时延小于目标时延的设计, 返回的 A 与 $NSet'_{Res}$ 分别对应满足条件的设计的列表, 对应设计剩余的 FPGA 资源的列表。OptimizeMemory() 函数寻找满足片外带宽限制, 并且总时延小于目标时延的设计列表 A' , 其中包含硬件设计参数中的分块参考 T^* 以及数据复用模式。最后, 由于可能有多个设计满足条件, 可以通过 PickBest() 函数再按某些特定约束从中挑选出满足的设计。

对于找到的多个设计参数组合, 可通过 2.3.5 节中所介绍的 Roofline 模型来进行评估, 此时对应加速器设计的计算密度 I 和性能 P 可通过以下公式得到:

$$I = Model\ Operations / N_{MA}^{total} \quad (3-24)$$

$$P = Model\ Operations / Latency^{total} \quad (3-25)$$

其中, N_{MA}^{total} 表示加速器访存的数据量 (读取和写回), $Latency^{total}$ 表示当前加速器的总时延。可画出 Roofline 模型进行评估 (图 3-13 与图 4-11 相同), 如图 3-13 所示:

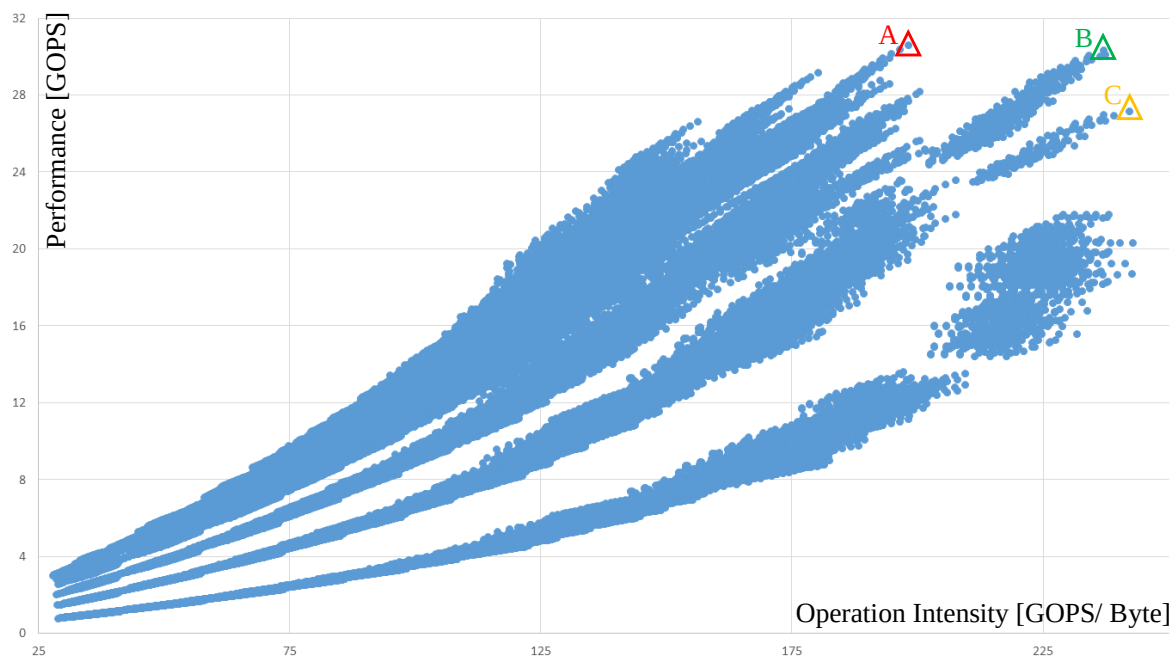


图 3-13 加速器设计空间探索

图中的每个点都表示一种满足约束的硬件设计参数组合; 与此对应的, 就是不同的加速器设计。其中, 横坐标表示计算密度, 即单位字节访存对应的计算量; 纵坐标表示性能。在给定 CNN 模型的条件 (即模型计算量为常数), 如果设计的计算密度越大, 则说明访存次数越少, 能耗越小; 计算性能越高, 表示加速器的性能越好。以图 3-13 中的设计点 A、B、C 为例, 点 A 对应的设计性能最优, 但是计算密度最低, 能耗大; 点 C 对应的设计能耗最小, 但是性能最差; 点 B 对应的设计性能较高同时能耗较小, 实际

就是能效最好的设计点。对于同一 CNN 模型，通过改变设计约束（主要是 FPGA 资源数、总线带宽），就可实现对不同规模的 FPGA 芯片上的加速器的可伸缩性。

3.5 本章小结

本章主要就基于 SIMD 架构的 CNN 加速器的规模可伸缩性进行了量化研究。首先，分析了 4 种基本的循环展开与对应的硬件架构，给出了对应循环展开下所需的资源。其次，分析循环分块与对应的片上缓存的影响，循环分块将卷积循环分为内外两个循环组，外循环负责片上缓存与片外存储的数据交互，内循环对循环展开形成的硬件加速架构进行控制并实现卷积运算。然后，分析了由循环交换产生的 3 种数据复用模式与对应的时延模型；最后，给出一种在资源受限情况下，搜索时延最小加速器的设计空间探索模板。

第四章 基于FPGA的SIMD二维展开的CNN加速器设计与评估

4.1 从CNN模型到FPGA加速器构建

主要描述从 CNN 模型到 FPGA 加速的完整流程，如图 4-1 所示：

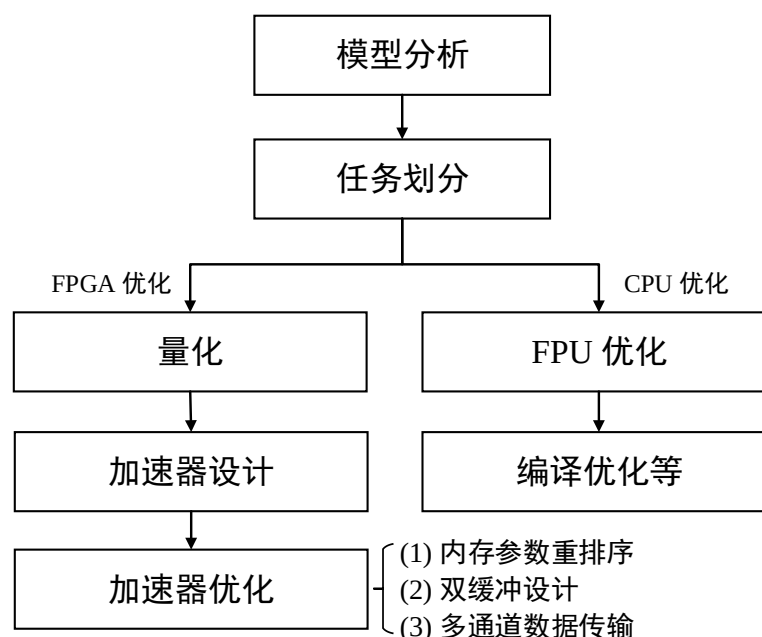


图 4-1 从 CNN 模型到 FPGA 加速的流程图

(1)模型分析与任务划分：首先，需要对模型进行深入分析，理解模型算法的原理和数据流。其次，根据模型在不同阶段所做的运算，考虑 FPGA 和 CPU 的特点，将任务分配到 FPGA 和 CPU。CPU 适合做控制性、串行、计算量较少的任务，而 FPGA 适合做并行度高、计算密集型的任务。

(2)针对分配到 FPGA 上的任务：量化、加速器设计、优化策略优化等。通过量化来简化计算复杂度，使得单位计算单元所耗费的资源大大减少，从而可以在相同的资源中，设计更多的并行计算单元，提高加速器性能。优化策略包括：参数重排序、双缓冲设计和多通道数据传输。

(3)针对分配到 CPU 上的任务：可利用 CPU 的浮点运算单元来加快浮点计算。同时，在编译时可采用编译优化加速任务处理。另外，还有诸如多线程并行、cache 缓存优化等优化策略。基于 CPU 的性能优化已非常成熟，这里不再详细叙述。

4.1.1 YOLOv2 任务划分

模型分析部分在 YOLOv2 相关概念中已经介绍，这里不再重复。

由于图像预处理和后处理部分计算量不大且在图像处理总延时中占比极少，也考虑到后处理涉及指数计算，FPGA 实现指数函数的拟合的资源耗费较大，所以将图像的预处理和后处理部分，放在 CPU 端处理。

YOLOv2 网络中一共涉及 4 类层：卷积层、池化层、路由层和重排序层。卷积层占据模型的大部分计算量，现有研究表明使用 FPGA 加速卷积是非常有效的^[16-39]。池化与

卷积类似，使用比较运算代替乘加运算，所以将卷积层和池化层的处理放在 FPGA 端。路由层可以通过预先设置内存偏移地址来实现。重排序层的抽样重排与 DMA 类似，但与传统 DMA 不同，所以也在 FPGA 端处理。

4.1.2 动态定点 16 位量化

CNN 对于数据精度有很强的鲁棒性，在保持准确度不变的前提下，可以通过降低数据位宽来减少计算所耗费的资源^[33]。随着数据位宽降低，传输数据量也随之降低。本文采用类似[29]和[56]的方法对卷积核权重和输入输出特征图动态定点 16 位量化。

定点数 x_{fixed} 可以由以下公式表示：

$$x_{fixed} = \sum_{i=0}^{bw-1} B_i * 2^{-exp} * 2^i \quad (4-1)$$

其中 bw 表示 x_{fixed} 的位宽， exp 表示定点数的阶码， $B_i \in \{0,1\}$ 。定点数 x_{fixed} 采用补码表示，最高位为符号位。

浮点数 x_{float} 与定点数 x_{fixed} 的相互转化如下：

$$x_{fixed} = (\text{int})(x_{float} * 2^{bw}) \quad (4-2)$$

$$x_{float} = (\text{float})x_{fixed} * 2^{-bw} \quad (4-3)$$

由于定点数位宽有限并且阶码固定。所以将 x_{float} 转换为 x_{fixed} ，再转换回新浮点数 x'_{float} 后，可能会有精度损失。

动态定点数据量化由 3 步组成，分别找到使整个网络在量化后的权重参数、输出特征图像素和中间结果与原始值相比误差最小的各层量化参数，每一步都可以看做求解一个贪心问题，转化为求针对每层的绝对误差和最小：

(1) 权重参数量化

首先，根据如下公式找到每层的最优 exp_w ：

$$exp_w = \underset{exp_w}{\operatorname{argmin}} \sum |W_{float} - W(bw, exp_w)| \quad (4-4)$$

W_{float} 代表某层的任意权重参数原始浮点值， $W(bw, exp_w)$ 表示在给定位宽 bw 和阶码 exp_w 下将 W_{float} 定点化后转换回浮点的新浮点数 W'_{float} 。希望找到在给定位宽 bw 的条件下，和原权重绝对误差和最小的阶码 exp_w 。偏置参数的量化与其类似，在此不过多叙述。

(2) 输入输出特征图量化

按网络的前向顺序对输入输出特征图进行量化，使得每一层的输出特征图像素有共同的阶码 exp_{out} 。阶码 exp_{out} 满足以下公式：

$$exp_o = \underset{exp_o}{\operatorname{argmin}} \sum |Px_{float} - \text{Out}(bw, exp_o)| \quad (4-5)$$

其中， Px_{float} 表示某层的原输出特征图浮点像素值， $\text{Out}(bw, exp_o)$ 表示给定位宽 bw 和阶码 exp_o 下定点化后转换回浮点的浮点数 Px'_{float} 。其余与权重参数量化类似。由于在预处理阶段将输入的 RGB 像素值转化到 [0,1] 区间，所以当位宽 bw 为 16 时，将第一层的输入特征图量化阶码定为 14。

(3) 中间结果量化

前两步的量化已将输入输出特征图和权重进行定点化。但是，在进行卷积乘加时，还是先转换成浮点进行计算，并且中间结果以浮点存储。

通过量化中间结果可以用定点数的乘加和移位运算来拟合浮点乘加运算，使得推断过程除预处理和后处理阶段外，在 FPGA 端都进行定点运算。中间结果的阶码 exp_{inter} 满足以下公式：

$$exp_{inter} = \underset{exp_{inter}}{\operatorname{argmin}} \sum |Inter_{float} - Inter(bw, exp_{inter})| \quad (4-6)$$

另外，使用动态定点量化时，激活函数 Leaky ReLU 也要进行相应的量化操作。当位宽 bw 为 16 时，使用与 $0xccc$ 相乘和右移 15 位的定点运算来拟合与 0.1 相乘的操作。量化后的 Leaky ReLU 如下所示：

$$f'(x) = (x < 0)? (x * 0xccc) \gg 15: x \quad (4-7)$$

4.2 基于SIMD架构的CNN加速器的设计与优化

除了路由层外，其他层都是将上一层的输出作为当前层的输入，输出结果作为下一层的输入，串行顺序处理。将路由层的功能通过预先设置存取地址偏移实现后，FPGA 加速器只需根据相关参数读取内存数据、处理数据和将数据写回内存，如图 4-2 (a)所示：

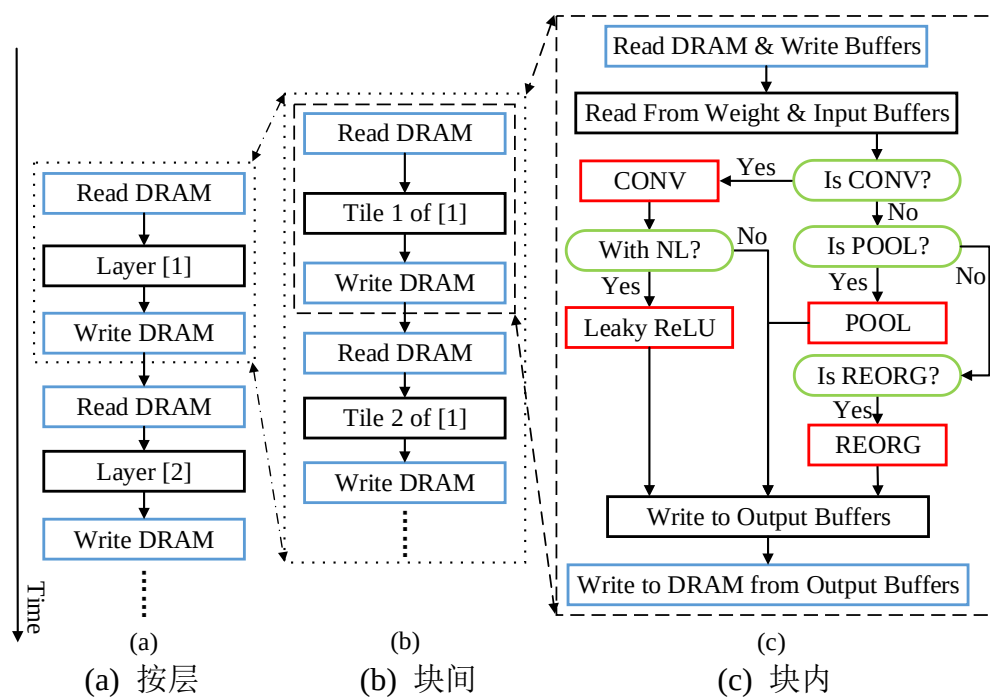


图 4-2 执行调度流

如图 4-2(b)所示，每次读取或写入多块数据，对 FPGA 片上缓存的数据块复用结束后，再进行下一次读取，保证每块数据只从片外读取少量次数以减少访存次数和传输数据量。对于每次取到的分块数据，根据图 4-2(c)的控制流处理：从片外读取分块数据到输入缓存，根据层的类别交由 CONV、POOL、REORG 模块进行处理。处理后的数据写回输出缓冲，待得到最终结果后写回片外存储。

考虑到三种数据复用模式所涉及的模块基本类似，这里选择输出特征图复用模式下的加速器设计进行设计与评估，另外两种复用模式不再过多叙述。

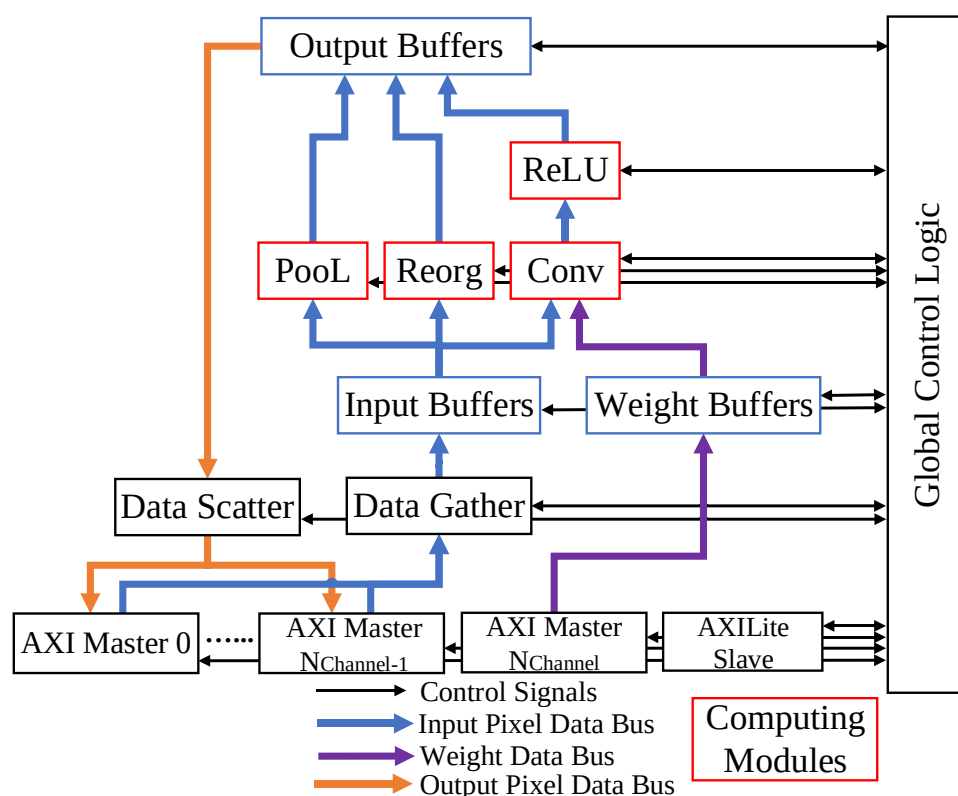


图 4-3 加速器总体架构

加速器总体架构如图 4-3 所示，加速器对外有多个 AXI Master 接口和一个 AXILite Slave 接口。通过 AXILite Slave 接口读写控制、数据和状态寄存器。多个 AXI Master 接口并发读取输入特征图和写回输出特征图，一个 AXI Master 接口负责读取每层权重参数。

Data Scatter 模块负责生成对应写入地址，并将从片外读到的输入特征图像素块分发到片上缓存。Data Gather 负责生成写回地址，将输出缓存中的输出特征图像素块写回片外。红色处理模块分别负责卷积层（CONV 和 Leaky ReLU）、最大池化层（POOL）及重排序层（REORG）的处理。用于读写特征图的 AXI Master 接口数由以下公式决定：

$$N_{\text{Channel}} = \text{Max}(N_{\text{Cin}}, N_{\text{Cout}}) \quad (4-8)$$

其中 N_{Cin} 表示并发读取特征图的 AXI Master 接口数， N_{Cout} 表示并发写回 DRAM 的 AXI Master 接口数。由于 AXI4 协议本身读写通道独立，读写操作并发，所以实际用于数据传输的 AXI Master 接口数由并发通道多的一侧决定。再加上用于权重读取的接口，实际 AXI Master 接口数 $N_{\text{AXIM}} = N_{\text{Channel}} + 1$ 。

另一方面，HLS 生成的 AXI4 Master 接口模块内部读写通道都拥有类似 DMA 的硬件结构，所以可以看做 $(N_{\text{Cin}} + N_{\text{Cout}} + 1)$ 个独立的 DMA 并发访存。

4.2.1 输入输出模块

由于 AXI4 Master 接口读写通道独立，所以读写操作可以并发执行。通过读通道从 DRAM 读取数据，由数据分发 (Data Scatter) 模块控制多个读通道的 DMA 完成输入特征图像素块的读取，并将读取的数据块分发到对应输入缓存。由数据收集 (Data Gather)

模块将输出缓存中的像素块写回 DRAM，控制多个写通道的 DMA 完成输出特征图像素块的写入。如图 4-4 所示：

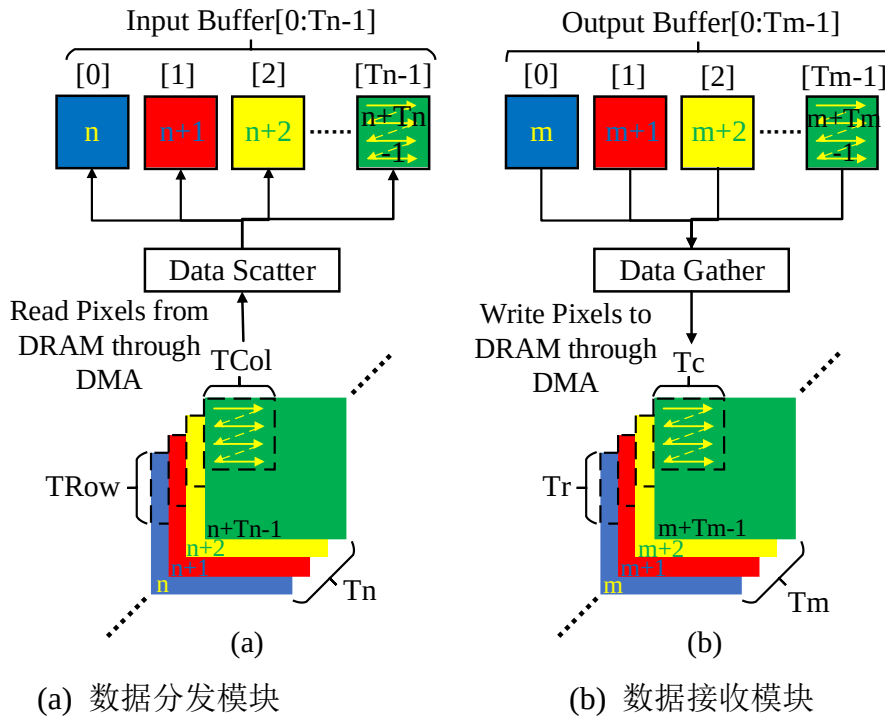


图 4-4 单通道数据传输

如图 4-4(a)所示，数据分发模块从内存顺序读取 T_{if} 张 $T_{iy} \times T_{ix}$ 大小的输入特征图像素块，每个像素块按行优先顺序读取并分发到 T_{if} 个独立的片上输入缓存。如图 4-4 (b) 所示，数据收集模块从 T_{of} 个独立的片上输出缓存中顺序读取 $T_{oy} \times T_{ox}$ 大小的像素块写回片外。

为了提高计算单元的效率，在数据分发和收集模块中还添加了额外的行缓冲，并且进行双缓冲设计。输入输出模块的双缓冲时序如图 4-5 所示：

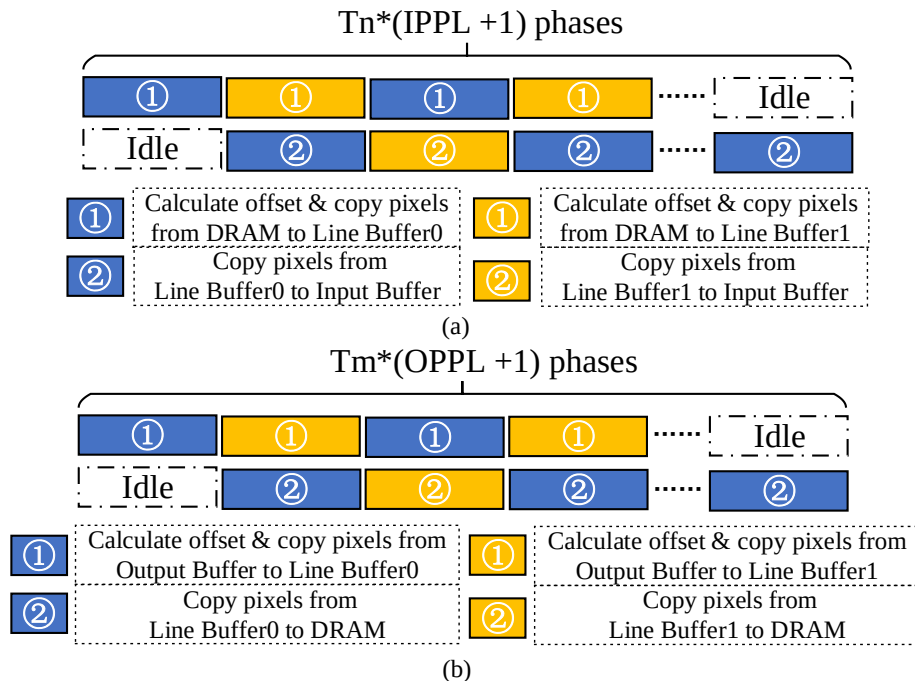


图 4-5 双缓冲设计时序图 (a) 输入模块时序 (b) 输出模块时序

1.对于数据分发模块从片外 DRAM 读取输入特征图像素块到片上缓存的过程:

如图 4-5(a)所示,数据分发模块的双缓冲的设计使得(1)访存地址、片上缓存偏移的计算、将输入特征图像素块从 DRAM 读取到行缓冲的时延和(2)像素从行缓冲写入输入缓存的时延重叠,从而减少了数据传输的时延。

输入行缓冲所需的存储资源 $N_{LoadLineBRAM}$ 为:

$$N_{LoadLineBRAM} = \left\lceil (Line\ Buffer\ Size_{Load}) \times \frac{Bitwidth}{C_{oneBRAM}} \right\rceil \times N_{Cin} \times 2 \quad (4-9)$$

行缓冲大小 $Line\ Buffer\ Size_{Load}$ 满足以下约束:

$$Line\ Buffer\ Size_{Load} \geq (Tox_{max} - 1) \times S_{max} + Nkx_{max} \quad (4-10)$$

其中 $C_{oneBRAM}$ 表示单块 BRAM 的存储能力(典型值为 18Kb), $Bitwidth$ 表示当前的数据宽度, Tox_{max} 表示各层中的最大 Tox 值, S_{max} 和 Nkx_{max} 以此类推。 N_{Cin} 指多通道并发输入数,此处无多通道数据传输情况下为 1。

根据片上缓存和特征图的宽度尽可能长地连续传输数据,提高带宽效率。行缓冲每次传输的输入特征图行数 $N_{TiyOneLine}$ 如下所示:

$$N_{TiyOneLine} = \begin{cases} \left\lfloor \frac{Line\ Buffer\ Size_{Load}}{Tix} \right\rfloor, & Tix \geq (Nox - 1) \times S + Nkx \\ 1, & \text{else} \end{cases} \quad (4-11)$$

只有输入特征图的宽度小于 Tix 时,每次行缓冲才会读取连续的多行像素。

(1) 访存地址、片上缓存偏移的计算、将输入特征图像素块从 DRAM 读取到行缓冲的时延 $Latency_{Load1}$ 公式如下:

$$Latency_{Load1} = \left\lceil \frac{Tif}{N_{Cin}} \right\rceil \times \left\{ \frac{Const}{Freq} + MM(N_{TiyOneLine} \times Tix \times \frac{Bitwidth}{Bitwidth_{Interface}}, BurstLength_{Max}) \right\} \quad (4-12)$$

其中 $Const$ 为与加速器模块相关常数(流水初始化时间间隔等), $Freq$ 为加速器工作的时钟频率, $Bitwidth_{Interface}$ 为 AXI Master 接口数据位宽, $BurstLength_{Max}$ 为设置的最大突发长度。

(2) 将行缓冲中的像素写入输入缓存的时延 $Latency_{Load2}$ 如下:

$$Latency_{Load2} = \lceil Tif / N_{Cin} \rceil \times (Const + N_{TRowOneLine} \times Tix) / Freq \quad (4-13)$$

上述两个处理过程双缓冲的循环次数 $Input\ Ping\ Pong\ Loops(IPPL)$ 如下所示:

$$IPPL = \lceil Tiy / N_{TRowOneLine} \rceil \quad (4-14)$$

因此,总输入特征图读取时延 $Latency_{LoadIFM}$:

$$Latency_{LoadIFM} = \frac{Const}{Freq} + Latency_{Load1} + Latency_{Load2} + (IPPL - 1) \times \max(Latency_{Load1}, Latency_{Load2}) \quad (4-15)$$

2.对于数据接收模块从片上输出缓存写出像素块到片外 DRAM 的过程:

如图 10(b)所示,数据接收模块将(1)访存地址、片上缓存偏移的计算、像素从输出缓存搬移到行缓冲的时延和(2)将行缓冲中的像素写回内存的时延重叠。

输出行缓冲所需存储资源 $N_{StoreLineBRAM}$ 为:

$$N_{StoreLineBRAM} = \lceil (Line\ Buffer\ Size_{Store}) \times Bitwidth / C_{oneBRAM} \rceil \times N_{Cout} \times 2 \quad (4-16)$$

N_{Cout} 表示多通道并发输出数, 此处为 1。输出行缓冲大小 $Line\ Buffer\ Size_{Store}$ 满足以下约束:

$$Line\ Buffer\ Size_{Store} \geq Tox_{max} \quad (4-17)$$

行缓冲每次传输实际输出特征图行数 $N_{ToyOneLine}$ 如下所示:

$$N_{ToyOneLine} = \begin{cases} \left\lceil \frac{Line\ Buffer\ Size_{Store}}{Tox} \right\rceil, & Tox \geq Nox \\ 1, & \text{else} \end{cases} \quad (4-18)$$

(1) 访存地址、片上缓存偏移的计算、从输出缓存搬移到行缓冲的时延 $Latency_{Store1}$ 如下:

$$Latency_{Store1} = \lceil Tox / N_{Cout} \rceil \times (Const + N_{ToyOneLine} \times Tox) / Freq \quad (4-19)$$

(2) 将行缓冲中的像素写回内存的时延 $Latency_{Store2}$ 如下所示:

$$Latency_{Store2} = \left\lceil \frac{Tox}{N_{Cout}} \right\rceil \times \left\{ \frac{Const}{Freq} + MM(N_{TrOneLine} \times Tox \times \frac{Bitwidth}{Bitwidth_{Interface}}, Burst\ Length_{Max}) \right\} \quad (4-20)$$

上述两个处理过程双缓冲的循环次数 $Output\ Ping\ Pong\ Loops(OPPL)$ 如下所示:

$$OPPL = \lceil Toy / N_{TrOneLine} \rceil \quad (4-21)$$

因此, 总输出时延 $Latency_{Store}$ 为:

$$Latency_{Store} = \frac{Const}{Freq} + Latency_{Store1} + Latency_{Store2} + (OPPL - 1) \times \max(Latency_{Store1}, Latency_{Store2}) \quad (4-22)$$

4.2.2 卷积模块

通过对卷积循环中输出特征图数和输入特征图数两维进行部分展开, 形成 $Pof \times Pif$ 个并行乘法计算单元和 Pof 个 $\lceil \log_2 Pif \rceil$ 深度的加法树, 流水处理乘加计算^[20]。其中, $Pof=Tox$, $Tif=Pif$ 。以 $Tox=2, Tif=4$ 为例, 如图 4-6 所示:

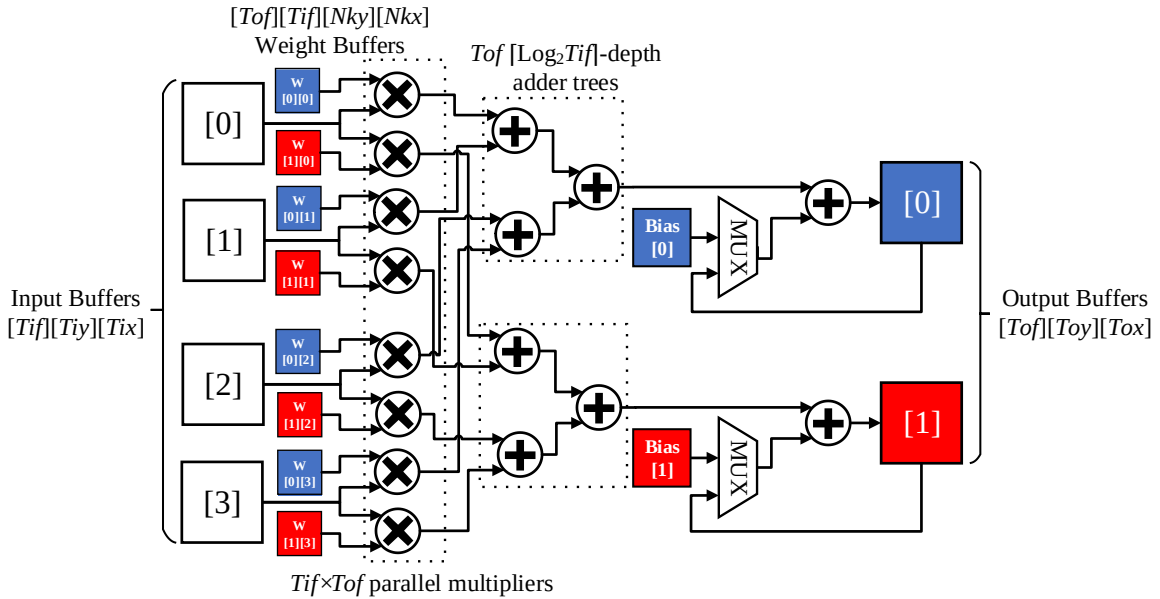


图 4-6 卷积模块 (Tif=4, Tox=2)

充满流水线后, 每个时钟周期卷积模块从对应的 Tif 个独立输入特征图缓存中读入 Tif 个相同位置的像素, 同时也从 $Tox \times Tif$ 个独立的卷积核缓存中读入相同位置的权重,

$Tof \times Tif$ 个并行乘法单元复用 Tif 个输入像素进行乘法计算。 Tof 个加法树将乘积两两相加，得到的结果和部分和累加后，写回对应输出缓存中。

卷积计算对应的时延为：

$$Latency_{Compute} = (Const + Nky \times Nkx \times Toy \times Tox) / Freq \quad (4-23)$$

同时，卷积模块的资源大部分用于乘法器和加法器的设计。而单一乘法器和加法器的资源耗费又与数据精度有关。由[7]和实际评估可知，乘法器和加法器主要耗费 DSP 和 LUT 资源。不同精度下，乘法器和加法器的资源耗费如表 4-1 所示：

表 4-1 资源耗费比较

	DSP	LUT
Adder (Float-32)	2	214
Mul. (Float-32)	3	135
Adder (Fixed-16)	-	47
Mul.&Shift(Fixed-16)	1	101

浮点 32 位精度时，实现单个乘法器和加法器的 DSP 和 LUT 的耗费很大。定点 16 位时，加法器只耗费少量 LUT，乘法器和移位电路耗费的资源与浮点 32 位相比减少了很多。因此，在保证数据精度的情况下，定点计算是很有优势的。

在输出并行度为 Tof ，输入并行度为 Tif 的条件下，卷积模块的主要 DSP 资源耗费 N_{DSP}^{Conv} 和 LUT 资源耗费 N_{LUT}^{Conv} 如下所示：

$$N_{DSP}^{Conv} = Tof \times Tif \times (Cost_{MulDSP} + Cost_{AddDSP}) \quad (4-24)$$

$$N_{LUT}^{Conv} = Tof \times Tif \times (Cost_{MulLUT} + Cost_{AddLUT}) + Tof \times Cost_{Select} \quad (4-25)$$

$Cost_{Mul}$ 和 $Cost_{Add}$ 分别对应不同精度下，乘法器和加法器的对应资源耗费 $Cost_{Select}$ 对应图 4-6 中的 2 选 1 选择电路和累加器的 LUT 耗费（约 32 个 LUT）。

4.2.3 池化模块

池化的作用是降低特征图的维数以及减少过拟合。YOLOv2 中的池化层皆为 $Nkx=Nky=S=2$ 的最大池化层。对单一输入特征图，以 2×2 大小且步长 $S=2$ 的窗口滑动，取其中的最大像素作为输出。最大池化操作与卷积操作类似，不同之处在于：

- (1) 只需要对单一输入特征图进行抽样。
- (2) 运算单元不是乘法器和加法器，而是比较器。

每个时钟周期，池化模块从 T_{pool} 个独立的输入特征图缓存中读取相同位置的一个像素与当前最大值比较，同时有 T_{pool} 个比较器在进行不同输入特征图的比较运算。比较 K^2 次后，将得到的最大值写入输出缓存中。如图 4-3 加速器总体架构图所示，由于各层共用输入和输出模块，所以池化模块的并行度 $T_{pool} \leq \min(Tif_{max}, Tof_{max})$ ，并且 $T_{pool}=Tif=Tof$ 。池化层的硬件结构，如图 4-7 所示：

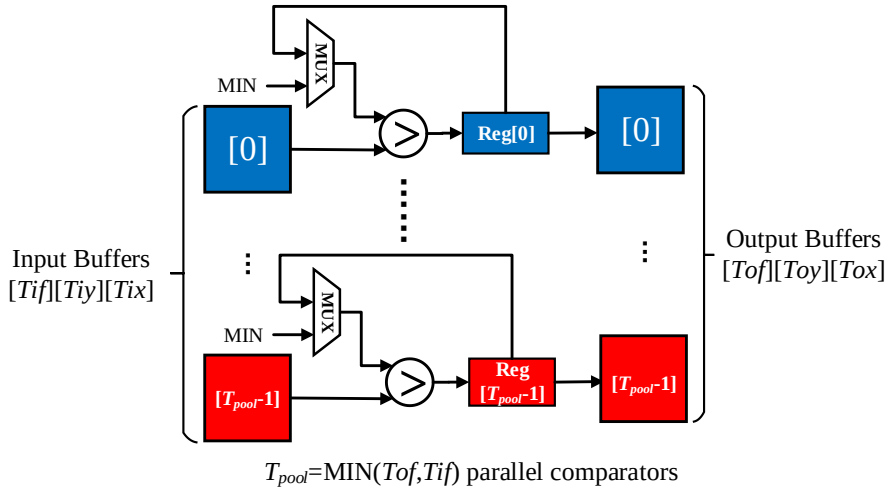


图 4-7 池化模块

池化模块的计算时延如下：

$$Latency_{compute} = (\text{Const} + N_{ky} \times N_{kx} \times T_{oy} \times T_{ox}) / Freq \quad (4-26)$$

池化模块主要耗费 LUT 资源用于多路选择器、比较器和寄存器的设计。考虑到其所耗费资源较少，这里不再过多说明。图 4-7 中的 MIN 表示给定的整型最小值常数。

4.2.4 重排序模块

重排序层对输入特征图像素抽样重排。重排序层的处理过程与最大池化层类似，都是对单一输入特征图的像素进行抽样。不同之处在于，最大池化层输出 $N_{ky} \times N_{kx}$ 邻域内的最大像素到单一输出特征图，而重排序层则是将 $N_{ky} \times N_{kx}$ 邻域内的像素输出到 $N_{ky} \times N_{kx}$ 张的输出特征图。所以，针对 YOLOv2 中 $N_{ky} = N_{kx} = S = 2$ 的重排序层，重排序模块只使用 1 块输入缓存和 4 块输出缓存，并且设计一个 4 路选择器，每次从输入缓存读取 1 个像素写入某个对应的输出缓存。对应重排序模块的时延如下：

$$Latency_{compute} = (\text{Const} + N_{ky} \times N_{kx} \times T_{oy} \times T_{ox}) / Freq \quad (4-27)$$

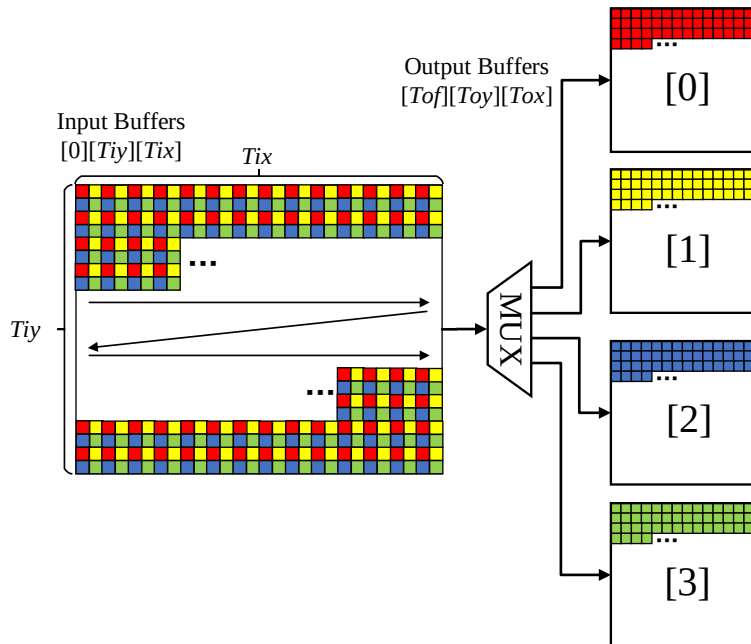


图 4-8 重排序模块

4.3 优化策略

4.3.1 参数重排序

参数重排序,把卷积层所需参数预先重排,使得每次传输的参数在 DRAM 中是连续的,可以以较大的突发长度来进行传输,有效提高了带宽的利用效率^{[29][33]}。

对于输入输出特征图像素,每次以较大的突发长度尽可能连续地传输数据;对于权重参数,由于只跟当前层有关,所以可以对各层权重根据当前层的块划分(这里指各层的 Tif 和 Tof) 进行内存参数重排序,以减少访存次数和增大突发长度。以 Layer 0 为例,如下图 4-9 所示:

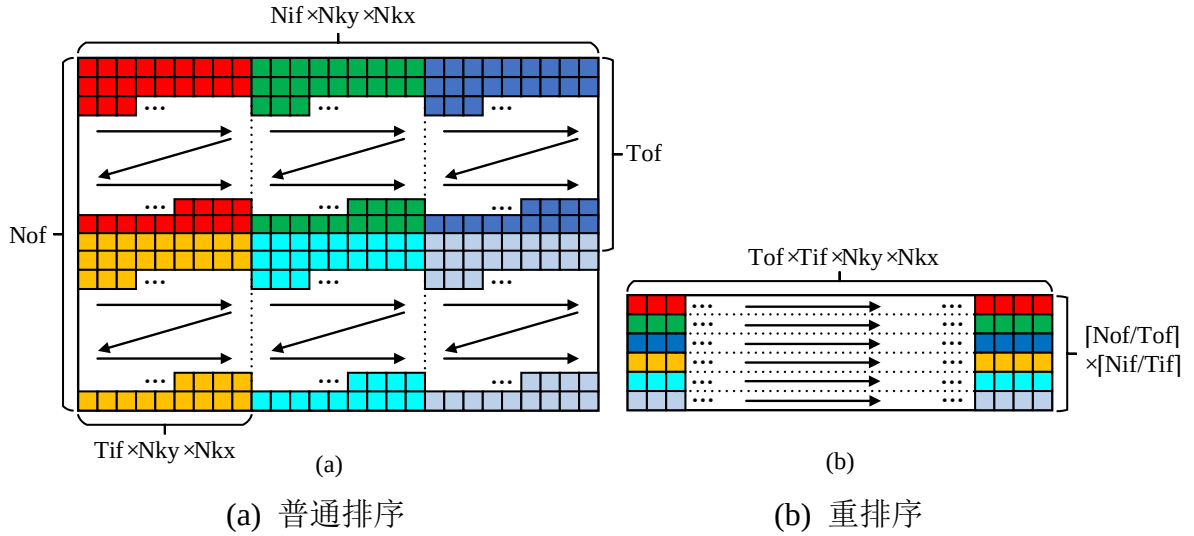


图 4-9 在 DRAM 中的参数排序

第 0 层的核权重为 $32 \times 3 \times 3 \times 3$ ($Nof \times Nif \times Nky \times Nkx$) 也可以看做 $32 \times 3 \times 9$ ($Nof \times Nif \times (Nky \times Nkx)$), 其中 $Nof=32, Nif=3, Nky=Nkx=3$ 。原始参数在内存中按行优先顺序存储。如图 4-9(a)所示,如果将卷积循环按 $Tof=16$ 和 $Tif=1$ 划分,那么就需要按箭头顺序从 DRAM 中读取 $16 \times 1 \times 9$ 大小的参数 6 次(先取所有红色的,再取所有绿色的、蓝色的、黄色的等)。但由于参数不连续,一共需要访存 $32 \times 3=96$ 次,每次的突发长度为 9。

如图 4-9(b)所示,参数重排序后,由于每次要读取的参数连续存放,只需访存 6 次,每次突发长度为 $16 \times 9=144$ 。减少访存次数并且增大突发长度,使得有效带宽大大增加。

参数读取模块与输入模块类似,也添加了行缓冲并进行了双缓冲的设计。参数重排序只是影响传输次数 Num_{Trans} 和每次传输的连续参数长度 $Length_{Trans}$ 如下所示:

$$Num_{Trans} = \begin{cases} Tof, Normal \\ Nky \times Nkx, Rearrangement \end{cases} \quad (4-28)$$

$$Length_{Trans} = \begin{cases} Tif \times Nky \times Nkx, Normal \\ Tof \times Tif, Rearrangement \end{cases} \quad (4-29)$$

Normal 表示不使用参数重排,从 DRAM 读取 Tof 次 $Tif \times Nky \times Nkx$ 个连续的参数;Rearrangement 表示经过参数重排序后,从 DRAM 读取 $Nky \times Nkx$ 次 $Tof \times Tif$ 个连续参数。

行缓冲所需的存储资源为:

$$N_{WLineBRAM} = \lceil Length_{Trans} \times Bitwidth / C_{oneBRAM} \rceil \times 2 \quad (4-30)$$

每次从 DRAM 读取连续 $Length_{Trans}$ 个权重参数到行缓冲中的时延如下：

$$Latency_{LoadW1} = \frac{Const}{Freq} + MM(Length_{Trans} \times \frac{Bitwidth}{Bitwidth_{Interface}}, Burst Length_{Max}) \quad (4-31)$$

将行缓冲中的 $Length_{Trans}$ 个参数写入片上参数缓存中的时延：

$$Latency_{LoadW2} = (Const + Length_{Trans}) / Freq \quad (4-32)$$

从 DRAM 读取 Num_{Trans} 次 $Length_{Trans}$ 个连续参数的读取时延，即参数读取时延如下：

$$Latency_{LoadW} = \frac{Const}{Freq} + Latency_{LoadW1} + Latency_{LoadW2} + (Num_{Trans} - 1) \times \text{Max}(Latency_{LoadW1}, Latency_{LoadW2}) \quad (4-33)$$

4.3.2 乒乓缓冲

通过乒乓缓冲的设计，将从片外 DRAM 读取数据的时延 $Latency_{Load}$ 、在片上进行数据处理的时延 $Latency_{Compute}$ 和将处理完的数据写回片外 DRAM 的时延 $Latency_{Store}$ 重叠，减少总时延^[7-8]。

由于读取权重和输入特征图像素块的过程是并发的，所以从片外 DRAM 读取数据的时延 $Latency_{Load}$ 由读取权重参数的时延 $Latency_{LoadW}$ 和读取输入特征图像素块的时延 $Latency_{LoadIFM}$ 中较大的一项决定：

$$Latency_{Load} = \text{Max}(Latency_{LoadW}, Latency_{LoadIFM}) \quad (4-34)$$

池化层和重排序层等没有权重参与的层 $Latency_{LoadW}$ 为 0。

卷积由六维乘加运算组成，所以与池化和重排序层不同，卷积计算包括了对输入特征图数维度 Nif 的遍历。故不使用乒乓时，卷积层各部分时延不重叠，输入特征图数维度 Nif 的内层循环需要重复 $[Nif/Tif]$ 次读取和计算操作，输出最终结果。此时，内层循环的时延 $Latency_{Inner}$ ：

$$Latency_{Inner} = \left\lfloor \frac{Nif}{Tif} \right\rfloor \times (Latency_{Load} + Latency_{Compute}) \quad (4-35)$$

卷积层处理的总时延为：

$$Latency = \left\lfloor \frac{Noy}{Toy} \right\rfloor \times \left\lfloor \frac{Nox}{Tox} \right\rfloor \times \left\lfloor \frac{Nof}{Tof} \right\rfloor \times (Latency_{Inner} + Latency_{Store}) \quad (4-36)$$

若使用乒乓，则读取时延 $Latency_{Load}$ 和卷积模块的计算时延 $Latency_{Compute}$ 重叠，读取和计算操作经 $([Nif/Tif]+1)$ 次循环后输出最终结果，此时内层循环时延为：

$$Latency_{Inner} = \left(\left\lfloor \frac{Nif}{Tif} \right\rfloor - 1 \right) \times \text{Max}(Latency_{Load}, Latency_{Compute}) + Latency_{Load} + Latency_{Compute} \quad (4-37)$$

写回时延与内层循环时延重叠，经 $([Nof/Tof]+1)$ 次循环后，遍历完输出特征图数维度 Nof 。此时，卷积层处理的总时延为：

$$Latency = \left\lfloor \frac{Noy}{Toy} \right\rfloor \times \left\lfloor \frac{Nox}{Tox} \right\rfloor \times \left\{ \left(\left\lfloor \frac{Nof}{Tof} \right\rfloor - 1 \right) \times \text{Max}(Latency_{Inner}, Latency_{Store}) + Latency_{Inner} + Latency_{Store} \right\} \quad (4-38)$$

与卷积层相比，池化层和重排序层缺少输入特征图数 Nif 维度，故只考虑读取、数据处理的时延三者是否重叠。在不使用乒乓时，三者线性进行，时间上没有重叠，处理总时延为：

$$Latency = \left\lfloor \frac{Noy}{Toy} \right\rfloor \times \left\lfloor \frac{Nox}{Tox} \right\rfloor \times \left\lfloor \frac{Nof}{Tof} \right\rfloor \times (Latency_{Load} + Latency_{Compute} + Latency_{Store}) \quad (4-39)$$

使用乒乓后，三者的处理过程两两重叠，此时处理总时延为：

$$Latency = \left\lfloor \frac{Noy}{Toy} \right\rfloor \times \left\lfloor \frac{Nox}{Tox} \right\rfloor \times \left\{ \left\lfloor \frac{Nof}{Tof} \right\rfloor \times \text{Max}(Latency_{Load}, Latency_{Compute}, Latency_{Store}) + Latency_{Load} + Latency_{Store} \right\} \quad (4-40)$$

使用乒乓时，对应输入缓存和输出缓存所耗费的 BRAM 资源数如下：

$$N_{InBufBRAM} = [(Tiy_{max} \times Tix_{max}) \times Bitwidth / C_{oneBRAM}] \times Tif \times 2 \quad (4-41)$$

$$N_{OutBufBRAM} = [(Toy_{max} \times Tox_{max}) \times Bitwidth_{inter} / C_{oneBRAM}] \times Tof \times 2 \quad (4-42)$$

其中 $Bitwidth_{inter}$ 表示卷积计算时中间结果的数据位宽，典型值为 32。对于权重缓存，数据量较小时使用 FF 和 LUT 资源设计；数据量较大时，使用 BRAM 资源设计，BRAM 的耗费如下：

$$N_{WBufBRAM} = \left\lceil Nky \times Nkx \times \frac{Bitwidth}{C_{oneBRAM}} \right\rceil \times Tif \times Tof \times 2 \quad (4-43)$$

不使用双缓冲时，对应耗费的 BRAM 资源数减半。

4.3.3 多通道数据传输

上文中的输入输出模块每次顺序地读取或写入多特征图的一个像素块。考虑到 DRAM 实际带宽大于单接口的总线带宽，通过多通道输入输出，可以有效减少传输时延^[52]。多通道数据传输，如图 4-10 所示：

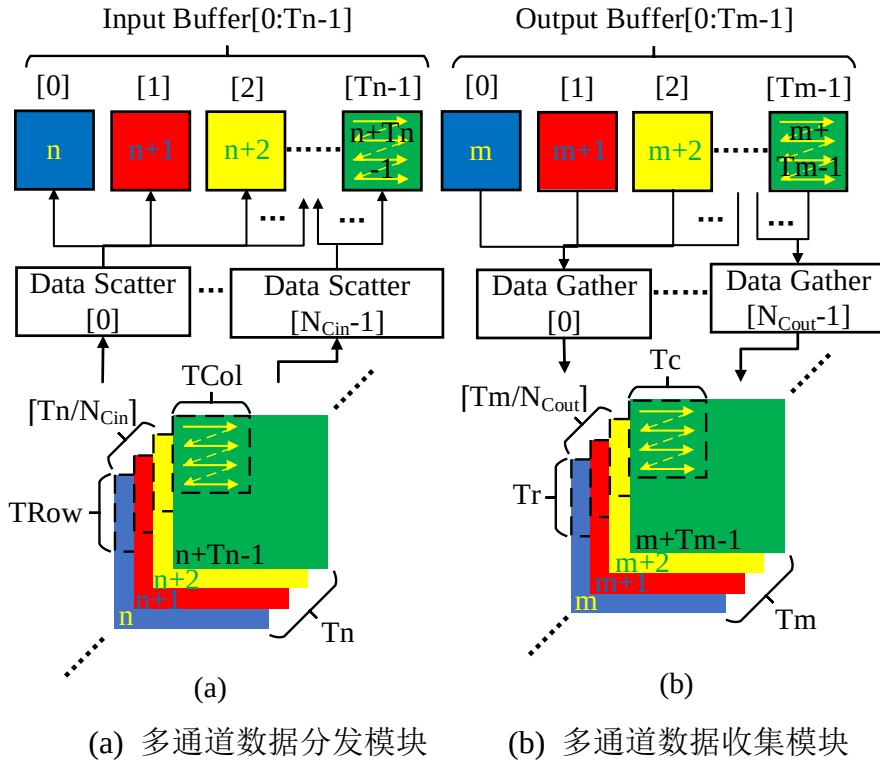


图 4-10 多通道数据传输

图 4-10 与图 4-4 类似，但是图 4-10 (a)中数据分发模块变为 N_{cin} 个子输入模块，每个子输入模块从 DRAM 读取 $[Tif/N_{cin}]$ 张输入特征图的像素块到片上缓存。图 4-10 (b)

中数据收集模块变为 N_{Cout} 个子输出模块，每个子输出模块将 $[Tof/N_{Cout}]$ 张输出特征图的像素块从片上缓存写回到片外。各子模块间无依赖性，并发地完成数据传输。此时，在公式(4-9)、(4-12)和(4-13)中 $N_{Cin} > 1$ ，公式(4-16)、(4-19)和(4-20)中 $N_{Cout} > 1$ 。同时，由于读写通道的增加，公式(4-9)和(4-16)中耗费的 BRAM 资源数也会随之增加。

4.4 实验评估

4.4.1 软硬件实验环境

CNN 模型：YOLOv2 和 YOLOv2-Tiny(416x416)^[43]，数据集为 COCO 数据集^[2]。

FPGA 平台：Xilinx Zedboard 开发板(Dual-core ARM-A9+FPGA)，其中 FPGA 的 BRAM_18Kb、DSP48E、FF 和 LUT 资源数分别为 280、220、106400 和 53200。

CPU 平台：服务器 CPU Intel Xeon E5-2620 v4(8 cores) 工作频率 2.1GHz，256GB 内存。Zedboard 上的双核 ARM-A9，时钟频率 667MHz，512MB 内存。

嵌入式 GPU 平台：Nvidia Jetson TX2 工作频率 850MHZ，8G 内存。

使用 Vivado HLS 2018.2 HLS 进行加速器设计，Vivado v2018.2 进行综合和布局布线。功耗采用 VC470 功耗测量仪外部测量系统功耗，对于 E5-2620 v4，采用散热设计功耗（Thermal Design Power, TDP）作为总功耗（实际功耗大于 TDP）。由于 Zedboard 是 ARM+FPGA 异构 SoC 平台，所以 FPGA 的功耗通过计算时的系统峰值功耗减去 FPGA 复位后的系统空闲功耗得到。

4.4.2 加速器的规模可伸缩性评估

本节对输出特征图复用模式下的加速器设计空间进行探索，与本章的加速器设计基本保持一致，通过改变外循环三维（图 3-9 中的 Loop-4、Loop-3、Loop-2）的硬件设计参数(Tof, Tif, Toy, Tox)对加速器规模的变化进行评估。为了简化设计空间，令 $N_{Cin}=N_{Cout}=1$ 。

Zedboard 评估板上的 Zynq7020 芯片拥有 220 个 DSP，并且根据表 4-1，在动态定点 16 位数据精度下，每个乘法器会耗费 1 个 DSP，加法器不耗费 DSP。考虑到当前工作时钟为 150MHz，系统的性能上限为 66GOPS ($220 \times 150 \times 2/1000 = 66$)。

Zedboard 评估板上的内存型号为 32bit DDR3-1066，理论最大带宽为 4.264GB/s；然而，由[47]可知，实际带宽效率只有 75%左右，所以最大带宽为 3.198GB/s。并且，对于资源耗费和硬件设计参数，还需要设置一些额外约束：

$$N_{DSP}^{Conv} \leq N_{DSP}^{Total} \times 85\% \quad (4-44)$$

$$N_{LUT}^{Conv} \leq N_{LUT}^{Total} \times 80\% \quad (4-45)$$

$$(N_{LoadLineBRAM} + N_{StoreLineBRAM} + N_{WLineBRAM} + N_{InBufBRAM} + N_{OutBufBRAM} + N_{WBufBRAM}) \leq N_{BRAM}^{Total} \times 85\% \quad (4-46)$$

$$Tof \geq 4, Toy \geq 26, Tox \geq 26 \quad (4-47)$$

由于当前的设计主要分析卷积模块的 DSP 和 LUT 耗费，其他模块的相应耗费并没有详细分析，所以在此需要留出一些资源用于其他模块的设计。对于硬件设计参数的约束（公式 4-53），主要考虑到重排序层的特殊性以及数据精度（16bit）与总线数据位宽（32bit）的差异。

同时, 由于使用 Roofline 模型对加速器进行评估, 模型计算量已知的情况下, 还需要知道对应不同硬件设计参数的加速器的访存次数, 公式如下:

$$N_{InMA} = \left\lceil \frac{Noy}{Toy} \right\rceil \times \left\lceil \frac{Nox}{Tox} \right\rceil \times \left\lceil \frac{Nof}{Tof} \right\rceil \times \left\lceil \frac{Nif}{Tif} \right\rceil \times Tif \times Tiy \times Tix \quad (4-48)$$

$$N_{OutMA} = \left\lceil \frac{Noy}{Toy} \right\rceil \times \left\lceil \frac{Nox}{Tox} \right\rceil \times \left\lceil \frac{Nof}{Tof} \right\rceil \times Tof \times Toy \times Tox \quad (4-49)$$

$$N_{WMA} = \left\lceil \frac{Noy}{Toy} \right\rceil \times \left\lceil \frac{Nox}{Tox} \right\rceil \times \left\lceil \frac{Nof}{Tof} \right\rceil \times \left\lceil \frac{Nif}{Tif} \right\rceil \times Tof \times Tif \times Nky \times Nkx \quad (4-50)$$

$$N_{MA}^{total} = \sum_{i=0}^{28} (N_{InMA}^i + N_{OutMA}^i + N_{WMA}^i) \quad (4-51)$$

其中, N_{InMA} 、 N_{OutMA} 和 N_{WMA} 分别表示读取输入特征图、写回输出特征图和读取卷积核参数的访存次数; N_{MA}^{total} 表示所有层的访存次数之和。对应加速器设计的计算密度 I 和性能 P 可通过公式 (3-24) 和 (3-25) 得到。

并且, 通过算法 3-1 找出当前硬件系统下满足约束的不同规模的加速器设计, 对应的不同规模加速器的设计空间, 以 Roofline 模型进行评估, 如图 4-11 所示:

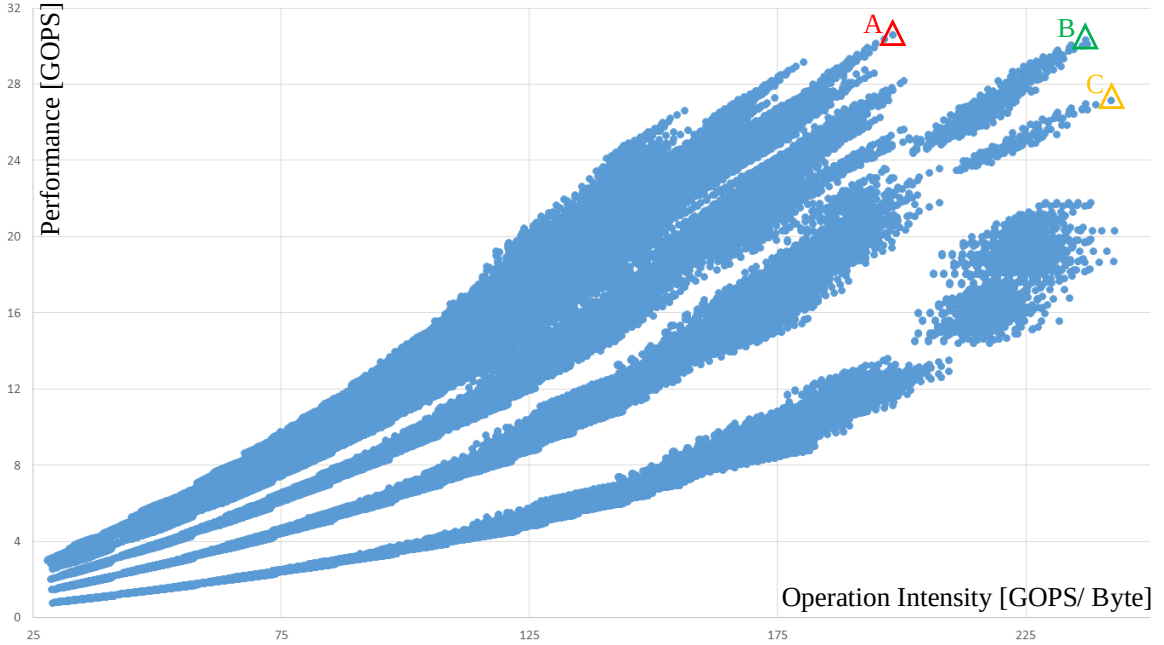


图 4-11 不同规模加速器的设计空间探索

如图 4-11 所示, 花费约 12s 找到 156396 种满足约束加速器设计, 所有找到的设计都处于性能受限区域。其中性能最优的设计为图中点 A ($Tif=4$, $Tof=44$, $Toy=26$, $Tox=50$, 30.61 GOPS), 能耗最低的设计为点 C ($Tif=2$, $Tof=86$, $Toy=26$, $Tox=26$, 27.15 GOPS), 兼顾两者的设计为点 B ($Tif=2$, $Tof=86$, $Toy=26$, $Tox=36$, 30.33GOPS)。

4.4.3 硬件设计参数与异构系统

选择动态定点 16 位精度下的加速器架构, 采用参数重排序、乒乓和多通道数据传输三种优化策略, 其中 $Tof_{max}=32$, $Tif_{max}=4$, $Toy_{max}=26$, $Tox_{max}=26$, $Nky_{max}=Nkx_{max}=3$, $S_{max}=2$, $N_{Cin}=4$, $N_{Cout}=2$, $Pof=Tof$, $Pif=Tif$, 其余硬件设计参数均为 1。给定以上参数后, 实际加速器设计已确定, 此时还需要确定各层针对当前加速器的 Tof 、 Tif 、 Toy 和 Tox 。

各层的 Toy 和 Tox 由以下公式得出:

$$T_{oy} = \text{Min}(\left\lceil \frac{(T_{oy_{max}} - Nky)}{s} \right\rceil + 1, T_{r_{max}}, N_{oy}) \quad (4-52)$$

$$T_{ox} = \text{Min}(\left\lceil \frac{(T_{ox_{max}} - Nkx)}{s} \right\rceil + 1, T_{c_{max}}, N_{ox}) \quad (4-53)$$

通过公式(4-52)和(4-53)，可以得出各层输入和输出缓存能够存储的像素块的最大宽高。对于 YOLOv2 中的池化层和重排序层， $S=Nky=Nkx=2$ 。

对于卷积层， T_{of} 和 T_{if} 由以下公式得出：

$$T_{of} = \text{Min}(N_{of}, T_{of_{max}}) \quad (4-54)$$

$$T_{if} = \text{Min}(N_{if}, T_{if_{max}}) \quad (4-55)$$

当 $T_{of_{max}}$ 和 $T_{if_{max}}$ 给定后，卷积计算模块的设计就已确定。因此，各卷积层的输入和输出并行度 T_{if} 和 T_{of} 不能超过相应最大值。

对于池化层， T_{of} 和 T_{if} 由以下公式得出：

$$T_{if} = T_{of} = \text{Min}(T_{of_{max}}, T_{if_{max}}, N_{of}) \quad (4-56)$$

由于各计算模块复用输入输出模块，所以池化层的并行度 $T_{pool} \leq T_{of}$ ，实验中 $T_{pool}=T_{of}$ 。 $S=2$ 的重排序层将一张输入特征图抽样重排为 4 张输出特征图，所以 $T_{if}=1$ ， $T_{of}=4$ 。此时，异构系统的整体连接如图 4-12 所示：

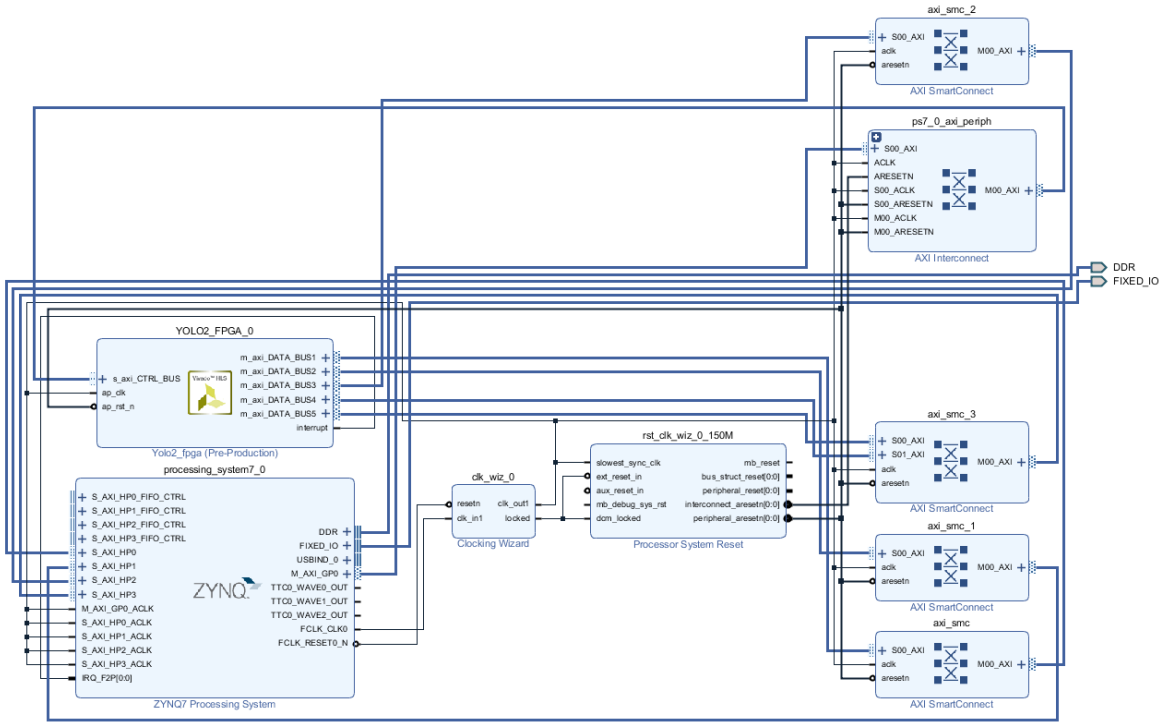


图 4-12 Zynq 异构系统

其中的 YOLO2_FPGA 对应封装后的 YOLOv2 加速器 IP 核。ZYNQ7 Processing System 对应 PS 端系统（ARM+外设），其余部分皆为 FPGA 端的 IP。YOLOv2 加速器对外有 1 个 AXILite Slave 接口与 PS 端的 GP0 接口连接，负责读写数据、控制和状态寄存器；还有 5 个 AXI Master 接口分别与 PS 端的 HP{0-3}接口连接（HP3 连接两个接口），负责读写特征图像素和权重。FPGA 端 IP 统一工作在 150MHz 时钟频率。

4.4.4 资源耗费评估

结合给定参数和上文中对各模块资源耗费的公式,将得到的估计资源耗费与实际资源耗费进行对比,如表 4-2 所示。

表 4-2 FPGA 资源耗费

	DSP	BRAM_36Kb	LUT	FF	Freq.
Estimated	128(58%)	85(61%)	20736(39%)	-	150MHz
Actual	153(70%)	88(63%)	37342(70%)	35785(34%)	150MHz

DSP 主要用于卷积模块中加法器和乘法器的设计。根据公式(4-24),DSP 耗费 128 个,实际设计中多出的 DSP 主要用于其他模块的设计。

BRAM_18Kb 用于实现存储量大的缓存,除上文中所提及的各模块对 BRAM 的耗费外,AXI Master 接口也会耗费 BRAM 来实现接口缓存(当前设计下,耗费 $N_{AXIM} \times 2 = 10$ BRAM)。偏置参数 bias 的缓存也耗费了额外的 2 个 BRAM,再根据公式(4-41)、(4-42)、(4-9)、(4-30)和(4-16),加上输入缓存($3 \times 4 \times 2$)、输出缓存($2 \times 32 \times 2$)、片上输入行缓冲(2)、权重行缓冲(2)和输出行缓冲(2×2)所耗费的 BRAM 资源,共耗费 BRAM_18b 170 个,即 85 个 BRAM_36Kb,与实际 BRAM 耗费接近。

由于单个权重缓存的存储量过小($N_{ky} \times N_{kx} \times Bitwidth = 9 \times 16 = 144b$),无法充分利用 BRAM。在设计时会考虑使用 LUT 和 FF 来实现权重缓存(当前设计下,单个权重缓存耗费约 3 个 LUT)。因此,权重缓存共耗费 $3 \times Tof_{max} \times Tif_{max} \times 2 = 768$ 个 LUT。由公式(4-25)可知,此时 16 位定点精度的卷积模块耗费 19968 个 LUT。所以,共耗费约 20736 个 LUT,剩余 LUT 用于其他模块的设计。

4.4.5 性能评估

(1) 理论与实际时延评估

YOLOv2 模型除路由层、图像的预处理和后处理外,剩余 29 层。其中包括 23 层卷积层、5 层池化层和 1 层重排序层。表 4-3 列出 29 层的评估结果(保留小数点后 3 位),列中的 R (Real) 表示该列数据是实测值, E (Estimated) 指该列数据是根据上文公式得到的估计值。

以相对误差对时延模型进行评价,相对误差 δ 公式如下:

$$\delta = \frac{|Estimated\ Value - Real\ Value|}{Real\ Value} \times 100\% \quad (4-57)$$

即测量的绝对误差与被测量的真值之比。

以表中最后一行总延时为例, YOLOv2 模型的总时延误差约为 4.9% ($\frac{|977.485 - 929.738|}{977.485} \approx 4.88\%$),总计算时延误差约为 0.8%,总输入时延误差约为 10.7%,总输出时延误差约为 10.4%。虽然本模型的主要误差仍集中于传输时延部分约为 10%,但已远小于[29]中的误差 47%。由于采用输出特征图复用模式,所以总输出时延只有 165.851ms,远小于总输入时延(665.98ms)。

表 4-3 实际与估计时延

Layer	Load Time R(ms)	Load Time E(ms)	Compute Time R(ms)	Compute Time E(ms)	Store Time R(ms)	Store Time E(ms)	Total Time R(ms)	Total Time E(ms)
Conv 0	3.830	2.901	10.583	10.455	43.173	38.120	57.293	51.421
Pool 1	17.487	13.197	9.344	9.247	20.521	19.137	24.826	23.179
Conv 2	17.029	11.603	41.762	41.629	21.593	19.060	54.641	52.582
Pool 3	8.798	6.599	4.691	4.623	10.276	9.568	11.368	10.579
Conv 4	17.373	11.603	41.738	41.615	10.813	9.530	45.505	44.709
Conv 5	15.511	10.507	4.808	4.690	5.427	4.765	18.460	13.036
Conv 6	17.372	11.603	41.736	41.615	10.812	9.530	45.508	44.709
Pool 7	4.489	3.299	2.368	2.312	5.160	4.784	5.436	5.037
Conv 8	16.695	11.603	41.728	41.609	5.427	4.765	42.926	42.560
Conv 9	14.786	10.507	4.805	4.687	2.736	2.383	15.606	11.176
Conv 10	16.687	11.603	41.727	41.609	5.426	4.765	42.924	42.560
Pool 11	2.204	1.650	1.206	1.156	2.599	2.392	2.668	2.455
Conv 12	11.141	10.793	41.724	41.605	2.735	2.383	42.061	41.919
Conv 13	9.249	6.967	4.800	4.685	1.389	1.191	9.504	7.153
Conv 14	11.142	10.793	41.724	41.605	2.733	2.383	42.063	41.919
Conv 15	9.255	6.967	4.800	4.685	1.387	1.191	9.505	7.153
Conv 16	11.141	10.793	41.723	41.605	2.733	2.383	42.064	41.919
Pool 17	1.170	0.871	0.635	0.581	1.070	1.123	1.204	1.139
Conv 18	44.161	43.172	42.142	41.813	1.073	1.104	44.542	43.533
Conv 19	13.259	9.936	5.215	4.891	0.557	0.552	13.329	9.990
Conv 20	44.167	43.172	42.142	41.813	1.073	1.104	44.543	43.533
Conv 21	13.257	9.936	5.215	4.891	0.558	0.552	13.333	9.990
Conv 22	44.163	43.172	42.142	41.813	1.073	1.104	44.546	43.533
Conv 23	88.288	86.344	84.222	83.620	1.072	1.104	88.658	86.705
Conv 24	88.281	86.344	84.229	83.620	1.073	1.104	88.661	86.705
Conv 25	2.347	1.742	1.241	1.171	0.380	0.298	2.540	1.900
Reorg 26	0.495	0.412	0.334	0.289	0.683	0.598	1.423	1.608
Conv 27	110.339	107.930	105.261	104.523	1.075	1.104	110.712	108.291
Conv 28	11.573	8.694	4.569	4.280	0.485	0.483	11.636	8.745
Conv Total	631.046	568.682	780.036	774.530	124.803	110.957	930.560	885.741
Pool Total	34.148	25.616	18.244	17.919	39.626	37.005	45.502	42.388
Total	665.980	594.711	799.066	792.737	165.851	148.559	977.485	929.738

对应表 4-3 的折线图，如图 4-13 和图 4-14 所示。理想情况下，图 4-13、图 4-14 中只看到 4 条曲线（实际与估计时延曲线两两重合），即时延模型与实际时延完美匹配。同时，如果计算时延与总时延接近，证明当前加速器的设计是非常高效的（这说明当前的瓶颈是在于计算，访存和传输时延都已被掩盖）。如图 4-13 所示，在刚开始的第 0 层至第 10 层实际与估计时延曲线并不能较好地重合，而之后的各层都基本保持一致；这是因为开始的几层输入输出特征图较大，当前的模型对输入输出时延的估计虽然有了较大的改进，但是仍有约 10% 的误差。

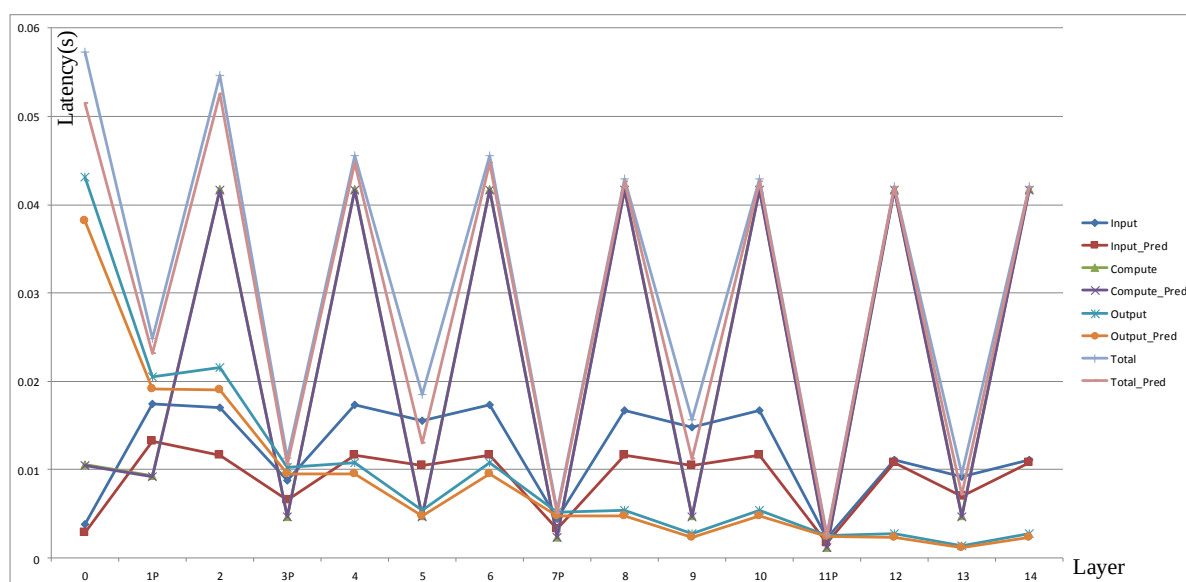


图 4-13 第 0-14 层的实际与估计时延

并且，在刚开始的第 0 层至第 6 层明显可以看到，总时延与计算时延并不接近，这就是由于各卷积层和池化层循环各维“形状”与加速器的设计不能较好地匹配导致。但是，此处只是给出一种总体上计算效率较高的设计，之后各层的总时延与计算时延都基本比较接近。由于采用输出特征图复用的数据模式，实际与估计的输出时延（图中最底部的两条重叠的折线）都远远小于图中的其他时延。对于实际与估计的计算时延曲线，由于误差极小（0.8%），基本保持重合。

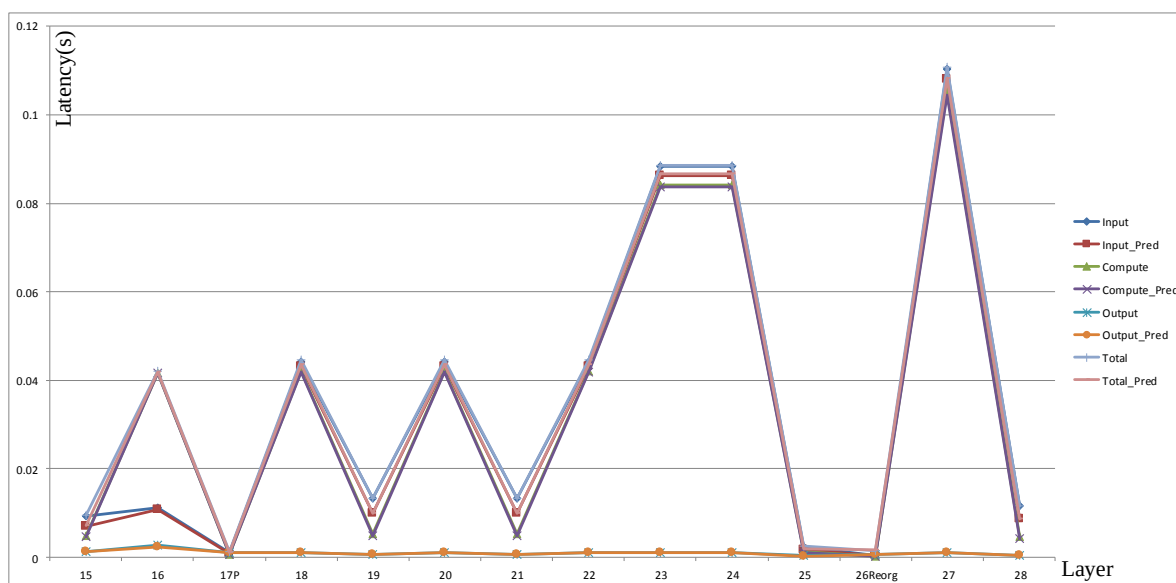


图 4-14 第 15-28 层的实际与估计时延

另外，对于图 4-13 中输入和输出的实际与估计时延误差较大，图 4-14 中误差较小的问题，个人看法：算法 2-1 仿真得到的不同突发长度下的有效带宽在突发长度较小时，误差较大；突发长度较大时，误差较小。而前几层的输入输出特征图的宽高远大于每次访存的特征图像素块的宽高，即实际每次访存的数据块的是不连续的，突发长度很低（考虑到数据精度是动态定点 16 位且 $Toy_{max}=26$ ，实际突发长度为 $26/2=13$ ）导致输出时延大，同时也导致实际与估计时延之间的误差很大。

(2) 与之前的基于 FPGA 的工作对比

当前的工作在性能上超过了之前的工作, 如表 4-4 所示。[57]基于 Intel 的 OpenCL 采用通用矩阵乘法对矩阵分块, 块间并行乘加的方式加速。然而, 需要每次对权重和输入特征图重排序, 增加了预处理时延和复杂度。[58]虽然通过缩小模型以及减小数据精度等方式将简化后的整个 YOLOv2-Tiny 映射到 FPGA 上, 但是层间没有乒乓, 访存与数据传输时延无法与计算时延重叠, 实际与理论最大性能差距很大。[59]将卷积核权重量化到二值, 用异或完成加法, 极大地降低了资源耗费, 同时也可工作在更高时钟频率。但在准确度方面会有很大损失, 必须通过其他方式补偿。当前的工作主要考虑规模的可伸缩性 (包括普适性和灵活性), 所以没有采用更低的精度与重新训练继续优化。

表 4-4 与其他基于 FPGA 的工作比较

	[57]	[58]	[59]	Ours	
CNN model	YOLOv2-Tiny	YOLOv2-Tiny	YOLOv2	YOLOv2	YOLOv2-Tiny
FPGA Board	Cyclone V	Zynq 7020	ZCU102	Zedboard	
Clock (MHz)	117	142	300	150	
Precision	Fixed-16	Fixed-8	Fixed-2	Fixed-16	
DSP (Used/Total)	122/224	110/220	377/2520	153/220	
Power (W)	-	-	4.5	1.2	0.83
Operations (GOP)	5.41	0.30	14.18	29.47	5.41
Performance (GOP/s)	19.45	7.46	347.44	30.15	21.97

(3) 与 CPU 的性能与能效对比

如表 4-5 所示, 与 CPU 相比, CPU+FPGA 的异构系统是双核 ARM-A9 能效的 86 倍左右, Xeon 的 120.4 倍。速度是双核 ARM-A9 的 112.9 倍, Xeon 的 7.3 倍左右。此时的功耗较低, 但是由于计算能力弱, 使得计算时延过长, 能效低。Xeon 与其相反, 计算性能强, 使得计算时延很短, 但是由于功耗过高, 导致能效低下。与前两者不同, CPU+FPGA 的异构系统功耗略高于双核 ARM-A9, 但使用 FPGA 加速后, 大幅度提高计算性能, 在计算时延和能效上都获得了不错的表现。

表 4-5 与 CPU 的比较

Device	CPU	CPU	CPU+FPGA
	E5-2620v4 (8 cores)	ARM-A9 (dual-core)	Zedboard
Technology(nm)	14	28	28
Clock(Hz)	2.10G	666.67M	666.67M +150M
Memory(MB)	257842	512	256+256
Peak System Power(W)	85	3.96	5.01 (3.81+1.20)
Performance(GOP/s)	4.11	0.27	30.15
Latency(s) per img	7.17	110.36	0.98
Power Efficiency (GOP/s/W)	0.05	0.07	6.02

基于 Xeon CPU 的评估结果由 YOLOv2 源码编译(-Ofast)执行得到, 数据精度为 float32。(Fixed16 执行时间约为 15.78s)

基于 FPGA+CPU 的评估结果(包括双核 ARM-A9)由 ARM-Linux-gcc(release -O3 -static)编译后执行得到。动态定点 16 位数据量化得到的模型, 在准确度上与原模型接近, COCO 数据集上 mAP 达到约 48.1%。

(4) 与嵌入式 GPU 的性能与能效对比

如表 4-6 所示, 与当前的主流嵌入式 GPU 相比, 由于进行评估的 CPU+FPGA 的异构平台过于低端且工艺相差过大, 无论是在性能还是能效上都处于劣势。但是, 如[59]所述, 如果考虑同等工艺下的 FPGA 芯片, 且采用更低的数据精度, 则在性能和能效上都能取得良好的效果。

表 4-6 与 GPU 的比较

Device	GPU Jetson TX2 (Float-32)	GPU Jetson TX2 (Float-16)	CPU+FPGA[59] ZCU102 (Fixed-2)	CPU+FPGA Zedboard (Fixed-16)
Technology(nm)	16	16	16	28
Clock(MHz)	850	850	-	666.67
			+300	+150
Memory(MB)	8192	8192	4096	256+256
Peak System Power(W)	5.8	6.8	4.5	5.0
				(3.81+1.20)
Performance(GOP/s)	229.89	279.99	347.44	30.15
Power Efficiency (GOP/s/W)	39.64	41.18	77.21	6.02

4.5 本章小结

首先, 介绍了从 CNN 模型到 FPGA 加速器设计与优化的整体流程, 然后简单地介绍了加速器所使用的动态定点 16 位数据精度的量化方法。其次, 详细描述基于 SIMD 架构输出特征图复用模式下的二维展开的 CNN 加速器设计, 并进行资源耗费与时延的评估, 结果表明: 实际资源耗费与时延与理论值基本一致。最后, 就不同的硬件设计参数组合对加速器的规模可伸缩性进行了评估。

第五章 基于FPGA动态可重构的CNN加速器设计与评估

5.1 可重构加速器架构研究

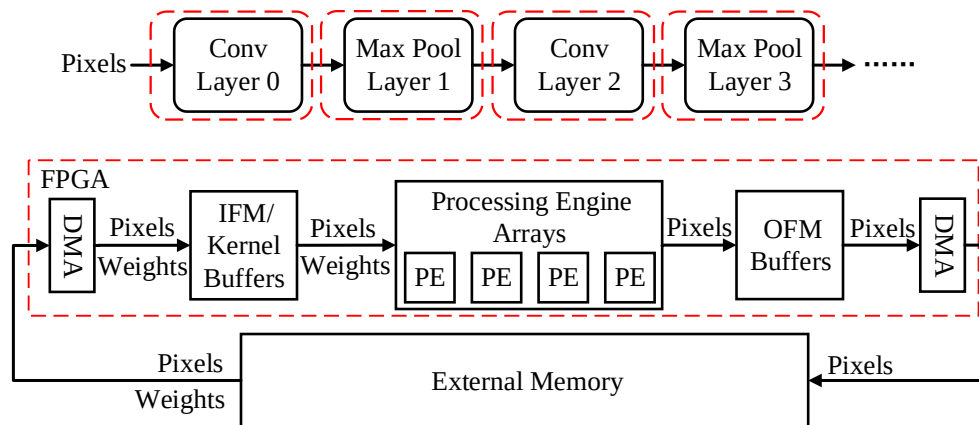


图 5-1 典型加速器与单映射关系

如图 5-1 所示，典型的加速器按网络顺序依次处理各层，FPGA 上的单个加速器每次只能处理某层，从片外存储读取输入特征图像素块和卷积核权重，经 PE 阵列处理得到最终结果，再写回片外。对于给定网络，将网络中的每一层按顺序映射到同一个加速器上进行处理，实质是对同一加速器的时分复用。此时，FPGA 的硬件逻辑只进行一次初始化配置，之后保持不变。由于各层维度“形状”的不同，将不同卷积层映射到相同架构时，往往并不能充分利用计算单元，导致计算单元的动态利用率低下。

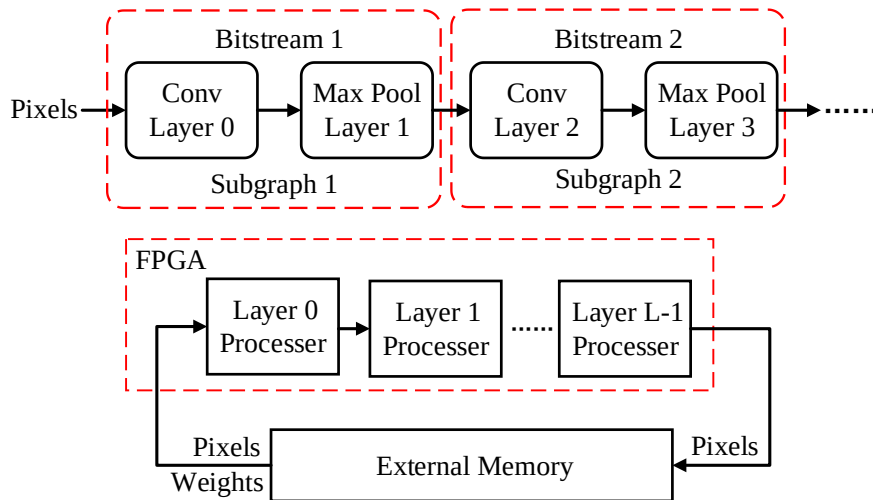


图 5-2 可重构加速器与多映射关系

如图 5-2 所示，可重构加速器与典型加速器有所不同，考虑到当前神经网络结构的复杂性与各层卷积“形状”的不同，一般会将整个网络划分为多个子图（本章中简化为串行级联的结构或单层加速器结构，实际也可各种复杂结构^[60]），分别对各子图定制加速器，使得对于各子图，计算单元的动态利用率大大提升。本章中，将各子图的加速器架构限定为串行级联的结构和单层加速器结构。单层的加速器架构类似图 5-1 和第四章中的设计；多层级联的加速器结构，实际为简单的流式结构，即只有第一层和最后一层

与片外有读写交互,中间层的结果都保留在 FPGA 片上,计算得到最终结果后传给下一层。卷积参数足够少的情况下,可以保存在对应层加速器内,否则也需要额外的访存。

另外,由于对各子图定制加速器,每个子图都有对应的配置流文件(某些条件下会直接使用前一个子图的加速器架构,此时不需要重构),在进行计算前需对整块 FPGA 进行重构。

与典型加速器相比,可重构加速器的优点是显而易见的:在给定硬件系统的条件下,各子图可以充分利用硬件资源,使得各子图内的性能得到极大的提高。此外,当使用层级联的流式架构时,各层间可直接在 FPGA 片上进行特征图的交互,不需访存,极大地降低了访存时延和功耗。

考虑到动态可重构加速器的灵活性,在此给出一种构建动态可重构加速器的流程:

(1) 初始子图划分: 根据网络结构和层间数据依赖性,在各层间设置断点,划分子图。断点表示相邻层不在同一子图,因此就形成了 N_1 个子图, $1 \leq N_1 \leq N_{\text{Total}}$ 。 N_{Total} 表示网络总层数。

(2) 子图再划分: 按初始子图依赖顺序,在各子图内根据时延模型、FPGA 配置时延代价、FPGA 资源数和片外带宽等约束,寻找当前子图最小时延所对应的架构,可能需要进行子图的递归划分。最后,形成子图 N_2 个, $1 \leq N_1 \leq N_2 \leq N_{\text{Total}}$ 。

5.2 基于FPGA的动态可重构CNN加速器设计

本节以 YOLOv2 为例进行可重构加速器设计与评估,单层加速器主体仍基于第四章的架构。已知,重构整块 Zynq 7020 的配置时延约 255ms。(约 15ms 为从 SD 卡文件系统中读取配置文件到内存的时延,剩余 240ms 为真正的配置时延。)

5.2.1 初始子图划分

由表 2-1 可知, YOLOv2 除最后的 detection 层(放入图像后处理中)外,仍剩余 Conv 层 23 层、Max Pool 层 5 层、Reorg 层 1 层、Route 层 2 层。在初始子图划分阶段,主要考虑层间依赖性,简化子图结构,在 Route 层划分断点,如图 5-3 所示:

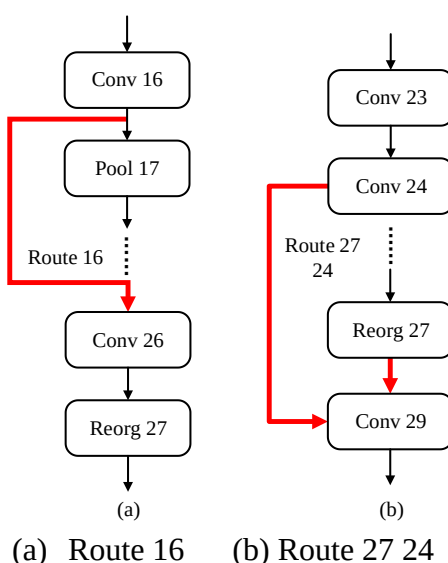


图 5-3 Route 层连接

图 5-3(a) Route 16 表示将第 16 层卷积层的输出作为第 26 层的输入。但是,第 16 层卷积层的输出同时也是第 17 层最大池化层的输入。如果将第 16 层与 26 层划分为同一子图,或者将第 16 层与 17 层划分为同一子图都会导致另外一层失去输入,因此在第 16 与 17 层间、第 16 与 26 层间添加断点。另一种划分方式,是将第 16、17 和 26 层划分为同一子图,在第 15 与 16 层间、第 26 与 27 层间、第 17 与 18 层间添加断点。考虑到后一种划分的复杂性,采用前一种划分方式。

图 5-3(b) Route 27 24 将第 27 层重排序层的输出与第 24 层卷积层的输出级联(27 层的输出特征图在前),作为第 29 层卷积层的输入。考虑到第 29 层有多个输入,在第 24 与 29 层间、第 27 与 29 层间添加断点。

另外,由表 4-3 可知,第 26 与 27 层的实际时延分别为 2.54ms 与 1.43ms,时延较小,所以在子图{26-27}内复用单层加速器架构。

综上,对网络中的 29 层(23 个卷积层、5 个最大池化层和 1 个重排序层)进行子图划分,将整个网络划分为 4 个子图,划分如下: {0-16, 17-24, 26-27n, 29-30}。(n 表示,该子图不用再进行划分,并且子图内使用单加速器架构分时复用。)此时,各子图间的依赖顺序如图 5-4 所示:

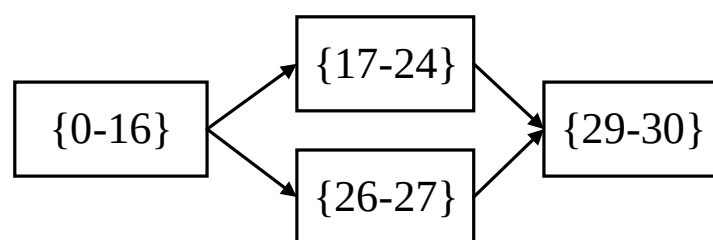


图 5-4 子图依赖性

此时,可将子图{0-16}中卷积层后的最大池化层与该卷积层融合,即卷积计算得到最终结果,不直接写回片外,而是经过额外的最大池化模块池化后再写回。此时,只会增加少量的 FPGA 资源耗费。与 4.3 中的设计相比,减去了最大池化层的输入访存时延和前一卷积层的输出访存时延。

初始子图划分,实际是根据某些先验知识对整个网络进行初步划分,以简化子图结构。在经过初始子图划分后,每个子图只存在两种层结构:(1)多层级联结构(2)单层结构。

5.2.2 架构设计

经子图再划分后,将整个网络中的各子图映射到两种典型架构:(1)单层加速器架构(与第四章中的架构类似,可能融合级联的最大池化层)(2)多层级联的加速器架构。

单层加速器架构与 4.2 节所述基本类似,除了在卷积层后级联的最大池化模块(即在 ReLU 模块后级联额外的 Pool 模块)。ReLU&Pool 模块的展开度 T_{pool} 只需满足 $T_{pool} \leq T_{of}$; 而单独的最大池化层处理模块的展开度 T_{pool} 需要满足 $T_{pool} \leq \min(T_{if}, T_{of})$ 。

多层级联加速器的整体架构，如图 5-5 所示：

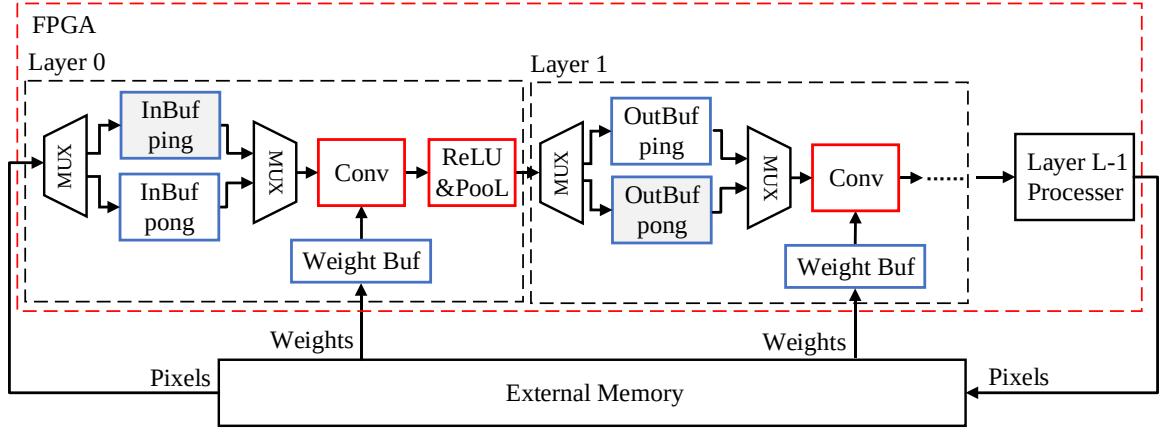


图 5-5 多层级联加速器整体架构

图 5-5 表示 L 层级联的加速器架构，除第 0 层与第 $L-1$ 层外，中间各层的输入输出都保存在 FPGA 片上。由乒乓缓冲设计实现各层间计算与数据交互的并发性。以 Layer 0 为例，当 Layer 0 从片外读取输入特征图到输入缓冲（InBuf pong）时，Layer 0 的卷积模块也在同时从输入缓冲（InBuf ping）中读取数据进行计算；与此同时，在 Layer 0 将卷积计算的结果写到输出缓冲（OutBuf pong）时，Layer 1 同时也在从输出缓冲（OutBuf ping）读取数据进行卷积计算。

对于多层级联的加速器架构，只有第一层与最后一层需要与片外进行特征图像素的交互，中间各层的结果都保存在 FPGA 片上缓存中。所以，相对于第四章中的设计，需要将中间层的输出缓存改为对应的行缓冲，行缓冲所耗费 BRAM 资源数如下：

$$N_{StoreLineBRAM} = \lceil [Nof/Tof] \times Nox \times Bitwidth \times Nky_{Next} / C_{oneBRAM} \rceil \times Tof \times 2 \quad (5-1)$$

其中 Nky_{Next} 表示该卷积层级联的下一层卷积层的卷积核宽。考虑到当前卷积层的维度展开，需要 $2 \times Tof$ 个独立的输出缓存，同时也要满足级联卷积层的需求，所以需要至少能计算出一行输出像素所需的存储空间。

对于中间层的输入时延计算过程，类似公式（4-13），将前一层输出行缓冲中的像素写入当前层输入缓存的时延 $Latency_{LoadIFM}$ 如下：

$$Latency_{LoadIFM} = (Const + Nky \times Nix) / Freq \quad (5-2)$$

中间层的输出时延计算，类似公式（4-19），从输出缓存搬移到输出行缓冲中的时延 $Latency_{Store}$ 如下：

$$Latency_{Store} = (Const + Nky_{Next} \times Nox) / Freq \quad (5-3)$$

由于片上各输出输出行缓冲彼此独立可并发读写（即同时有 Tif 个输出行缓冲进行读取， Tof 个输出行缓冲在进行写入），所以输入输出时延只需考虑单一行缓冲的读取或写入时延。同时，由于通过输出双缓冲设计实现层间并发，实际的层间传输时延由当前层输出时延和后一层的输入时延中较大的一项决定。

中间层的权重参数数量如果过大，则时延公式与公式(4-33)相同；如果参数量不大，能够全部保存在 FPGA 片上缓存中，此时权重读取时延 $Latency_{LoadW}$ 如下：

$$Latency_{LoadW} = (Const + Tif \times Tof \times Nkx \times Nky) / Freq \quad (5-4)$$

各层总时延的公式与公式 (4-40) 相同, 连续 k 层子图 i 的时延 $Latency_i$ 如下所示 (此处子图 i 不递归划分, 连续 k 层卷积层, 级联池化层看做卷积层的附加层, 未考虑重构时延):

$$Latency_i = \begin{cases} i \times \text{Max}(Latency_i^0, Latency_i^1, \dots, Latency_i^{k-1}), & \text{Cascade Layer} \\ \sum_{z=0}^{k-1} Latency_i^z, & \text{Reused Single Layer} \end{cases} \quad (5-5)$$

当子图 i 中各层时分复用单层加速器架构时, 子图总时延为各层的时延之和; 子图 i 使用层级联的加速器架构时, 子图时延由各层间的最大时延决定 (瓶颈所在), 因此在设计参数组合搜索时, 需尽量保证各层的时延相对接近, 才可能得到较低的时延。

此外, 通过批量数据分时复用同一子图架构来分摊重构时延, 分摊后的重构时延公式如下:

$$Latency_{BatchConfig} = Latency_{Reconfig} / N_{Batch\ Size} \quad (5-6)$$

其中, $Latency_{Reconfig}$ 表示重构整块 FPGA 的重构时延, $N_{Batch\ Size}$ 表示批量数据的大小, $Latency_{BatchConfig}$ 即分摊后的重构时延。($N_{Batch\ Size}$ 需要根据划分给 FPGA 与 CPU 的共享内存的大小、当前权重参数的所占内存、Linux 分页等因素综合考虑)。

5.2.3 子图再划分

按子图依赖顺序遍历子图, 对各子图根据时延模型、FPGA 配置时延、FPGA 资源数和片外带宽等约束, 寻找当前子图最小时延所对应的架构。

假设, 当前子图为具有连续 k 层的子图 i , 则需要考虑 4 种情况下的时延, 取其中最小的情况所对应的架构为当前子图的架构:

(1) 可复用前子图架构的情况下, 复用前子图架构: 不需考虑重构时延 $Latency_{BatchConfig}$, 对应时延为 $Latency_0$ 。

(2) 多层时分复用单层加速器架构: 考虑重构时延 $Latency_{BatchConfig}$, 对应时延为 $Latency_1$:

$$Latency_1 = Latency_{BatchConfig} + Latency_i \quad (5-7)$$

(3) 满足流式架构的约束条件下, 采用多层级联的加速器架构: 考虑重构时延 $Latency_{BatchConfig}$, 对应时延为 $Latency_2$:

$$Latency_2 = Latency_{BatchConfig} + Latency_i \quad (5-8)$$

采用多层级联的流式架构的子图需满足 FPGA 资源数 (DSP、LUT、BRAM 等) 和系统带宽 Bandwidth 等约束, 约束条件如下:

$$\sum_{l=0}^{k-1} N_{DSP}^l \leq N_{DSP}^{Total} \quad (5-9)$$

$$\sum_{l=0}^{k-1} N_{LUT}^l \leq N_{LUT}^{Total} \quad (5-10)$$

$$\sum_{l=0}^{k-1} N_{BRAM}^l \leq N_{BRAM}^{Total} \quad (5-11)$$

$$\sum_{l=0}^{k-1} Bandwidth_l \leq Bandwidth_{Total} \quad (5-12)$$

(4) 不满足流式架构的约束, 如果重构时延 $Latency_{BatchConfig}$ 小于前三种情况下的最小值 $\text{Min}(Latency_0, Latency_1, Latency_2)$, 考虑在子图内各处添加断点, 对子图递归地进行再划分。

可添加断点数不多的情况下, 考虑采用穷举遍历的方式, 遍历整个网络的所有子图的情况, 以寻找最优解; 否则, 可采用动态规划或贪心算法来寻找最优解或近优解。

最终得到的整个网络的总时延 $Latency$ ，公式如下所示：

$$Latency = \sum_{i=0}^{N_p} Latency_i \quad (5-13)$$

其中， N_p 为子图数，总时延为各子图时延之和。

5.3 实验评估

5.3.1 软硬件实验环境

CNN 模型：YOLOv2^[43]，模型所对应的数据集是 COCO 数据集^[2]。

FPGA 平台：Xilinx Zedboard 开发板(Dual-core ARM-A9+FPGA)，其中 FPGA 的 BRAM_18Kb、DSP48E、FF 和 LUT 资源数分别为 280、220、106400 和 53200。

使用 Vivado HLS 2018.2 HLS 进行加速器设计，Vivado v2018.2 综合和布局布线。

5.3.2 加速器的规模可伸缩性评估

考虑到加速器设计空间过大，此处将子图{0-16}与{26-27}合并，子图{17-24}与{29-30}合并，不复用前子图的加速器架构，且各子图内采用单层加速器时分复用的方式对各层进行处理。

对各子图的设计空间约束与 4.4.2 节类似，这里不重复叙述。此外，由公式 (5-6)、(5-7) 和 (5-8) 可知，各子图时延需考虑批量数据的大小 $N_{Batch\ Size}$ 。

批量数据的大小 $N_{Batch\ Size}$ 由可用共享内存大小决定。由于进行评估的 Zedboard 开发板仅有 512MB 内存，将高地址的 256MB 内存 (0x10000000-0x1FFFFFFF) 划分为共享内存供 CPU 与 FPGA 进行数据交互。其中，卷积权重参数放在 0x10000000-0x1C269FFF 范围的内存中 (使用动态定点 16 位量化后的权重大小约为 97.1MB)，即输入输出特征图可使用的内存为 0x1C26A000-0x1FFFFFFF (0x3D96000)。考虑最大池化层融合的情况，每张图像需要 0x3F6000 的存储空间，所以批量数据的大小 $N_{Batch\ Size}$ 不能超过 15 (0x3D96000/0x3F6000=15)，因此设 $N_{Batch\ Size}=15$ 。此时，分摊后的重构时延 $Latency_{BatchConfig}$ 为 17ms。

同时，考虑最大池化层融合的情况，对应最大池化层和卷积层的访存次数和数据量也会相应减少，公式如下：

$$N_{InMA}^{Pool} = \begin{cases} 0, Cascade\ Layer \\ \left\lceil \frac{N_{oy}}{T_{oy}} \right\rceil \times \left\lceil \frac{N_{ox}}{T_{ox}} \right\rceil \times \left\lceil \frac{N_{of}}{T_{of}} \right\rceil \times \left\lceil \frac{N_{if}}{T_{if}} \right\rceil \times T_{if} \times T_{iy} \times T_{ix}, Single\ Layer \end{cases} \quad (5-12)$$

$$N_{OutMA}^{Conv} = \begin{cases} 0, Cascade\ Layer \\ \left\lceil \frac{N_{oy}}{T_{oy}} \right\rceil \times \left\lceil \frac{N_{ox}}{T_{ox}} \right\rceil \times \left\lceil \frac{N_{of}}{T_{of}} \right\rceil \times T_{of} \times T_{oy} \times T_{ox}, Single\ Layer \end{cases} \quad (5-13)$$

如图 5-6 所示为对应的不同规模的可重构加速器的设计空间，花费约 8s 找到 156396 种满足约束加速器设计，所有找到的设计都处于性能受限区域。其中性能最优的设计为图中点 A (子图 1 [$T_{if}=4, T_{of}=44, T_{oy}=26, T_{ox}=52$], 子图 2 [$T_{if}=2, T_{of}=86, T_{oy}=42, T_{ox}=26$], 32.90 GOPS)，能效最优的设计为点 C (子图 1 [$T_{if}=1, T_{of}=86, T_{oy}=26, T_{ox}=26$], 子图 2 [$T_{if}=2, T_{of}=86, T_{oy}=38, T_{ox}=26$], 22.23 GOPS)，兼顾两者的设计为点 B (子图 1 [$T_{if}=2, T_{of}=86, T_{oy}=26, T_{ox}=36$], 子图 2 [$T_{if}=2, T_{of}=86, T_{oy}=32, T_{ox}=28$], 30.92 GOPS)。

与图 4-11 相比,除了在性能上有所提升外,由于采用卷积层融合级联的最大池化层的方式,访存数据量明显小于 4.4 中的设计,所以图 5-6 中对应的最大计算密度(横坐标)普遍大于前者;性能上,由于重构时延代价,实际性能与 4.4 的设计相比提升不大。

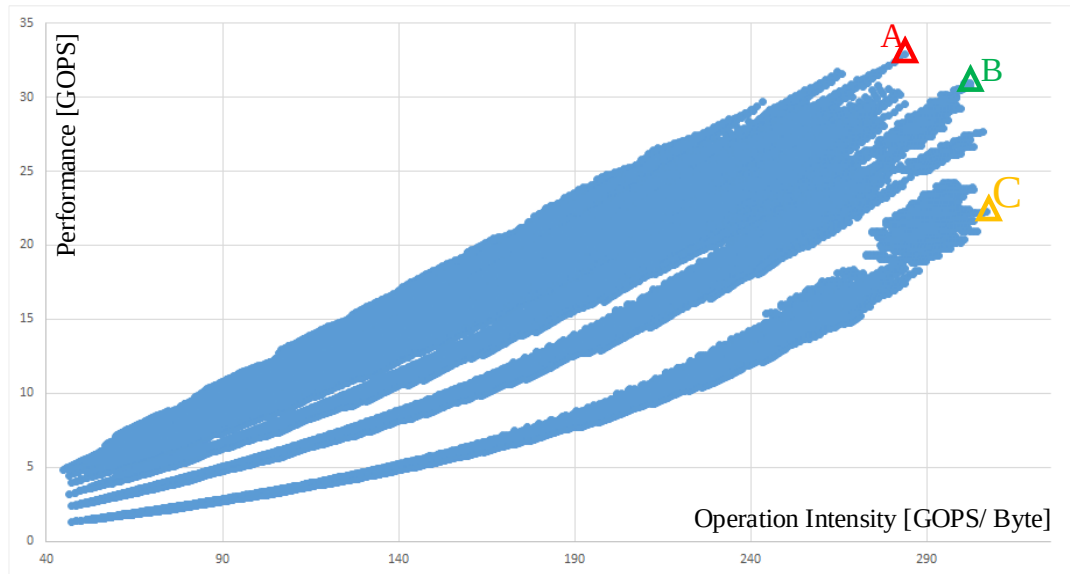


图 5-6 可重构加速器的设计空间探索

5.3.3 资源耗费评估

选择动态定点 16 位精度下的加速器架构,采用参数重排序、乒乓和多通道数据传输三种优化策略。其中,子图{0-16,26-27}的硬件设计参数为: $Tof_{max}=32$, $Tif_{max}=4$, $Toy_{max}=52$, $Tox_{max}=14$ (考虑到级联的最大池化层,故 Tox_{max} 为偶数);子图{17-24,29-30}的硬件设计参数为: $Tof_{max}=48$, $Tif_{max}=4$, $Toy_{max}=13$, $Tox_{max}=13$, $Nky_{max}=Nkx_{max}=3$, $S_{max}=2$, $N_{Cin}=4$, $N_{Cout}=2$, $Pof=Tof$, $Pif=Tif$, 其余参数均为 1。

结合给定参数和上文中对各模块资源耗费的公式,将估计与实际资源耗费对比,如表 5-1 所示:

表 5-1 FPGA 资源耗费

	DSP	BRAM_36Kb	LUT	FF	Freq.
Estimated-1	128(58%)	89(64%)	30976(58%)	-	150MHz
Actual-1	153(70%)	92(65%)	35831(67%)	35878(34%)	150MHz
Estimated-2	192(87%)	89(61%)	31104(58%)	-	150MHz
Actual-2	211(96%)	64(45%)	45418(85%)	42946(40%)	150MHz

对于子图 1,根据公式(4-24),DSP 耗费 128 个,实际设计中多出的 DSP 主要用于其他模块的设计。AXI Master 接口耗费 10 个 BRAM,偏置参数 bias 的缓存耗费 2 个 BRAM;再根据公式(4-41)、(4-42)、(4-9)、(4-30)和(4-16),加上输入缓存($3 \times 4 \times 2$)、输出缓存($2 \times 32 \times 2$)、片上输入行缓冲(4×2)、权重行缓冲(2)和输出行缓冲(2×2)所耗费的 BRAM 资源,共耗费 BRAM_18b 178 个,即 89 个 BRAM_36Kb,与实际 BRAM 耗费接近。关于 LUT 的部分与 4.4 中相同,在此不重复叙述。

对于子图 1, 根据公式(4-24), DSP 耗费 192 个, 实际设计中多出的 DSP 主要用于其他模块的设计。AXI Master 接口耗费 10 个 BRAM, 偏置参数 bias 的缓存耗费 2 个 BRAM; 再根据公式(4-41)、(4-42)、(4-9)、(4-30)和(4-16), 加上输入缓存(4×2)、输出缓存(48×2)、权重行缓冲(2)和输出行缓冲(2×2)所耗费的 BRAM 资源, 共耗费 BRAM_18b 122 个, 即 61 个 BRAM_36Kb, 与实际 BRAM 耗费接近。由于单个权重缓存的存储量过小, 无法充分利用 BRAM, 使用 LUT 和 FF 来实现权重缓存(此时, 单个权重缓存耗费约 3 个 LUT, 32 个 FF)。因此, 权重缓存共耗费 $3 \times Tof_{max} \times Tif_{max} \times 2 = 1152$ 个 LUT。由公式(4-25)可知, 此时 16 位定点精度的卷积模块耗费 29952 个 LUT。所以, 共耗费约 31104 个 LUT, 剩余 LUT 用于其他模块的设计。

5.3.4 性能评估

(1) 理论与实际时延评估

YOLOv2 模型包括 23 层卷积层、5 层池化层和 1 层重排序层。由于将级联的最大池化层与卷积层融合, 最终包括 25 层(4 层融合层、19 层卷积层、1 层池化层、1 层重排序层)。表 5-2 列出 25 层的评估结果(保留小数点后 3 位), 列中的 R (Real) 表示该列数据是实测值, E (Estimated) 指该列数据是根据上文公式得到的估计值。

以表中最后一行总延时为例, YOLOv2 模型的总时延误差约为 0.5%, 总计算时延误差约为 0.8%, 总输入时延误差约为 9.8%, 总输出时延误差约为 13.9%。本模型的主要误差仍集中于传输时延部分, 约为 10%。仍采用输出特征图复用模式, 总输出时延约为 99.504ms, 远小于总输入时延(634.379ms)。

表 5-2 实际与估计时延

Layer	Load Time R(ms)	Load Time E(ms)	Compute Time R(ms)	Compute Time E(ms)	Store Time R(ms)	Store Time E(ms)	Total Time R(ms)	Total Time E(ms)
Conv 0F	3.378	2.449	10.678	10.550	30.920	25.867	42.817	39.969
Conv 2F	15.545	10.118	42.155	42.022	15.467	12.934	50.979	49.786
Conv 4	16.563	10.793	44.933	44.810	8.181	6.898	44.322	47.196
Conv 5	12.469	7.465	5.163	5.045	4.111	3.449	12.321	9.347
Conv 6F	16.562	10.793	44.931	44.810	8.180	6.898	44.612	47.484
Conv 8	15.885	10.793	44.923	44.803	4.111	3.449	42.528	45.565
Conv 9	10.274	5.995	5.160	5.041	2.078	1.724	7.940	6.505
Conv 10F	15.877	10.793	44.922	44.803	4.110	3.449	42.671	45.709
Conv 12	21.934	21.586	44.991	44.872	2.970	2.618	42.389	45.366
Conv 13	9.807	7.525	5.225	5.110	1.507	1.309	9.552	7.729
Conv 14	21.935	21.586	44.991	44.872	2.968	2.618	42.388	45.366
Conv 15	9.813	7.525	5.225	5.110	1.505	1.309	9.556	7.729
Conv 16	21.934	21.586	44.990	44.872	2.968	2.618	42.389	45.366
Pool 17	1.048	0.749	0.635	0.581	1.070	1.123	1.074	1.138
Conv 18	43.237	42.248	29.078	28.749	1.105	1.136	42.908	42.524
Conv 19	9.821	6.498	3.688	3.364	0.573	0.568	6.941	6.563
Conv 20	43.243	42.248	29.078	28.749	1.105	1.136	42.910	42.524

Conv 21	9.819	6.498	3.688	3.364	0.574	0.568	6.940	6.563
Conv 22	43.239	42.248	29.078	28.749	1.105	1.136	42.910	42.524
Conv 23	86.440	84.496	58.093	57.491	1.104	1.136	85.504	84.772
Conv 24	86.433	84.496	58.100	57.491	1.105	1.136	85.506	84.772
Conv 25	2.486	1.881	1.347	1.277	0.409	0.327	2.551	2.055
Reorg 26	0.411	0.328	0.337	0.292	0.703	0.618	1.341	1.564
Conv 27	108.029	105.620	72.600	71.862	1.107	1.136	106.802	105.896
Conv 28	8.196	5.317	3.041	2.752	0.467	0.465	5.719	5.379
Total	634.379	571.633	677.048	671.443	99.504	85.626	865.570	869.389

对应表 5-2 的折线图，如图 5-7 和图 5-8 所示。理想情况下，图 5-7、图 5-8 中只能看到 4 条曲线（实际与估计时延曲线两两重合），即时延模型与实际时延完美匹配。同时，如果计算时延与总时延接近，证明当前加速器的设计是非常高效的（这说明当前的瓶颈是在于计算，访存和传输时延都已被掩盖）。

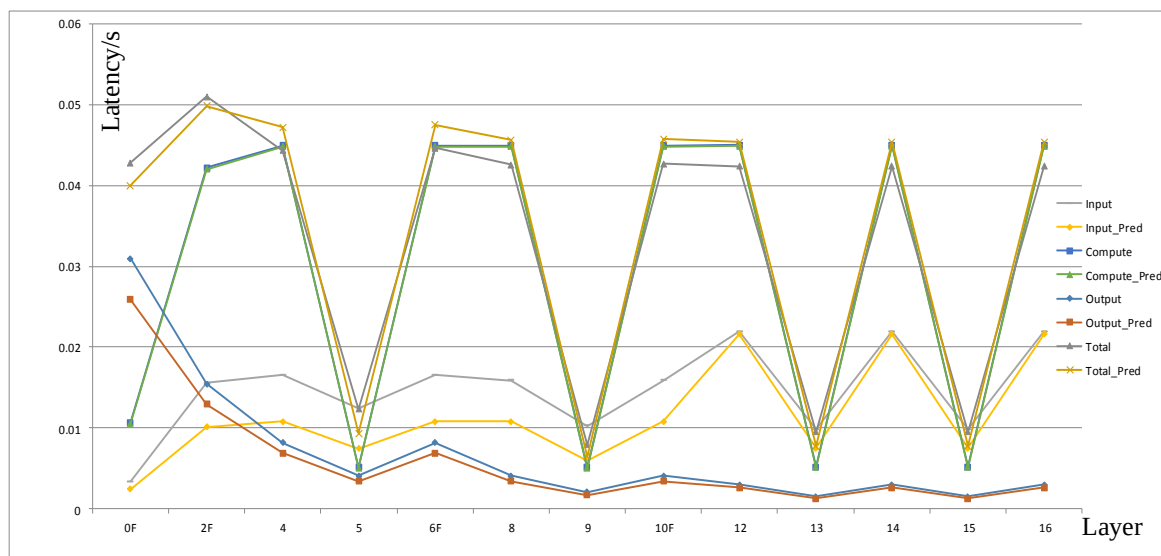


图 5-7 第 0-16 层的实际与估计时延

与图 4-13 相比，图 5-7 中级联层时延有明显降低，例如：图 4-13 中的第 0 层卷积层与第 1 层池化层的总时延为 82.119ms；而在图 5-7 中的对应层时延只有 42.817ms，这既受益于在 FPGA 片上级联卷积层与最大池化层，同时也有定制加速器架构的因素在内。此外，由于卷积层与最大池化层的级联，减少了访存数据量，图 5-7 中的前几层的访存时延误差与图 4-13 相比有了很大的改善。

与图 4-14 相比，图 5-8 中的 17-24 层由于定制加速器架构，大大地提高了计算单元的动态利用率，并且由于输出特征图规模一致（ 13×13 ），有效地利用了片上缓存空间。并且由于仍采用输出特征图复用模式，所以图 5-8 中的输出时延与其他时延相比，基本可以忽略。

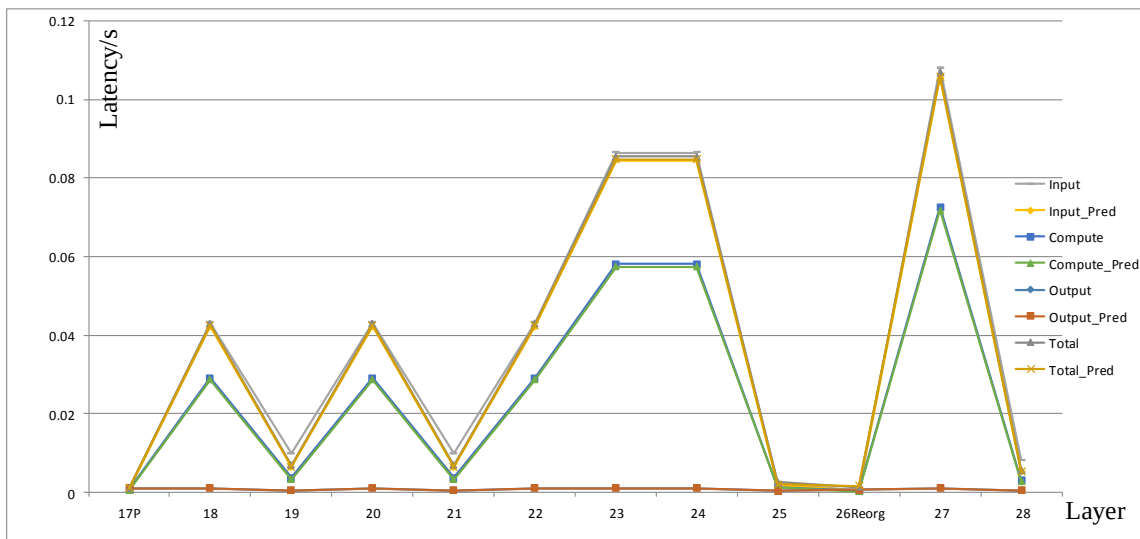


图 5-8 第 17-30 层的实际与估计时延

(2) 性能对比

如表 5-3 所示，与 4.4 节中基于 SIMD 架构二维展开的单层加速器相比，基于 FPGA 动态可重构的加速器，通过处理批量数据的方式，有效地分摊了重构时延，使得吞吐率得到了极大的提升；然而，就处理每张图片的时延来说，由于重构时延过于高昂，仍是 4.4 中的单层加速器架构占有优势。所以，在实际应用中，应具体问题具体分析，结合当前的应用场合探索并设计出合适的加速器架构。

表 5-3 与 4.4 节的工作比较

	4.4	Current
CNN model	YOLOv2	YOLOv2
FPGA Board	Zedboard	Zedboard
Clock (MHz)	150	150
Precision	Fixed-16	Fixed-16
DSP (Used/Total)	153/220	153/220 211/220
Operations (GOP)	29.47	29.47
Latency(s/per img)	0.98	1.12
Throughout (GOP/s)	30.15	33.39

5.4 本章小结

首先，介绍了基于 FPGA 动态可重构加速器与典型加速器的异同，给出了设计动态可重构加速器的整体流程。以 YOLOv2 为例进行可重构加速器的设计与评估，使用批量数据分时复用各子图的加速器架构的方式来分摊重构时延，通过将级联的最大池化层与卷积层融合减少访存时延，有效地提高了基于 FPGA 动态可重构的神经网络加速器的性能。

主要结论与展望

主要结论

近年来,以深度神经网络为代表的深度学习算法在许多计算机视觉任务上取得了巨大突破,如图像分类、目标检测、画质增强等应用。由于 FPGA 低功耗、低延时的特性,使其非常适用于一些对功耗有严格限制的小批量流式应用场合。此外,通过配置改变 FPGA 硬件逻辑,对具体应用定制硬件,非常适合深度学习这种日新月异、不断改变的场合。因此, FPGA 成为了一种极具潜力的选择。

目前,大规模使用 FPGA 加速深度学习的制约主要有两点:

(1) 开发效率低。然而,随着高层次综合技术不断成熟,已经可以使用 C、C++ 等高级语言进行 FPGA 的算法描述,开发效率低的问题正被逐渐缓解。

(2) 规模可伸缩性差。受制于芯片资源,对于复杂算法往往需要升级芯片或进行多片划分,没有办法灵活地部署到不同硬件上去。

因此本文立足于研究基于 FPGA 的神经网络加速器的规模可伸缩性。通过研究现有的基于 FPGA 的神经网络加速器的特点和 FPGA 本身特有的可重构特性,探索如何将针对特定神经网络算法的加速器灵活地部署到不同规模的 FPGA 上去。

研究并设计了一种具有可伸缩性的基于 FPGA 的 SIMD 二维部分展开的 CNN 加速器架构。通过对所设计的加速器深入分析和建模,使得加速器性能与资源耗费模型与实际加速器基本一致。同时,通过改变加速器的硬件设计参数来调整加速器规模,以达到规模可伸缩性的目的。此外,结合 FPGA 的动态可重构特性,探索基于 FPGA 的动态可重构加速器的设计流程。本文的主要工作如下:

(1) 基于 FPGA 的规模可伸缩性的研究

根据现有研究,对 SIMD 架构的神经网络加速器进行研究,分析三种循环优化策略对应的硬件设计参数以及数据复用模式对规模可伸缩性的影响,并设计一种具有规模可伸缩性的 CNN 加速器进行评估。三种循环优化策略分别为:循环展开、循环分块与循环交换。循环展开,通过对卷积循环各维展开来设计对应的并行乘加单元以提高性能,给出了 4 种最基本的循环展开以及对应的资源耗费。循环分块,考虑卷积计算的数据局部性,通过对卷积循环分块来使得每次只需从片外读取少量输入特征图像素块和卷积核权重,复用多次后,再读取其他部分,极大地减少了访存数据量和访存时延。循环交换,通过交换卷积循环各维顺序来改变实际数据流,其中有 3 种主要的循环顺序以达到数据复用的目的:输入特征图复用、输出特征图复用和卷积核复用。给出了三种数据复用模式的对应伪代码以及硬件架构图,并构建了对应数据模式下的简单时延模型。

(2) 基于 FPGA 的 SIMD 二维部分展开的 CNN 加速器设计与评估

使用高层次综合设计并实现了一种在输入和输出特征图数二维部分展开的输出特征图复用的 SIMD 架构的 CNN 加速器,并采用了参数重排序、多通道数据传输等优化策略来减少访存与传输时延。以目前在业界被广泛应用的 YOLOv2 目标检测算法[1]为例,叙述将 CNN 模型映射到 FPGA 上的完整流程。同时,对加速器的性能和所需资源

进行深入分析和建模，将实际传输延时考虑在内，极大地缩小了加速器理论时延与实际时延的误差。对由不同硬件设计参数与数据复用模式产生的不同规模的 CNN 加速器进行评估与对比。相关代码已在 github 开源。

（3）改进基于 FPGA 的 SIMD 二维展开的 CNN 加速器架构中的输入和输出模块

改进了基于 FPGA 的 SIMD 二维展开的 CNN 加速器架构中的输入和输出模块，使得输入输出特征图的像素块能够以较大的突发长度进行传输，并对输入输出模块进行了双缓冲设计，使得片上偏移和访存地址的计算、片上缓存的传输时延与片外存储的访存时延能够重叠，有效提高了总线带宽的利用率，同时降低了传输与访存时延。

（4）基于 FPGA 动态可重构的 CNN 加速器的设计与评估

分析基于 FPGA 动态可重构技术的 CNN 加速器与典型加速器的异同，给出了设计基于 FPGA 动态可重构的 CNN 加速器的整体流程。以 YOLOv2 为例进行加速器的设计与评估，使用批量数据分时复用各子图的加速器架构的方式来分摊重构时延，通过将级联的最大池化层与卷积层融合减少访存时延，有效地提高了加速器性能。

展望

本课题针对基于 FPGA 的神经网络加速器的规模可伸缩性进行了研究与评估，虽然取得了一定的成果，但是由于研究时间与本人能力的限制，仍有许多可以进一步研究的地方：

（1）结合 3 种不同的数据复用模式，设计新的加速器架构。能够根据不同层的维度“形状”，配置对应的时延最小的数据复用模式。

（2）多 FPGA 构建大规模协同的 CNN 加速器。实际是从集群和数据中心的角度来思考规模可伸缩性的问题，此时需要考虑 FPGA 间的连接拓扑对 FPGA 间传输时延的影响，以及将网络合理划分到各 FPGA 上的相应算法。

（3）提高当前加速器性能：采用 Winograd 快速卷积乘法加速卷积计算；另外，也可以考虑采用 8 位及以下的数据精度来减少单位计算单元的资源耗费。

（4）使用部分可重构技术，只对部分 FPGA 区域重构以减少配置时延代价。

（5）考虑类似寒武纪的编译优化思路，构建类似通用处理器的专用加速器。给定网络后，对网络结构进行分析并生成对应指令流，其余思路与通用处理器类似。

致 谢

时光飞逝，岁月如梭，三年研究生生涯即将画上句号。说起来，这已经是我在江南大学的第七年，回想本科四年的懵懂无知，研究生三年的经历真的是非常值得珍惜。有过遗憾，有过欢乐，同时感受最深的还是研究生阶段训练出的研究素养。

首先，我要感谢柴志雷老师，在这三年的研究生生涯中，遇到了很多的困难，但大部分都在于柴老师的讨论中得到了解决。其次，柴老师对研究生的培养，完全是按照博士生的培养方式，个人研究素养的培养使得之后无论从事什么行业都能够迅速上手，有所突破。

另外，我要感谢实验室的师兄师姐，李申师兄、仇越师姐还有早已毕业多年的王芝斌师兄、柴镇师兄和沈镇师兄。正是有各位师兄师姐在我初来实验室时给予的帮助，我才能够研究的道路上奋力前进。还需要感谢微软亚洲研究院的张宸同学，关于高层次综合和突发传输向他请教了很多的问题。

感谢我的室友，涂序文与武帅，虽然我每天很晚回宿舍，但是大家都比较晚睡觉，没有打扰到谁。感谢夏珺、薛伟、王田辰和宗海燕等同学三年研究生生涯的陪伴与相互扶持，没有你们的鼓励，我应该也没有信心能完成研究。

研究生三年亦苦亦甜、如切如磋、如琢如磨，如人饮水冷暖自知。

参考文献

- [1] Deng J, Dong W, Socher R, et al. Imagenet: A large-scale hierarchical image database[C]//2009 IEEE conference on computer vision and pattern recognition(CVPR). Piscataway: IEEE, 2009: 248-255.
- [2] Lin T Y, Maire M, Belongie S, et al. Microsoft coco: Common objects in context[C]. 2014 European conference on computer vision(ECCV). Cham: Springer, 2014: 740-755.
- [3] Redmon J, Farhadi A. YOLO9000: better, faster, stronger[C]//Proceedings of the 2017 IEEE conference on computer vision and pattern recognition(CVPR). Piscataway: IEEE, 2017: 7263-7271.
- [4] Dong C, Loy C C, He K, et al. Image super-resolution using deep convolutional networks[J]. IEEE transactions on pattern analysis and machine intelligence, 2016, 38(2): 295-307.
- [5] Nurvitadhi E, Sheffield D, Sim J, et al. Accelerating binarized neural networks: Comparison of FPGA, CPU, GPU, and ASIC[C]. 2016 International Conference on Field-Programmable Technology (FPT). Piscataway: IEEE, 2016: 77-84.
- [6] Nurvitadhi E, Sim J, Sheffield D, et al. Accelerating recurrent neural networks in analytics servers: Comparison of FPGA, CPU, GPU, and ASIC[C]. 2016 26th International Conference on Field Programmable Logic and Applications (FPL). Piscataway: IEEE, 2016: 1-4.
- [7] Nurvitadhi E, Venkatesh G, Sim J, et al. Can FPGAs beat GPUs in accelerating next-generation deep neural networks?[C]. Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2017: 5-14.
- [8] Ma Y, Zheng T, Cao Y, et al. Algorithm-hardware co-design of single shot detector for fast object detection on FPGAs[C]. 2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD). Piscataway: IEEE, 2018: 1-8.
- [9] Yu J, Guo K, Hu Y, et al. Real-time object detection towards high power efficiency[C]. 2018 Design, Automation & Test in Europe Conference & Exhibition (DATE). Piscataway: IEEE, 2018: 704-708.
- [10] 王玉伟.深入理解 CPU 和异构计算芯片 GPU/FPGA/ASIC (下) GPU/FPGA/ASIC[EB/OL]. <https://cloud.tencent.com/developer/article/1004746>, 2017-06-19
- [11] Cong J, Fang Z, Lo M, et al. Understanding performance differences of FPGAs and GPUs[C]. 2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). Piscataway: IEEE, 2018: 93-96.
- [12] Chung E, Fowers J, Ovtcharov K, et al. Serving DNNs in real time at datacenter scale with project brainwave[J]. IEEE Micro, 2018, 38(2): 8-20.
- [13] Ouyang J. XPU: A programmable FPGA accelerator for diverse workloads[DB/OL]. https://www.hotchips.org/wp-content/uploads/hc_archives/hc29/HC29.21-Monday-Pub/HC29.21.40-Processors-Pub/HC29.21.410-XPU-FPGA-Ouyang-Baidu.pdf, 2019-4-13
- [14] Cong J, Liu B, Neuendorffer S, et al. High-level synthesis for FPGAs: From prototyping to deployment[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2011, 30(4): 473-491.
- [15] Nane R, Sima V M, Pilato C, et al. A survey and evaluation of FPGA high-level synthesis tools[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2016, 35(10): 1591-1604.
- [16] Lu L, Liang Y, Xiao Q, et al. Evaluating fast algorithms for convolutional neural networks on FPGAs[C]. 2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). Piscataway: IEEE, 2017: 101-108.
- [17] Zhang C, Prasanna V. Frequency domain acceleration of convolutional neural networks on CPU-FPGA shared memory system[C]. Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2017: 35-44.

- [18]Li H, Fan X, Jiao L, et al. A high performance FPGA-based accelerator for large-scale convolutional neural networks[C]. 2016 26th International Conference on Field Programmable Logic and Applications (FPL). Piscataway: IEEE, 2016: 1-9.
- [19]Guan Y, Liang H, Xu N, et al. FP-DNN: An automated framework for mapping deep neural networks onto FPGAs with RTL-HLS hybrid templates[C]. 2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). Piscataway: IEEE, 2017: 152-159.
- [20]Nakahara H, Fujii T, Sato S. A fully connected layer elimination for a binarized convolutional neural network on an FPGA[C]. 2017 27th International Conference on Field Programmable Logic and Applications (FPL). Piscataway: IEEE, 2017: 1-4.
- [21]Prost-Boucle A, Bourge A, Párot F, et al. Scalable high-performance architecture for convolutional ternary neural networks on FPGA[C]. 2017 27th International Conference on Field Programmable Logic and Applications (FPL). Piscataway: IEEE, 2017: 1-7.
- [22]Wang J, Lou Q, Zhang X, et al. Design flow of accelerating hybrid extremely low bit-width neural network in embedded fpga[C]. 2018 28th International Conference on Field Programmable Logic and Applications (FPL). Piscataway: IEEE, 2018: 163-1636.
- [23]Guo K, Sui L, Qiu J, et al. From model to FPGA: Software-hardware co-design for efficient neural network acceleration[C]. 2016 IEEE Hot Chips 28 Symposium (HCS). Piscataway: IEEE, 2016: 1-27.
- [24]Parashar A, Rhu M, Mukkara A, et al. Scnn: An accelerator for compressed-sparse convolutional neural networks[C]. 2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA). New York: IEEE, 2017: 27-40.
- [25]Han S, Liu X, Mao H, et al. EIE: efficient inference engine on compressed deep neural network[C]. 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA). New York: IEEE, 2016: 243-254.
- [26]Han S, Kang J, Mao H, et al. ESE: Efficient speech recognition engine with sparse lstm on fpga[C]. Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2017: 75-84.
- [27]Ma Y, Cao Y, Vrudhula S, et al. An automatic RTL compiler for high-throughput FPGA implementation of diverse deep convolutional neural networks[C]. 2017 27th International Conference on Field Programmable Logic and Applications (FPL). Piscataway: IEEE, 2017: 1-8.
- [28]Zhang C, Sun G, Fang Z, et al. Caffeine: Towards uniformed representation and acceleration for deep convolutional neural networks[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018.
- [29]Qiu J, Wang J, Yao S, et al. Going deeper with embedded fpga platform for convolutional neural network[C]. Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2016: 26-35.
- [30]Venieris S I, Bouganis C S. fpgaConvNet: A framework for mapping convolutional neural networks on FPGAs[C]. 2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). Piscataway: IEEE, 2016: 40-47.
- [31]Venieris S I, Bouganis C S. Latency-driven design for FPGA-based convolutional neural networks[C]. 2017 27th International Conference on Field Programmable Logic and Applications (FPL). Piscataway: IEEE, 2017: 1-8.
- [32]Kästner F, Janßen B, Kautz F, et al. Hardware/Software Codesign for Convolutional Neural Networks Exploiting Dynamic Partial Reconfiguration on PYNQ[C]. 2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). Piscataway: IEEE, 2018: 154-161.
- [33]Zhang C, Li P, Sun G, et al. Optimizing fpga-based accelerator design for deep convolutional neural networks[C]. Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2015: 161-170.

- [34]Rahman A, Oh S, Lee J, et al. Design space exploration of FPGA accelerators for convolutional neural networks[C]. 2017 Proceedings of the Conference on Design, Automation & Test in Europe(DATE). Piscataway: IEEE, 2017: 1147-1152.
- [35]Ma Y, Cao Y, Vrudhula S, et al. Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks[C]. Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2017: 45-54.
- [36]Wei X, Yu C H, Zhang P, et al. Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs[C]. Proceedings of the 54th Annual Design Automation Conference 2017(DAC). New York: ACM, 2017: 29.
- [37]Wei X, Liang Y, Li X, et al. TGPA: tile-grained pipeline architecture for low latency CNN inference[C]. 2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD). New York: ACM, 2018: 58.
- [38]Shi F, Li H, Gao Y, et al. Sparse Winograd Convolutional neural networks on small-scale systolic arrays[C]. Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2019: 118-118.
- [39]Zhang X, Wang J, Zhu C, et al. Dnnbuilder: an automated tool for building high-performance dnn hardware accelerators for fpgas[C]. Proceedings of the International Conference on Computer-Aided Design. New York: ACM, 2018: 56.
- [40]Suzuki K. Artificial neural networks-Methodological advances and biomedical applications[M]. London: InTech, 2011. 5-6
- [41]Convolutional neural network From Wikipedia, the free encyclopedia[DB/OL]. https://en.wikipedia.org/wiki/Convolutional_neural_network, 2019-03-26
- [42]Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]. Proceedings of the IEEE conference on computer vision and pattern recognition. Piscataway: IEEE, 2016: 779-788.
- [43]Darknet: an open source neural network framework[DB/OL]. <https://github.com/pjreddie/darknet/>, 2019-03-26
- [44]Ioffe S, Szegedy C. Batch normalization: accelerating deep network training by reducing internal covariate shift[C]. Proceedings of the 32nd International Conference on Machine Learning. Brookline, MA: JMLR, 2015: 448-456.
- [45]UG998: Introduction to FPGA Design with Vivado High-Level Synthesis[DB/OL]. https://www.xilinx.com/support/documentation/sw_manuals/ug998-vivado-intro-fpga-design-hls.pdf , 2019-03-26
- [46]UG909: Vivado Design Suite User Guide Partial Reconfiguration[DB/OL]. https://www.xilinx.com/support/documentation/sw_manuals/xilinx2018_3/ug909-vivado-partial-reconfiguration.pdf, 2019-03-26
- [47]UG585: Zynq-7000 SoC Technical Reference Manual[DB/OL]. https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf, 2019-03-26
- [48]Li J, Yan G, Lu W, et al. SmartShuttle: Optimizing off-chip memory accesses for deep learning accelerators[C]. 2018 Design, Automation & Test in Europe Conference & Exhibition (DATE). Piscataway: IEEE, 2018: 343-348.
- [49]Tu F, Yin S, Ouyang P, et al. Deep convolutional neural network architecture with reconfigurable computation patterns[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2017, 25(8): 2220-2233.
- [50]Motamedi M, Gysel P, Ghiasi S. PLACID: a platform for FPGA-based accelerator creation for DCNNs[J]. ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM), 2017, 13(4): 62.

- [51]Ma Y, Cao Y, Vrudhula S, et al. Optimizing the convolution operation to accelerate deep neural networks on FPGA[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2018, 26(7): 1354-1367.
- [52]Shen Y, Ferdman M, Milder P. Maximizing CNN accelerator efficiency through resource partitioning[C]. Proc. of the 44th Annual International Symposium on Computer Architecture (ISCA). New York: ACM, 2017. 535-547.
- [53]戴维 A. 帕特森, 约翰 L. 亨尼斯主编. 计算机组成与设计 硬件/软件接口[M]. 第 6 版. 王党辉等译. 北京: 机械工业出版社, 2018. 368-376
- [54]吴艳霞, 梁楷, 刘颖等. 深度学习 FPGA 加速器的进展与趋势[J]. 计算机学报, 2018, 41(1): 1-21.
- [55]陆维娜, 胡瑜, 叶靖等. 面向卷积神经网络加速器吞吐量优化的 FPGA 自动化设计方法[J]. 计算机辅助设计与图形学学报, 2018, 30(11): 2164-2173.
- [56]Shan L, Zhang M, Deng L, et al. A dynamic multi-precision fixed-point data quantization strategy for convolutional neural network[C]. In: Xu W, Xiao L, Li J, Zhang C, Zhu Z, eds. Proc. of the 20th CCF National Conference on Computer Engineering and Technology (NCCET). Singapore: Springer, 2016. 102-111.
- [57]Wai Y J, bin Mohd Yussof Z, bin Salim S I, et al. Fixed Point Implementation of Tiny-Yolo-v2 using OpenCL on FPGA[J]. International Journal of Advanced Computer Science and Applications, 2018, 9(10): 506-512.
- [58]张雲轲, 刘丹. 基于小型 Zynq SoC 硬件加速的改进 TINY YOLO 实时车辆检测算法实现[J]. 计算机应用, 2019, 39(1): 192-198.
- [59]Nakahara H, Yonekawa H, Fujii T, et al. A lightweight yolov2: A binarized cnn with a parallel support vector regression for an fpga[C]. Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2018: 31-40.
- [60]Venieris S I, Bouganis C S. fpgaConvNet: Mapping Regular and Irregular Convolutional Neural Networks on FPGAs[J]. IEEE transactions on neural networks and learning systems, 2018 (99): 1-17.

附录：作者在攻读硕士学位期间发表的论文、成果与科研项目

发表论文情况：

- [1] **Chen C**, Xia J, Yang W, et al. A PYNQ-compliant Online Platform for Zynq-based DNN Developers[C]. Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: ACM, 2019: 185-185. (CCF B 类会议, 已发表)
- [2] 陈辰, 柴志雷, 夏珺. 基于 Zynq 7000 FPGA 异构平台的 YOLOv2 加速器设计与实现[J]. 计算机科学与探索, 2019: 1-19. (CSCD 核心, 已发表)
- [3] 陈辰, 严伟, 夏珺, 柴志雷. 基于 FPGA 的深度学习目标检测系统的设计与实现[J]. 电子技术应用. (已录用)
- [4] **Chen Chen**, Jun Xia and Zhilei Chai. Exploring the Quick Development Scheme for FPGA-based CNN Accelerator with Feasible Performance[C]. International Symposium on Advanced Parallel Processing Technology 2019. (EI 会议, 审稿中)

获奖情况：

“基于 FPGA 的画质增强系统”获第九届全国大学生服务外包创新创业大赛一等奖（A 类竞赛，排名 2）

“基于 FPGA 的目标检测系统”获 2018 年第二届全国大学生 FPGA 创新设计邀请赛二等奖（B 类竞赛，排名 1）

参加科研项目：

国家重点研发计划专项（2016YFC0801001）：基于监控内容感知的高效安全视频编码芯片设计

GitHub 开源项目：

https://github.com/dhm2013724/yolov2_xilinx_fpga